UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE DELL'INFORMAZIONE

Corso di Laurea Magistrale in Ingegneria Informatica

ELABORATO DI ARCHITETTURA DEI SISTEMI DIGITALI

Prof. Nicola Mazzocca - Prof.ssa Alessandra De Benedictis

a.a. 2024-25

Studenti:

Russo Paola M63001828

Vitale Luciano M63001774

Gruppo 30

Sommario

Capitolo 1 - Reti combinatorie elementari 5
Esercizio 1: Multiplexer 16:1 5
Esercizio 1.1 5
Progetto e architettura 5
Implementazione 7
Simulazione 10
Esercizio 1.2 12
Progetto e architettura 12
Implementazione 13
Simulazione 14
Esercizio 1.3 16
Sintesi su board di sviluppo 16
Esercizio 2: Sistema ROM + M 18
Esercizio 2.1 18
Progetto e architettura 18
Implementazione 19
Simulazione 21
Esercizio 2.2 23
Sintesi su board di sviluppo 23
Capitolo 2: Reti sequenziali elementari 24
Esercizio 3: Riconoscitore di sequenza 24
Esercizio 3.1 24
Progetto e architettura 24
Implementazione 25
Simulazione 27
Esercizio 3.2 30
Sintesi su board di sviluppo 30
Timing analysis 32
Esercizio 4: Shift register 34
Esercizio 4.1 34
Progetto e architettura 34
Implementazione 35
Simulazione 40
Esercizio 5: Cronometro 44

Esercizio 5.1 44
Progetto e architettura 44
Implementazione 46
Simulazione 50
Esercizio 5.2 54
Sintesi su board di sviluppo 54
Esercizio 6: Sistema di lettura-elaborazione-scrittura PO_PC 65
Esercizio 6.1 65
Progetto e architettura 65
Implementazione 67
Simulazione 72
Esercizio 6.2 74
Sintesi su board di sviluppo 74
Timing analysis 77
Capitolo 3: Macchine Aritmetiche 78
Esercizio 7 – Moltiplicatore di Booth 78
Esercizio 7.1 78
Progetto e architettura 78
Implementazione 80
Simulazione 86
Esercizio 7.2 89
Sintesi su board di sviluppo 89
Capitolo 4: Handshaking 92
Esercizio 8 – comunicazione con handshaking 92
Esercizio 8.1 92
Progetto e architettura 92
Implementazione 94
Simulazione 105
Capitolo 5: Processore 107
Esercizio 9 110
Analisi BIPUSH 110
Analisi SWAP 112
Modifica del codice operativo 113
Capitolo 6: Interfaccia Seriale 115
Esercizio 10 115

Progetto e architettura 115
Implementazione 117
Simulazione 127
Capitolo 7: Switch Multistadio 129
Esercizio 11 – switch multistadio 129
Esercizio 11.1 129
Progetto e architettura 129
Implementazione 130
Simulazione 135
Capitolo 8: Prova Dicembre 2024 137
Esercizio 12 (prova di esame del 19 dicembre 2024) 137
Progetto e architettura 137
Implementazione 140
Simulazione 151
Appendice 155
Multiplexer 2:1 155
Progetto e architettura 155
Implementazione 156
Demultiplexer 1:2 156
Progetto e architettura 156
Implementazione 157
Contatore modulo N 158
Progetto e architettura 158
Implementazione 159
ROM 160
Progetto e architettura 160
Implementazione 161
Registro 162
Progetto e architettura 162
Implementazione 163

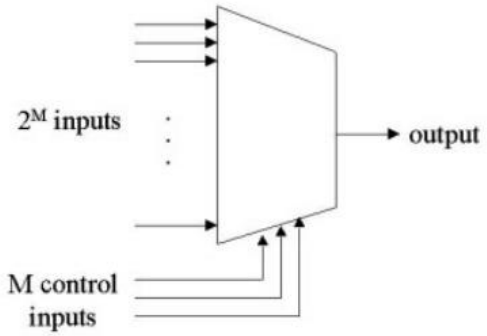
Capitolo 1 - Reti combinatorie elementari

Esercizio 1: Multiplexer 16:1

Il seguente esercizio, strutturato in più punti, riguarda la progettazione e l'implementazione di reti combinatorie per l'instradamento dei segnali e la connessione tra sistemi. La macchina principe che realizza tali funzionalità è il multiplexer.

Nella versione indirizzabile, presenta in ingresso M linee di selezione e $N = 2^M$ linee dato ed in uscita un singolo segnale. Il dispositivo agisce collegando una ed una sola linea di ingresso all'unica linea di uscita. La selezione avviene mediante una stringa in ingresso, rappresentante un numero codificato in rappresentazione binaria posizionale, che identifica una specifica linea.

Di seguito ne mostriamo l'interfaccia nella versione più generale:



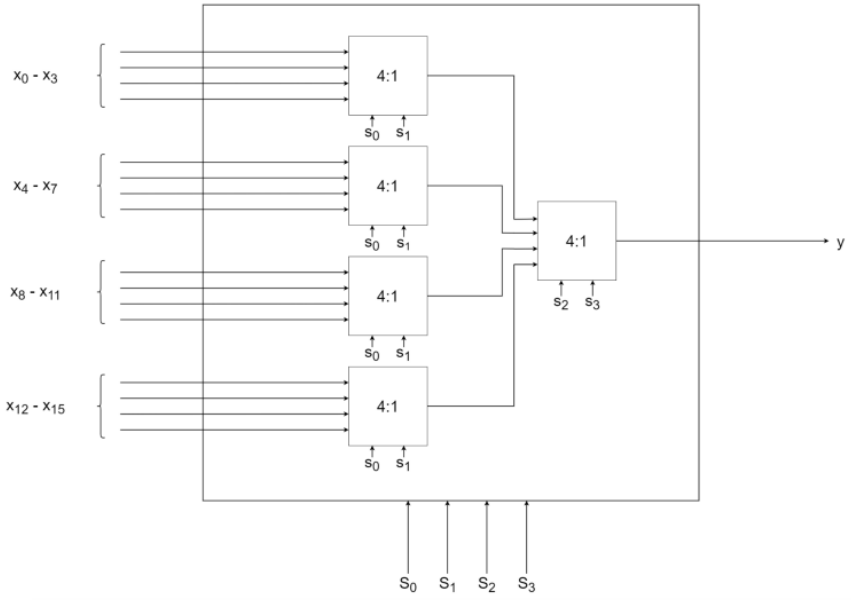
Esercizio 1.1

Progettare, implementare in VHDL e testare mediante simulazione un **multiplexer indirizzabile 16:1**, utilizzando un approccio di progettazione per composizione a partire da **multiplexer 4:1.**

Progetto e architettura

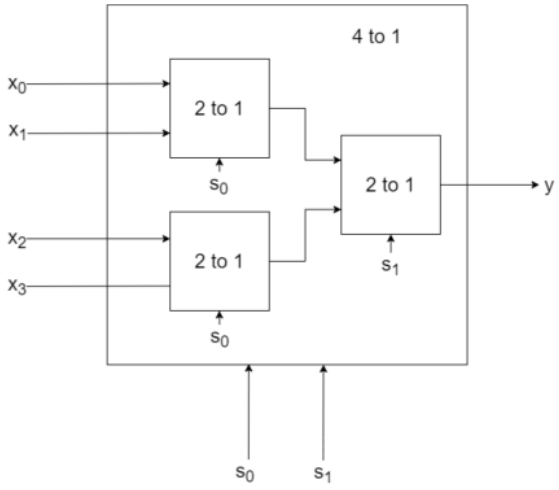
Il multiplexer 16:1 presenta 16 ingressi, $\log_2 16$ ingressi di selezione ed un'unica uscita. Per progettarlo abbiamo usato un approccio top down: siamo partiti dall'interfaccia del componente e, successivamente, lo abbiamo modellato mediante composizione ad albero di MUX 4:1.

Il procedimento di composizione si articola su più livelli ed è il seguente: si collegano gli ingressi del MUX 16:1 agli ingressi di più MUX 4:1 e si collega l'uscita finale all'uscita di un MUX 4:1. Infine, si collegano le uscite dei MUX del primo livello agli ingressi del MUX del secondo livello.



Per consentire un corretto instradamento dei segnali è necessario collegare opportunamente gli ingressi di selezione: man mano che si avanza di livello si utilizzano i bit più significativi del vettore di bit rappresentante gli ingressi di selezione per selezionare le uscite dei MUX dei livelli precedenti.

Il passo successivo è stato quello di modellare il MUX 4:1 mediante composizione modulare di più MUX 2:1 disposti ad albero. Il procedimento che abbiamo seguito è analogo al precedente.

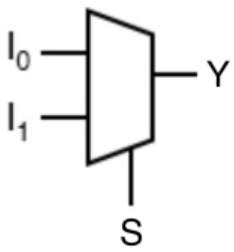


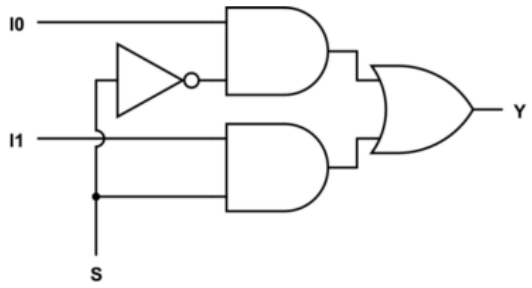
Infine, abbiamo realizzato il MUX 2:1: il dispositivo più adatto a decidere quale tra due diversi segnali convogliare verso un'unica uscita. Per descriverlo siamo partiti direttamente dalla fase

di sintesi, servendoci dell'espressione booleana che ne descrive il funzionamento: L'uscita Y è infatti pari all'ingresso I_0 quando la linea di selezione è bassa, è invece pari ad I_1 quando è alta.

$$Y = (I_0 \cdot \overline{S}) + (I_1 \cdot S)$$

Di seguito mostriamo l'interfaccia del componente e il rispettivo disegno architetturale.





Abbiamo scelto un approccio mediante composizione poiché in tal modo abbiamo potuto costruire una "libreria" di componenti da riutilizzare in progetti futuri.

Implementazione

Per quanto riguarda l'implementazione in VHDL abbiamo seguito un approccio bottom up nella realizzazione dei 3 componenti: siamo partiti dal MUX 2:1 fino ad arrivare al MUX 16:1.

Per il MUX 2:1 avente due segnali in ingresso, una sola linea di selezione ed un'unica uscita abbiamo definito la seguente interfaccia:

```
entity mux_2_1 is
  Port (
  a0 : in std_logic;
  a1 : in std_logic;
  s : in std_logic;
  y1 : out std_logic
  );
end mux_2_1;
```

Abbiamo poi scelto, per la descrizione del comportamento, un approccio dataflow partendo dall'espressione booleana prima descritta.

```
architecture dataflow of mux_2_1 is

begin
    y1 <= (a0 AND (NOT s)) OR (a1 AND s);

end dataflow;
```

Specifichiamo che l'implementazione su FPGA non sarà realizzata a partire da combinazione di porte logiche ma tramite *look-up table* in cui sarà memorizzata la tabella di verità che regola il comportamento di questo oggetto.

Per il MUX 4:1 avente 4 segnali di ingresso, 2 linee di selezione ed un'unica uscita abbiamo definito invece l'interfaccia:

```
entity mux_4_1 is
  Port (
    b : in std_logic_vector (0 to 3);
    s0 : in std_logic;
    s1 : in std_logic;
    y0 : out std_logic
   );
end mux_4_1;
```

Dove l'ingresso per comodità è stato definito come un vettore di 4 bit.

Per realizzarlo come composizione di 3 MUX 2:1 abbiamo scelto una descrizione del comportamento di tipo structural.

```
architecture structural of mux_4_1 is
    signal u : std_logic_vector (0 to 1);
    component mux_2_1 is
        port (
            a0 : in std_logic;
            a1 : in std_logic;
            s : in std_logic;
            y1 : out std_logic
        );
    end component;
```

```
begin
       mux0_1 : FOR i in 0 to 1 GENERATE m : mux_2_1
        port map (
            a0 => b(i*2),
            a1 => b(i*2 +1),
            s => s0,
            y1 => u(i)
        );
        end GENERATE;
        mux2 : mux_2_1
        port map (
            a0 => u(0),
            a1 => u(1),
            s => s1,
            y1=>y0
        );

end structural;
```

All'interno dell'architettura abbiamo definito ed istanziato i componenti utilizzati, i MUX 2:1, e due segnali per collegare i MUX del primo layer, generati con il costrutto *for generate*, al MUX del secondo layer.

Per quanto riguarda il MUX 16:1, l'implementazione in linguaggio VHDL segue la stessa logica del MUX 4:1. L'unica differenza ovviamente risiede nel numero di ingressi dato e di selezione e nel numero di segnali di collegamento e sottocomponenti usati.

```
entity mux_16_1 is
  Port (
    c : in std_logic_vector (0 to 15);
    s : in std_logic_vector (0 to 3);
    y : out std_logic
  );
end mux_16_1;


architecture structural of mux_16_1 is
    signal u : std_logic_vector(0 to 3);

    component mux_4_1 is
    port (
        b : in std_logic_vector (0 to 3);
        s0 : in std_logic;
        s1 : in std_logic;
        y0 : out std_logic );
    end component;
```

```
begin
    mux0_3 : FOR i in 0 to 3 GENERATE m : mux_4_1
    port map (
        b (0 to 3) => c(i*4 to i*4+3),
        s0 => s(0),
        s1 => s(1),
        y0 => u (i)
    );
    end GENERATE;

    mux4 : mux_4_1
    port map (
        b (0 to 3) => u (0 to 3),
        s0 => s(2),
        s1 => s(3),
        y0 => y
    );


end structural;
```

Simulazione

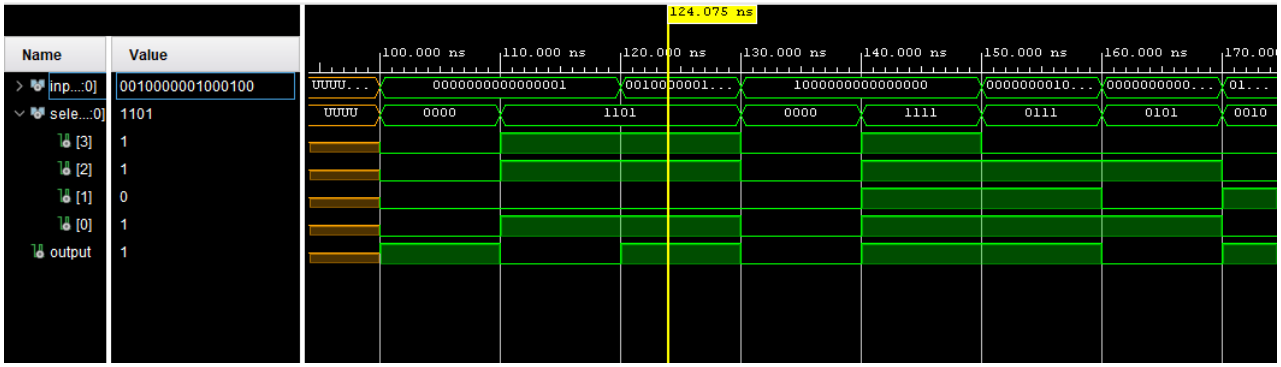
Di seguito si riporta il testbench usato per testare il MUX 16.1. Per brevità omettiamo in questa documentazione i codici VHDL utilizzati per la descrizione dei testbench, ed i risultati, seppur positivi, del testing del multiplexer 2:1 e 4.1.

Il sistema è stato testato dando in ingresso al top module differenti stringhe di input dato e differenti stringhe di selezione, abbiamo poi verificato gli output attesi mediante il costrutto assert.

```
21
22  library IEEE;
23  use IEEE.STD_LOGIC_1164.ALL;
24
25  entity mux_16_1_tb is
26  --  vuoto perchè non stiamo creando un oggetto
27  end mux_16_1_tb;
28
29  architecture Behavioral of mux_16_1_tb is
30     component mux_16_1 is
31      Port (
32      c : in std_logic_vector (0 to 15);
33      s : in std_logic_vector (0 to 3);
34      y : out std_logic
35     );
36     end component;
```

```
42      signal inputs       : std_logic_vector (15 downto 0);
43      signal selections   : std_logic_vector (3 downto 0);
44      signal output       : std_logic;
45
46      begin
47
48          uut : mux_16_1
49              port map(
50                  c => inputs,
51                  s => selections,
52                  y => output
53              );
54
55      stim_proc : process
56          begin
57
58              -- Caso 1: 1° ingresso attivo, selezione "0000" (1)
59              wait for 100 ns;
60              inputs <= "0000000000000001";
61              selections <= "0000";
62              wait for 10 ns;
63              ASSERT output = '1'
64                  REPORT "test case 1 fallito" SEVERITY failure;
65
66              -- Caso 2: 1° ingresso attivo, selezione "1101" (14)
67              selections <= "1101";
68
69            wait for 10 ns;
70            ASSERT output = '0'
71                REPORT "test case 2 fallito" SEVERITY failure;
72
73
74            -- Caso 3: 3°, 7°, 14° ingresso atttivo con selezione "1101" (14)
75            inputs <= "0010000001000100";
76            wait for 10 ns;
77            ASSERT output = '1'
78               REPORT "test case 3 fallito" SEVERITY failure;
79
80            -- Caso 4: 16° ingresso attivo, selezione "0000" (1)
81            inputs <= "1000000000000000";
82            selections <= "0000";
83            wait for 10 ns;
84            ASSERT output = '0'
85                REPORT "test case 4 fallito" SEVERITY failure;
86
87            -- Caso 5: 16° ingresso attivo, selezione "1111" (16)
88            selections <= "1111";
89            wait for 10 ns;
90            ASSERT output = '1'
91                REPORT "test case 5 fallito" SEVERITY failure;
92
93            -- Caso 6: 8° ingresso attivo, selezione "0111" (8)
94            inputs <= "0000000010000000";
```

L'output della simulazione è invece osservabile in figura.



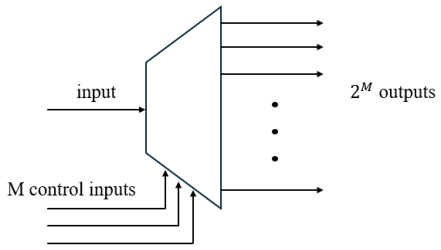
Esercizio 1.2

Utilizzando il componente sviluppato al punto precedente, progettare, implementare in VHDL e testare mediante simulazione una **rete di interconnessione a 16 sorgenti e 4 destinazioni**.

Progetto e architettura

Per ottenere la rete di interconnessione richiesta abbiamo bisogno, oltre al componente prima realizzato, del suo speculare: il demultiplexer.

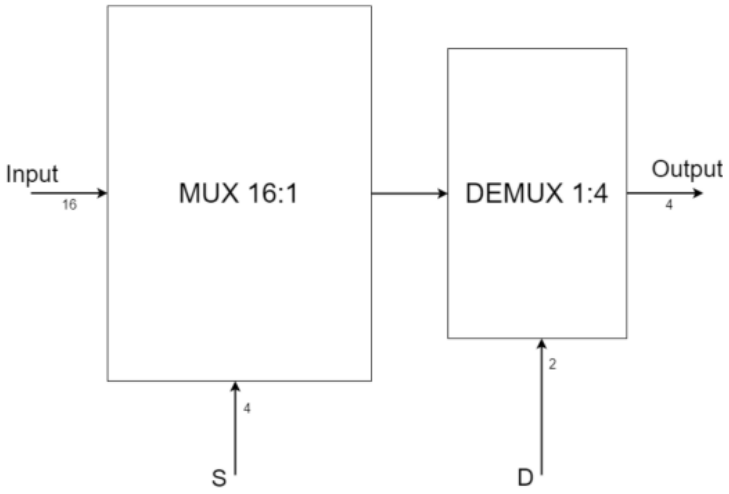
Nella sua versione indirizzabile presenta un singolo ingresso dato, M ingressi di selezione e $N = 2^M$ linee di uscita. Il dispositivo opera collegando la singola linea in input ad una delle possibili uscite e, come nel multiplexer, la logica di inoltro è governata dalle linee di selezione.



Il DEMUX 4:1 è quello che fa al caso nostro essendo composto da 1 singolo input e 4 output.

Collegando l'output del multiplexer 16:1 all'input del demultiplexer riusciamo a creare la rete di interconnessione richiesta.

La macchina risultante ha quindi 16 linee di ingresso dato e 4 linee di uscita. Le linee di selezione in totale diventano 6: le prime 4 permettono di determinare la sorgente mentre le ultime due la destinazione. In figura si riporta l'architettura complessiva.



Implementazione

La rete sfrutta il Multiplexer 16:1 realizzato nel precedente paragrafo e un Demultiplexer 1:4, la cui implementazione comportamentale è riportata di seguito.

```
29  entity demux_1_4 is
30      Port (
31          a : in std_logic;
32          s : in std_logic_vector (1 downto 0);
33          y : out std_logic_vector (3 downto 0)
34     );
35  end demux_1_4;
36
37  architecture Behavioral of demux_1_4 is
38      begin
39          y(0)<= a AND (NOT s(0)) AND (NOT s(1));
40          y(1)<= a AND  s(0) AND (NOT s(1));
41          y(2)<= a AND (NOT s(0)) AND s(1);
42          y(3)<= a AND  s(0) AND s(1);
43  end Behavioral;
44
```

Dunque, con approccio strutturale, abbiamo realizzato la nostra rete di interconnessione, definendone innanzitutto l'interfaccia all'interno dell'entity:

• 16 linee di ingresso dati che rappresentano le sorgenti;

• 4 linee di uscita che rappresentano le destinazioni;

• 6 linee di selezione, 4 per il MUX e 2 per il DEMUX.

Dopodiché, abbiamo definito un'architecture di tipo *structural* utilizzando solo i due componenti di interesse, che abbiamo collegato in maniera opportuna tramite l'apposito segnale di appoggio "u".

```
22  library IEEE;
23  use IEEE.STD_LOGIC_1164.ALL;
24  use IEEE.NUMERIC_STD.ALL;
25
26  entity rete_16_4 is
27      Port (
28          ar  : in std_logic_vector (15 downto 0);
29          sr  : in std_logic_vector (5 downto 0);
30          yr  : out std_logic_vector (3 downto 0)
31      );
32  end rete_16_4;
33
34  architecture Structural of rete_16_4 is
35
36      component mux_16_1 is
37          port (
38              c : in std_logic_vector (15 downto 0);
39              s : in std_logic_vector (3 downto 0);
40              y : out std_logic
41          );
42      end  component;
43
44      component demux_1_4 is
45          port (
46              a : in std_logic;
47              s : in std_logic_vector (1 downto 0);
```

```
            y : out std_logic_vector (3 downto 0)
        );
    end component;

    signal u : std_logic;

    begin

        LEVEL1 : mux_16_1
            port map (
                c => ar,
                s => sr (5 downto 2),
                y => u
            );

        LEVEL2 : demux_1_4
            port map (
                a => u,
                s => sr (1 downto 0),
                y => yr
            );

end Structural;
```

Simulazione

Per verificare il corretto funzionamento della rete da noi implementata, abbiamo scritto un testbench per definire gli input con cui stimolare il MUX 16:1 e le asserzioni sugli output da verificare.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity rete_16_4_tb is
end rete_16_4_tb;

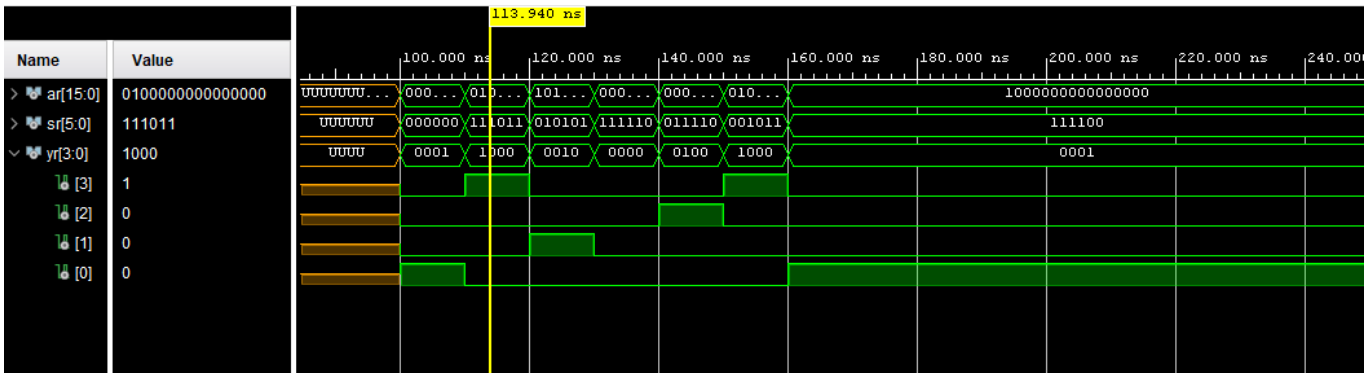
architecture Behavioral of rete_16_4_tb is
    -- Dichiarazione del componente sotto test
    component rete_16_4 is
        Port (
            ar : in std_logic_vector (15 downto 0); -- 16 ingressi (1-16)
            sr : in std_logic_vector (5 downto 0);  -- 6 bit di selezione (4 per il mux, 2 per il demux)
            yr : out std_logic_vector (3 downto 0)  -- 4 uscite del demux (1-4)
        );
    end component;
    -- Segnali di test
    signal ar : std_logic_vector (15 downto 0); -- 16 ingressi
    signal sr : std_logic_vector (5 downto 0);  -- 6 bit di selezione
    signal yr : std_logic_vector (3 downto 0);  -- 4 uscite

begin
    -- Istanziazione della rete sotto test
    UUT: rete_16_4
        port map (
            ar => ar,
            sr => sr,
            yr => yr
        );

    -- Processo di stimolo
    stimulus: process
    begin
        -- Caso 1: Ingresso 1 attivo, selezione mux 1, demux 1
        wait for 100 ns;
```

```
55          ar <= "0000000000000001"; -- Ingresso 1 attivo (bit 0)
56          sr <= "000000";           -- Mux: 1 (0000), Demux: 1 (00)
57          wait for 10 ns;
58          ASSERT yr = "0001"        -- Uscita 1 attiva
59              REPORT "Caso 1 fallito" SEVERITY failure;
60
61          -- Caso 2: Ingresso 15 attivo, selezione mux 15, demux 4
62          ar <= "0100000000000000"; -- Ingresso 15 attivo (bit 14)
63          sr <= "111011";           -- Mux: 15 (1110), Demux: 4 (11)
64          wait for 10 ns;
65          ASSERT yr = "1000"        -- Uscita 4 attiva
66              REPORT "Caso 2 fallito" SEVERITY failure;
67
68          -- Caso 3: Ingressi pari attivi, selezione mux 6, demux 2
69          ar <= "1010101010101010"; -- Ingressi pari attivi (2, 4, 6, ...)
70          sr <= "010101";           -- Mux: 6 (0101), Demux: 2 (01)
71          wait for 10 ns;
72          ASSERT yr = "0010"        -- Uscita 2 attiva
73              REPORT "Caso 3 fallito" SEVERITY failure;
74
75          -- Caso 4: Nessun ingresso attivo, selezione mux 16, demux 3
76          ar <= "0000000000000000"; -- Nessun ingresso attivo
77          sr <= "111110";           -- Mux: 16 (1111), Demux: 3 (10)
78          wait for 10 ns;
79          ASSERT yr = "0000"        -- Nessuna uscita attiva
80              REPORT "Caso 4 fallito" SEVERITY failure;
82           -- Caso 5: Ingresso 8 attivo, selezione mux 8, demux 3
83          ar <= "0000000010000000"; -- Ingresso 8 attivo (bit 7)
84          sr <= "011110";           -- Mux: 8 (0111), Demux: 3 (10)
85          wait for 10 ns;
86          ASSERT yr = "0100"        -- Uscita 3 attiva
87              REPORT "Caso 5 fallito" SEVERITY failure;
88
89          -- Caso 6: Ingressi dispari attivi, selezione mux 3, demux 4
90          ar <= "0101010101010101"; -- Ingressi dispari attivi (1, 3, 5, ...)
91          sr <= "001011";           -- Mux: 3 (0010), Demux: 4 (11)
92          wait for 10 ns;
93          ASSERT yr = "1000"        -- Uscita 4 attiva
94              REPORT "Caso 6 fallito" SEVERITY failure;
95
96          -- Caso 7: Ingresso 16 attivo, selezione mux 16, demux 1
97          ar <= "1000000000000000"; -- Ingresso 16 attivo (bit 15)
98          sr <= "111100";           -- Mux: 16 (1111), Demux: 1 (00)
99          wait for 10 ns;
100         ASSERT yr = "0001"        -- Uscita 1 attiva
101             REPORT "Caso 7 fallito" SEVERITY failure;
102
103         -- Fine del test
104         wait;
105     end process;
106
107 end Behavioral;
```

L'output della simulazione è osservabile in figura. E' possibile visualizzare i diversi valori delle uscite in corrispondenza delle variazioni in termini di ingressi attivi e segnali di selezione.



Esercizio 1.3

Sintetizzare ed implementare su board il progetto della rete di interconnessione sviluppato al punto 1.2, utilizzando gli switch per fornire gli input di selezione e i led per visualizzare i 4 bit di uscita. Per quanto riguarda i 16 bit dato in input, essi devono essere immessi mediante switch, 8 bit alla volta, sviluppando un'apposita "rete di controllo" per l'acquisizione che utilizzi due bottoni della board per caricare rispettivamente la prima e la seconda metà del dato in ingresso.

Sintesi su board di sviluppo

Per l'implementazione della rete di interconnessione su board abbiamo aggiunto, rispetto all'esercizio precedente, un'ulteriore entità chiamata CONTROL_UNIT che preleva gli input forniti tramite board.

```
13  entity control_unit is
14    Port (
15      firstButton: in std_logic;
16      secondButton : in std_logic;
17      loadButton: in std_logic;
18      switch: in std_logic_vector(7 DOWNTO 0);
19      ctr : in std_logic_vector(5 DOWNTO 0);
20      CLK: in std_logic;
21      led : out std_logic_vector(3 DOWNTO 0)
22
23    );
24  end control_unit;
25
26  ARCHITECTURE arch OF control_unit IS
27      COMPONENT rete_16_4 IS
28          PORT (
29          ar : IN STD_LOGIC_VECTOR(15 DOWNTO 0);
30          sr : IN STD_LOGIC_VECTOR(5 DOWNTO 0);
31          yr : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
32          );
33      END COMPONENT;
34  signal address: std_logic_vector(15 DOWNTO 0);
35  signal control: std_logic_vector(5 downto 0);
36
37  BEGIN
38       net: rete_16_4
39          PORT MAP(
40          ar => address,
41          sr => control,
42          yr => led
43          );
44
45  process(CLK)
46  BEGIN
47      if(rising_edge(CLK)) then
48          if(firstButton = '1') then
49              address(7 downto 0) <= switch;
50          end if;
51          if(secondButton = '1') then
52              address(15 downto 8) <= switch;
53          end if;
54          if(loadButton = '1') then
55              control(1 downto 0) <= ctr(1 downto 0);
56              control(5 downto 2) <= ctr(5 downto 2);
57          end if;
58      end if;
59  end process;
60  END ARCHITECTURE;
```

Tramite 8 switches inseriamo i 16 bit di address divisi in due gruppi da 8 bit e caricati, quindi, tramite due bottoni diversi.

Il file dei constraints per la board NEXYS-A7-100T-Master è stato modificato nel seguente modo:

```
## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { CLK }]; #IO_L12P_T1_MRCC_35 Sch=clk100mhz
#create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {CLK100MHZ}];


##Switches
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { switch[0] }]; #IO_L24N_T3_RS0_15 Sch=sw[0]
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { switch[1] }]; #IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { switch[2] }]; #IO_L6N_T0_D08_VREF_14 Sch=sw[2]
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { switch[3] }]; #IO_L13N_T2_MRCC_14 Sch=sw[3]
set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { switch[4] }]; #IO_L12N_T1_MRCC_14 Sch=sw[4]
set_property -dict { PACKAGE_PIN T18   IOSTANDARD LVCMOS33 } [get_ports { switch[5] }]; #IO_L7N_T1_D10_14 Sch=sw[5]
set_property -dict { PACKAGE_PIN U18   IOSTANDARD LVCMOS33 } [get_ports { switch[6] }]; #IO_L17N_T2_A13_D29_14 Sch=sw[6]
set_property -dict { PACKAGE_PIN R13   IOSTANDARD LVCMOS33 } [get_ports { switch[7] }]; #IO_L5N_T0_D07_14 Sch=sw[7]
set_property -dict { PACKAGE_PIN T8    IOSTANDARD LVCMOS18 } [get_ports { ctr[0] }]; #IO_L24N_T3_34 Sch=sw[8]
set_property -dict { PACKAGE_PIN U8    IOSTANDARD LVCMOS18 } [get_ports { ctr[1] }]; #IO_25_34 Sch=sw[9]
set_property -dict { PACKAGE_PIN R16   IOSTANDARD LVCMOS33 } [get_ports { ctr[2] }]; #IO_L15P_T2_DQS_RDWR_B_14 Sch=sw[10]
set_property -dict { PACKAGE_PIN T13   IOSTANDARD LVCMOS33 } [get_ports { ctr[3] }]; #IO_L23P_T3_A03_D19_14 Sch=sw[11]
set_property -dict { PACKAGE_PIN H6    IOSTANDARD LVCMOS33 } [get_ports { ctr[4] }]; #IO_L24P_T3_35 Sch=sw[12]
set_property -dict { PACKAGE_PIN U12   IOSTANDARD LVCMOS33 } [get_ports { ctr[5] }]; #IO_L20P_T3_A08_D24_14 Sch=sw[13]
#set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { SW[14] }]; #IO_L19N_T3_A09_D25_VREF_14 Sch=sw[14]
#set_property -dict { PACKAGE_PIN V10   IOSTANDARD LVCMOS33 } [get_ports { SW[15] }]; #IO_L21P_T3_DQS_14 Sch=sw[15]
```

Dunque, 8 switches sono utilizzati per 8 bit di address e i successivi 6 switches sono utilizzati per inserire i segnali di selezione.

```
## LEDs
set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { led[0] }]; #IO_L18P_T2_A24_15 Sch=led[0]
set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { led[1] }]; #IO_L24P_T3_RS1_15 Sch=led[1]
set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { led[2] }]; #IO_L17N_T2_A25_15 Sch=led[2]
set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports { led[3] }]; #IO_L8P_T1_D11_14 Sch=led[3]
```

```
##Buttons
#set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { CPU_RESETN }]; #IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resetn
set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports {  loadButton }]; #IO_L9P_T1_DQS_14 Sch=btnc
#set_property -dict { PACKAGE_PIN M18   IOSTANDARD LVCMOS33 } [get_ports { BTNU }]; #IO_L4N_T0_D05_14 Sch=btnu
set_property -dict { PACKAGE_PIN P17   IOSTANDARD LVCMOS33 } [get_ports {  firstButton }]; #IO_L12P_T1_MRCC_14 Sch=btnl
set_property -dict { PACKAGE_PIN M17   IOSTANDARD LVCMOS33 } [get_ports {  secondButton }]; #IO_L10N_T1_D15_14 Sch=btnr
#set property -dict { PACKAGE_PIN P18   IOSTANDARD LVCMOS33 } [get ports { BTND }]; #IO L9N T1 DQS D13 14 Sch=btnd
```

Utilizzando il bottone centrale per il load, il bottone si sinistra per caricare i primi 8 bit di address e il bottone di destra per caricare i suoi successivi 8 bit.

Esercizio 2: Sistema ROM + M

Esercizio 2.1

Progettare, implementare in VHDL e testare mediante simulazione un **sistema S** composto da una **ROM** puramente combinatoria di 16 locazioni da 8 bit ciascuna e da una macchina combinatoria **M** che opera come segue: fornito al sistema un indirizzo A di 4 bit, il sistema restituisce il valore contenuto nella ROM all'indirizzo A opportunamente "trasformato" attraverso la macchina M. Il comportamento della macchina M è totalmente a scelta dello studente, l'unico vincolo è che essa prenda in ingresso 8 bit e ne fornisca in uscita 4.

Progetto e architettura

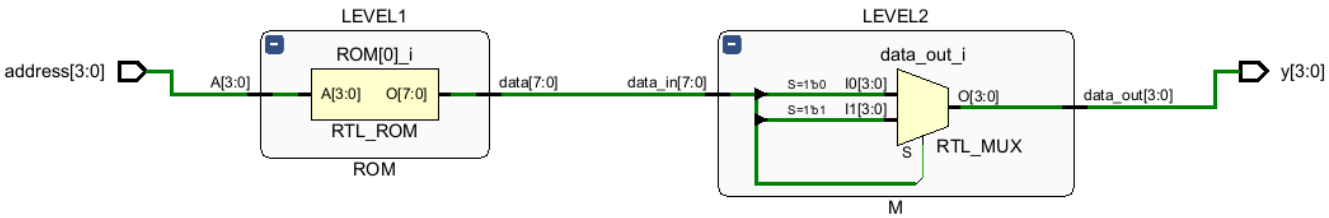
Una ROM (Read-Only Memory) è una memoria di sola lettura e non volatile, cioè capace di conservare i dati presenti anche in assenza di alimentazione.

A seconda delle dimensioni, una ROM è dotata di un determinato numero di locazioni di memoria. Ognuna di esse può memorizzare un certo numero di bit: per accedere ai dati memorizzati si utilizza l'indirizzo delle corrispondenti locazioni di memoria che li mantengono.

Poiché il sistema richiesto prevede l'interazione di due componenti, abbiamo innanzitutto realizzato singolarmente la memoria ROM e la macchina combinatoria M, per poi connetterle opportunamente.

La ROM realizzata, come è possibile osservare in figura, ha un'interfaccia che prevede un segnale di ingresso di 4 bit, cioè l'indirizzo di una delle $2^4 = 16$ locazioni di memoria a cui si vuole accedere in lettura, e un segnale di uscita di 8 bit, contenente il valore letto in quella posizione. Dal punto di vista architetturale, abbiamo gestito le locazioni come un vettore di 16 elementi di dimensione 8 bit ciascuno, come richiesto dalla traccia, e abbiamo definito un'architettura di tipo behavioral per accedere ai valorime morizzati nella ROM.

La macchina combinatoria M prende in ingresso l'uscita a 8 bit della ROM e, a seconda del valore del segnale di abilitazione, fornisce un'uscita su 4 bit che coincide con l'uscita del sistema complessivo.



Implementazione

L'entity della ROM definisce il segnale di 4 bit in ingresso, contenente l'indirizzo A della locazione a cui si vuole accedere, e il segnale di 8 bit data in uscita, che rappresenta il dato letto dalla ROM. Essendo un'architettura behavioral, abbiamo definito la procedura get_data che, dato un indirizzo in ingresso, restituisce il valore contenuto nella locazione identificata da quell'indirizzo.

```
15  library IEEE;
16  use IEEE.STD_LOGIC_1164.ALL;
17  use IEEE.NUMERIC_STD.ALL;
18
19  entity ROM is
20      Port (
21          A       : in std_logic_vector (3 downto 0);
22          data    : out std_logic_vector (7 downto 0)
23          --dato contenuto nella locazione di memoria di indirizzo A
24      );
25  end ROM;
26
27  architecture Behavioral of ROM is
28
29      type MEMORY_16_8 is array (0 TO 15) of std_logic_vector(7 downto 0);
30      constant ROM : MEMORY_16_8 := (
31          b"01000001",
32          b"01100111",
33          b"11010010",
34          b"10001101",
35          b"00111010",
36          b"01000100",
37          b"00101100",
38          b"01000100",

39          b"11001101",
40          b"01101011",
41          b"10001110",
42          b"11101011",
43          b"10011111",
44          b"01001101",
45          b"11001100",
46          b"00111011"
47      );
48
49  begin
50      get_data : process (A)
51      begin
52          data <= ROM (to_integer(unsigned(A)));
53
54      end process get_data;
55  end Behavioral;
```

L'entity della macchina M, invece, prevede 8 bit in ingresso trasformati in 4 bit in uscita.

Se il bit meno significativo data_in(0) è '0', data_out riceve i 4 bit più significativi di data_in, ovvero data_in(7 downto 4). Se data_in(0) è '1', data_out riceve i 4 bit meno significativi, ovvero data_in(3 downto 0).

```vhdl
entity M is
  Port (
    data_in : in std_logic_vector (7 downto 0);
    data_out: out std_logic_vector (3 downto 0)
  );
end M;

architecture Behavioral of M is
begin
--  se il LSB di data in è 0 inoltra i primi 4 bit più significativi
    convert_data : process (data_in)
    begin
    case data_in(0) is
        when '0' => data_out <= data_in(7) & data_in(6) & data_in(5) & data_in(4);
        when '1' => data_out <= data_in(3) & data_in(2)& data_in(1) & data_in(0);
        when others => data_out <= "0000";
    end case;
    end process;

end Behavioral;
```

Il sistema S è stato realizzato di conseguenza per composizione, utilizzando i due componenti visti, la sua interfaccia, definita nell'entity, prevede:

• address: l'indirizzo A della locazione a cui accedere;

• y: segnale di uscita.

L'architecture è di tipo *structural*, per cui abbiamo definito innanzitutto i componenti necessari e poi i collegamenti tra le diverse linee di ingresso e di uscita; a tal fine, sfruttiamo il segnale di appoggio "u" per stabilire il collegamento tra output della ROM e input di M.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity sistema is
    Port (
        address : in std_logic_vector (3 downto 0);
        y       : out std_logic_vector (3 downto 0)
    );
end sistema;

architecture structural of sistema is
    signal u : std_logic_vector (7 downto 0) :=(others => '0');

    component ROM is
        port (
            A   : in std_logic_vector (3 downto 0);
            data: out std_logic_vector (7 downto 0)
        );
    end component;

    component M is
        port (
            data_in : in std_logic_vector (7 downto 0);
```

```
46              data_out: out std_logic_vector (3 downto 0)
47          );
48      end component;
49
50      begin
51      LEVEL1 : ROM
52          port map (
53              A => address ,
54              data => u
55          );
56
57       LEVEL2 : M
58       port map (
59          data_in  => u ,
60          data_out => y
61       );
62
63  end structural;
```

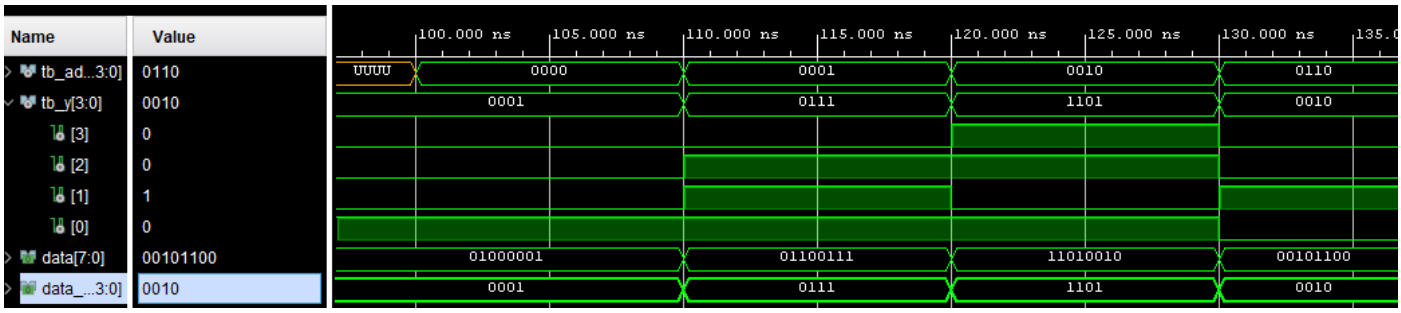
Simulazione

Abbiamo simulato il funzionamento del sistema S attraverso un testbench che, dato un indirizzo di memoria, verifica se l'uscita del sistema coincide con il valore contenuto nelle locazioni di memoria con quell'indirizzo.

```
24  entity tb_sistema is
25  end tb_sistema;
26
27  architecture Behavioral of tb_sistema is
28      -- Componenti da testare
29      component sistema is
30          Port (
31              address : in std_logic_vector (3 downto 0);
32              y       : out std_logic_vector (3 downto 0)
33          );
34      end component;
35
36      -- Segnali locali per il test
37      signal tb_address : std_logic_vector(3 downto 0);
38      signal tb_y       : std_logic_vector(3 downto 0);
39
40
41      begin
42      -- Instanza del DUT (Device Under Test)
43      uut: sistema
44          port map (
45              address => tb_address,
46              y       => tb_y
47          );
```

```
50      -- Processo di stimolo
51      stimulus: process
52      begin
53
54          wait for 100 ns;
55          -- Test Case 1: Address = 0000
56          tb_address <= "0000";
57          wait for 10 ns;
58          ASSERT tb_y = "0001"
59              REPORT "Caso 1 fallito: Uscita non corrisponde a 0000" SEVERITY failure;
60
61          -- Test Case 2: Address = 0001
62          tb_address <= "0001";
63          wait for 10 ns;
64          ASSERT tb_y = "0111"
65              REPORT "Caso 2 fallito: Uscita non corrisponde a 0001" SEVERITY failure;
66
67          -- Test Case 3: Address = 0010
68          tb_address <= "0010";
69          wait for 10 ns;
70          ASSERT tb_y = "1101"
71              REPORT "Caso 3 fallito: Uscita non corrisponde a 0010" SEVERITY failure;
72
73          -- Test Case 4: Address = 0110
74          tb_address <= "0110";
75          wait for 10 ns;
76          ASSERT tb_y = "0010"
77              REPORT "Caso 4 fallito: Uscita non corrisponde a 1111" SEVERITY failure;
78
79              -- Test Case 4: Address = 0110
80          tb_address <= "0110";
81          wait for 10 ns;
82          ASSERT tb_y = "0010"
83              REPORT "Caso 4 fallito: Uscita non corrisponde a 1111" SEVERITY failure;
84          -- Fine della simulazione
85
86              -- Test Case 5: Address = 1111
87          tb_address <= "1111";
88          wait for 10 ns;
89          ASSERT tb_y = "1011"
90              REPORT "Caso 5 fallito: Uscita non corrisponde a 1111" SEVERITY failure;
91          -- Fine della simulazione
92
93
94          wait;
95      end process;
96
97  end Behavioral;
```

Di seguito è riportato il risultato della simulazione.



In *data_out* è possibile osservare il corretto funzionamento della macchina M, la quale propaga i 4 bit più significativi di *data* se l'LSB è 0, i 4 meno significativi altrimenti

Esercizio 2.2

Sintetizzare ed implementare su board il progetto della rete di interconnessione sviluppato al punto 1.2, utilizzando gli switch per fornire gli input di selezione e i led per visualizzare i 4 bit di uscita. Per quanto riguarda i 16 bit dato in input, essi devono essere immessi mediante switch, 8 bit alla volta, sviluppando un'apposita "rete di controllo" per l'acquisizione che utilizzi due bottoni della board per caricare rispettivamente la prima e la seconda metà del dato in ingresso.

Sintesi su board di sviluppo

Per la sintesi su board del progetto sistema ROM+M non è stato necessario aggiungere componenti ma è stato sufficiente modificare il file dei constraints per la board NEXYS-A7-100T-Master.

```
## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { CLK100MHZ }]; #IO_L12P_T1_MRCC_35 Sch=clk100mhz
#create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {CLK100MHZ}];


##Switches
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { address[0] }]; #IO_L24N_T3_RS0_15 Sch=sw[0]
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { address[1] }]; #IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { address[2] }]; #IO_L6N_T0_D08_VREF_14 Sch=sw[2]
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { address[3] }]; #IO_L13N_T2_MRCC_14 Sch=sw[3]
```

Collegando i primi 4 switches ai 4 bit di *address* e i primi 4 led ai 4 bit dell'uscita *y*.

```
## LEDs
set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { y[0] }]; #IO_L18P_T2_A24_15 Sch=led[0]
set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { y[1] }]; #IO_L24P_T3_RS1_15 Sch=led[1]
set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { y[2] }]; #IO_L17N_T2_A25_15 Sch=led[2]
set property -dict { PACKAGE PIN N14   IOSTANDARD LVCMOS33 } [get ports { y[3] }]; #IO L8P T1 D11 14 Sch=led[3]
```

Capitolo 2: Reti sequenziali elementari

Esercizio 3: Riconoscitore di sequenza

Un riconoscitore di sequenze è una macchina sequenziale notevole.

Una macchina sequenziale è un sistema la cui uscita in un certo istante non dipende solo dal valore degli ingressi nello stesso istante, ma anche dagli ingressi precedenti. Viene quindi introdotto il concetto di **stato**, che può anche essere definito come memoria e che rappresenta le informazioni relative all'attività passata della macchina. In questo caso, il concetto di stato ci è utile in quanto, per riconoscere una sequenza di bit, non basta conoscere solo l'ingresso all'istante corrente, ma anche bisogna anche ricordare in qualche modo gli ingressi passati. Una macchina sequenziale può essere modellata come un automa a stati finiti (ASF).

Esercizio 3.1

Progettare, implementare in VHDL e testare mediante simulazione una macchina in grado di riconoscere la sequenza **101.** La macchina prende in ingresso un segnale binario i che rappresenta il dato, un segnale A di tempificazione e un segnale M di modo, che ne disciplina il funzionamento, e fornisce un'uscita Y alta quando la sequenza viene riconosciuta. In particolare,

- se M=0, la macchina valuta i bit seriali in ingresso a gruppi di 3 (sequenze non sovrapposte),
- se M=1, la macchina valuta i bit seriali in ingresso uno alla volta, tornando allo stato iniziale ogni volta che la sequenza viene correttamente riconosciuta (sequenze parzialmente sovrapposte).

Progetto e architettura

Abbiamo progettato il riconoscitore come un ASF di Mealy, per cui abbiamo realizzato il grafo degli stati e delle transizioni sia per il caso di riconoscimento senza sovrapposizione sia per quello di riconoscimento con sovrapposizione parziale.

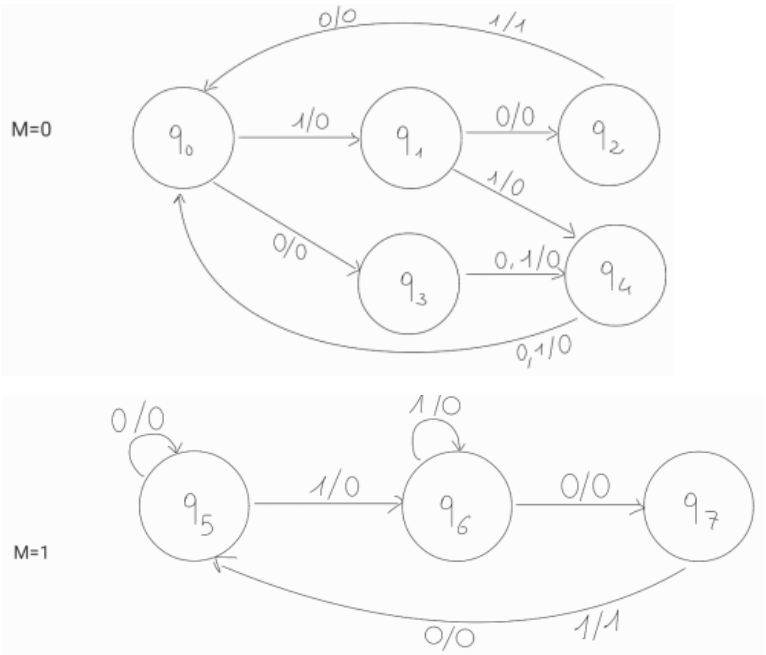
Dopo aver modellato l'automa attraverso i grafi necessari, abbiamo progettato il riconoscitore descrivendolo secondo un approccio *behavioral*. In particolare, abbiamo definito due comportamenti diversi:

• macchina combinatoria: si occupa di determinare le transizioni di stato e le uscite sulla base del modo selezionato, dello stato attuale e dell'ingresso applicato;

• memoria: si occupa di aggiornare lo stato e l'uscita in corrispondenza del fronte di salita (*rising edge*) del segnale di clock.

Il segnale di clock è un segnale di tempificazione che regola la corretta evoluzione della macchina e la sua presenza è fondamentale in quanto il riconoscitore è una macchina sincrona, per questo c'è bisogno di questo segnale per garantire che non avvengano transizioni indesiderate e che si giunga in stati di funzionamento non corretti, dato che altrimenti l'evoluzione della macchina continuerebbe fintanto che è applicato l'ingresso.

Riportiamo in figura gli ASF nelle due configurazioni del riconoscitore di sequenza.



Implementazione

Il riconoscitore prevede clock, reset, modo di funzionamento e bit "i" in ingresso. L'insieme degli stati comprende 8 elementi e vi sono due segnali di stato corrente, inizializzato a q0, e di stato prossimo.

```
11  library IEEE;
12  use IEEE.STD_LOGIC_1164.ALL;
13
14  entity riconoscitore is
15      Port (
16          i, rst, m   : in std_logic; --ingresso, reset, modo
17          A           : in std_logic; --clock
18          y           : out std_logic --uscita
19      );
20  end riconoscitore;
21
22  architecture Behavioral of riconoscitore is
23
24      type stato is (q0,q1,q2,q3,q4,q5,q6,q7);
25      signal stato_corrente : stato := q0;
26      signal stato_prossimo : stato := stato_corrente;
27      signal y_temp : std_logic;
```

```
33      begin
34          update : process (stato_corrente, i, m)
35          begin
36          Case m is
37              when '0' =>     --non sovrapposte
38                  Case stato_corrente is
39
40                      when q0 =>
41                          if (i = '0') then
42                              stato_prossimo <= q3;
43                              y_temp <= '0';
44                          else
45                              stato_prossimo <= q1;
46                              y_temp <= '0';
47                          end if;
48
49                      when q3 =>
50                          stato_prossimo <= q4;
51                          y_temp <= '0';
52
53                      when q4 =>
54                          stato_prossimo <= q0;
55                          y_temp <= '0';
56
57                      when q1 =>
58                          if (i = '0') then
59                              stato_prossimo <= q2;
60                              y_temp <= '0';
61                          else
62                              stato_prossimo <= q4;
63                              y_temp <= '0';
64                          end if;
65                      when q2 =>
66                          if (i='0') then
67                              stato_prossimo <= q0;
68                              y_temp <= '0';
69                          else
70                              stato_prossimo <= q0;
71                              y_temp <= '1';
72                          end if;
73                      when others =>
74                          stato_prossimo <= q0;
75                  end case;
76
77              when '1' =>   --sovrapposte parzialmente
78                  Case stato_corrente is
79
80                      when q5 =>
81                          if (i='0') then
82                              stato_prossimo <= q5;
```

```
                            y_temp <= '0';
                        else
                            stato_prossimo <= q6;
                            y_temp <= '0';
                        end if;

                    when q6 =>
                        if (i='0') then
                            stato_prossimo <= q7;
                            y_temp <= '0';
                        else
                            stato_prossimo <= q6;
                            y_temp <= '0';
                        end if;

                    when q7 =>
                        if (i='0') then
                            stato_prossimo <= q5;
                            y_temp <= '0';
                        else
                            stato_prossimo <= q5;
                            y_temp <= '1';
                        end if;
                    when others =>
                        stato_prossimo <= q5;
                end case;
```

```
            when others =>  --default
--                stato_prossimo <= q0;
                y_temp <= '0';
        end case;
        end process;

        mem : process (A)
        begin
        if (rising_edge(A)) then

            if (rst = '1' and m = '1') then
                stato_corrente <= q5;
                y <= '0';

            elsif (rst = '1' and m = '0') then
                stato_corrente <= q0;
                y <= '0';

            else
                stato_corrente <= stato_prossimo;
                y <= y_temp;

            end if;
        end if;
        end process;
end Behavioral;
```

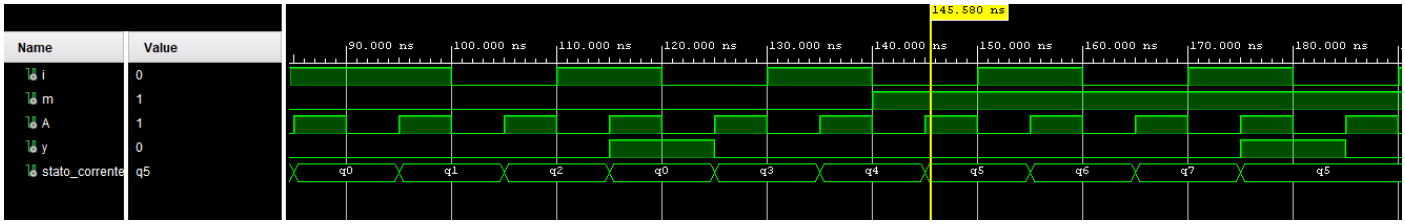
Simulazione

Di seguito è riportato il codice del testbench per la simulazione del riconoscitore, inizialmente per sequenze non sovrapposte e, successivamente, modificando la modalità di funzionamento, per sequenze non sovrapposte.

```
20  library IEEE;
21  use IEEE.STD_LOGIC_1164.ALL;
22
23  entity tb_riconoscitore is
24  end tb_riconoscitore;
25
26  architecture behavioral of tb_riconoscitore is
27
28      component riconoscitore_m
29          port(
30              i   : in std_logic;          -- ingresso
31              rst : in std_logic;          -- reset
32              m   : in std_logic;          -- modalità
33              A   : in std_logic;          -- clock
34              y   : out std_logic          -- uscita
35          );
36      end component;
37
38      -- Dichiarazione dei segnali
39      signal i : std_logic := '0';         -- ingresso
40      signal rst : std_logic := '0';       -- reset
41      signal m : std_logic := '0';         -- modalità
42      signal A : std_logic := '0';         -- clock
43      signal y : std_logic;                -- uscita
44
45      -- Periodo del clock
46      constant clk_period : time := 10 ns;
47
48  begin
49
50      -- Istanza del modulo riconoscitore
51      uut: entity work.riconoscitore
52          port map (
53              i => i,
54              rst => rst,
55              m => m,
56              A => A,
57              y => y
58          );
59
60      -- Generazione del clock
61      clk_process : process
62      begin
63          A <= '0';
64          wait for clk_period / 2;
65          A <= '1';
66          wait for clk_period / 2;
67      end process;
68
69      -- Stimoli di ingresso (sequenza di test)
70      stim_proc: process
71      begin
72
73          -- Test con sequenza di valori di i
74          m <= '0';  -- modalità non sovrapposte
75          i <= '0';
76          wait for 10 ns;
77          i <= '1';
78          wait for 10 ns;
79          i <= '0';
```

```
106         i <= '0';
107         wait for 10 ns;
108         i <= '1';
109         wait for 10 ns;
110         i <= '0';
111         wait for 10 ns;
112         i <= '1';
113         wait for 10 ns;
114         i <= '0';
115         wait for 10 ns;
116         i <= '1';
117         wait for 30 ns;
118         i <= '0';
119         wait for 10 ns;
120         i <= '1';
121         wait for 10 ns;
122         i <= '0';
123         wait for 10 ns;
124         i <= '1';
125         wait for 10 ns;
126
127         -- Completamento simulazione
128         wait;
129     end process;
130
131 end;
```

In figura è riportato il risultato della simulazione, in cui si può osservare l'evoluzione dello stato corrente e dell'uscita in due modalità differenti.



Esercizio 3.2

Sintetizzare e implementare su board la rete sviluppata al punto precedente, utilizzando uno switch S1 per codificare l'input i e uno switch S2 per codificare il modo M, in combinazione con due bottoni B1 e B2 utilizzati rispettivamente per acquisire l'input da S1 e S2 in sincronismo con il segnale di tempificazione A, che deve essere ottenuto a partire dal clock della board. Infine, l'uscita Y può essere codificata utilizzando un led.

Sintesi su board di sviluppo

Per effettuare la sintesi su board, è stato necessario implementare una control_unit in cui istanziare due bottoni e il riconoscitore di sequenze progettato e utilizzare due switch per input e modo di funzionamento del riconoscitore.

```
15  entity control_unit is
16    Port (
17      BTNL : in std_logic;
18      BTNR : in std_logic;
19      A, rst : in std_logic;
20      SW1, SW0 : in std_logic;
21      Y : out std_logic
22    );
23  end control_unit;
24
25  architecture Behavioral of control_unit is
26      component ButtonDebouncer
27      generic (
28          CLK_period: integer := 10;  -- periodo del clock (della board) in nanosecondi
29          btn_noise_time: integer := 10000000 -- durata stimata dell'oscillazione del bottone in nanosecondi
30                                              -- il valore di default è 10 millisecondi
31      );
32      Port ( RST : in STD_LOGIC;
33             CLK : in STD_LOGIC;
34             BTN : in STD_LOGIC;
35             CLEARED_BTN : out STD_LOGIC);
36      end component;
37
38      component riconoscitore_board
39      Port (
40          i, rst, m   : in std_logic; --ingresso, reset, modo
41          A           : in std_logic; --clock
42          Bi, Bm      : in std_logic;
43          y           : out std_logic --uscita
44      );
45      end component;
46
47      signal temp_cleared_btn0 : std_logic;
48      signal temp_cleared_btn1 : std_logic;
49  begin
50      BTN0 : ButtonDebouncer
51      generic map(
52          CLK_period => 10,  -- periodo del clock (della board) in nanosecondi
53          btn_noise_time => 10000000 -- durata stimata dell'oscillazione del bottone in nanosecondi
54                                     -- il valore di default è 10 millisecondi
55      )
56      Port map(
57             RST => RST,
58             CLK => A,
59             BTN => BTNL,
60             CLEARED_BTN => TEMP_CLEARED_BTN0
61             );
62
63      BTN1 : ButtonDebouncer
64      generic map(
```

```
65          CLK_period => 10,  -- periodo del clock (della board) in nanosecondi
66          btn_noise_time => 10000000 -- durata stimata dell'oscillazione del bottone in nanosecondi
67                                          -- il valore di default è 10 millisecondi
68      )
69      Port map(
70             RST => RST,
71             CLK => A,
72             BTN => BTNR,
73             CLEARED_BTN => TEMP_CLEARED_BTN1
74             );
75      R : riconoscitore_board
76      Port map(
77          i   => SW0,
78          rst => rst,
79          m   => SW1,
80          Bi => TEMP_CLEARED_BTN0,
81          Bm => TEMP_CLEARED_BTN1,
82          A   => A,
83          y   => Y
84      );
85
86  end Behavioral;
```

Per poter utilizzare i due bottoni B1 e B2 rispettivamente per acquisire l'input da SW0 e SW1 in sincronismo con il segnale di tempificazione A, nel riconoscitore sono stati aggiunti due segnali in ingresso: Bi e Bm da associare all'uscita del ButtonDebouncer nella control_unit.

```
14  entity riconoscitore_board is
15      Port (
16          i, rst, m   : in std_logic; --ingresso, reset, modo
17          A           : in std_logic; --clock
18          Bi, Bm      : in std_logic;
19          y           : out std_logic --uscita
20      );
21  end riconoscitore_board;
22
23  architecture Behavioral of riconoscitore_board is
24
25      type stato is (q0,q1,q2,q3,q4,q5,q6,q7);
26      signal stato_corrente : stato := q0;
27      signal stato_prossimo : stato := stato_corrente;
28      signal y_temp : std_logic;
29      signal  m_temp : std_logic := '0';
30
31      begin
32          update : process (stato_corrente, i, m_temp)
33          begin
34          Case m_temp is
35              when '0' =>    --non sovrapposte
36                  Case stato_corrente is
37
38                      when q0 =>
```

```
115         mem : process (A)
116         begin
117         if (rising_edge(A)) then
118
119             if (rst = '1' and m_temp = '1') then
120                 stato_corrente <= q5;
121                 y <= '0';
122
123             elsif (rst = '1' and m_temp = '0') then
124                 stato_corrente <= q0;
125                 y <= '0';
126
127             elsif Bi='1' then
128                 stato_corrente <= stato_prossimo;
129                 y <= y_temp;
130
131             elsif BM = '1' then
132                 m_temp <= M;
133                 if (M = '1') then
134                     stato_corrente <= q5;
135
136                 elsif (M = '0') then
137                     stato_corrente <= q0;
138
139                 end if;
```

Di conseguenza, abbiamo modificato i constraints della NEXYS-A7-100T-Master per utilizzare i primi due switches della board

```
6  ## Clock signal
7  set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { A }]; #IO_L12P_T1_MRCC_35 Sch=clk100mhz
8  create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {A}];
9
10
11 ##Switches
12 set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { SW0 }]; #IO_L24N_T3_RS0_15 Sch=sw[0]
13 set property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get ports { SW1 }]; #IO L3N T0 DQS EMCCLK 14 Sch=sw[1]
```

E tre bottoni, di cui quello centrale per il reset del sistema.

```
73 ##Buttons
74 #set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { CPU_RESETN }]; #IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resetn
75 set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { rst }]; #IO_L9P_T1_DQS_14 Sch=btnc
76 #set_property -dict { PACKAGE_PIN M18   IOSTANDARD LVCMOS33 } [get_ports { BTNU }]; #IO_L4N_T0_D05_14 Sch=btnu
77 set_property -dict { PACKAGE_PIN P17   IOSTANDARD LVCMOS33 } [get_ports { BTNL }]; #IO_L12P_T1_MRCC_14 Sch=btnl
78 set property -dict { PACKAGE_PIN M17   IOSTANDARD LVCMOS33 } [get ports { BTNR }]; #IO L10N T1 D15 14 Sch=btnr
```

Timing analysis

Per effettuare la timing analysis abbiamo utilizzato l'apposito tool di report fornito da Vivado; le analisi sono state condotte a valle di sintesi e implementazione per avere delle stime più precise, dato che dopo l'implementazione sono disponibili anche i tempi di routing.
Con il periodo di clock di 10ns (di default), otteniamo uno slack di 4,991 ns.

```
## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { A }]; #IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {A}];
```

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	4,991 ns	Worst Hold Slack (WHS):	0,229 ns	Worst Pulse Width Slack (WPWS):	4,500 ns
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	0,000 ns	Total Pulse Width Negative Slack (TPWS):	0,000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	199	Total Number of Endpoints:	199	Total Number of Endpoints:	76

All user specified timing constraints are met.

Per capire quale è la frequenza massima di funzionamento possiamo ridurre il periodo del primary clock fino a ottenere un WNS negativo.
Possiamo ottenere la frequenza limite come FMAX =1/(T−WNS).
Nel nostro caso, ciò succede per periodi di clock minori o uguali a 3,74 nano-secondi:

```
## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { A }]; #IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 3.74 -waveform {0 1.5} [get_ports {A}];
```

Con questo periodo otteniamo uno slack molto vicino allo zero mentre con periodi inferiori lo slack sarebbe risultato negativo. Per questo assumiamo il limite minimo del periodo a 3,74ns.

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	0,007 ns	Worst Hold Slack (WHS):	0,249 ns	Worst Pulse Width Slack (WPWS):	1,000 ns
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	0,000 ns	Total Pulse Width Negative Slack (TPWS):	0,000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	199	Total Number of Endpoints:	199	Total Number of Endpoints:	76

All user specified timing constraints are met.

Esercizio 4: Shift register

Lo shift register rappresenta una rete sequenziale fondamentale; è composto da una serie di registri (flip-flop) che consentono di memorizzare stringhe di bit e, in corrispondenza di un apposito segnale di abilitazione (shift), i dati memorizzati al loro interno traslano di una posizione o più, verso sinistra o verso destra a seconda della realizzazione del registro a scorrimento.

Esercizio 4.1

Progettare, implementare in VHDL e testare mediante simulazione un registro a scorrimento di N bit in grado di shiftare a destra o a sinistra di un numero Y variabile di posizioni a seconda di una opportuna selezione. In particolare, i valori possibili di Y sono 1 e 2. L'utente tramite selezione deve scegliere di quante posizioni shiftare. Il componente deve essere realizzato utilizzando sia un a) approccio comportamentale sia un b) approccio strutturale.

Nota: il numero di bit del registro deve essere implementato come un *generic*, e dall'esterno deve poter essere scelta la modalità di funzionamento mediante opportuni segnali di selezione.

Progetto e architettura

Lo Shift Register progettato è un grado di operare in 4 configurazioni complessive:

- S=0 , Y=0 : shift da destra verso sinistra di una posizione;
- S=0 , Y=1 : shift da destra verso sinistra di due posizioni;
- S=1 , Y=0 : shift da sinistra verso destra di una posizione;
- S=1 , Y=1 : shift da sinistra verso destra di due posizioni;

È presente un demultiplexer a monte che, sulla base del dato in ingresso e del valore S usato come selezione, attiva l'ingresso del primo flip-flop: Eo per S=0 ed Ei per S=1.

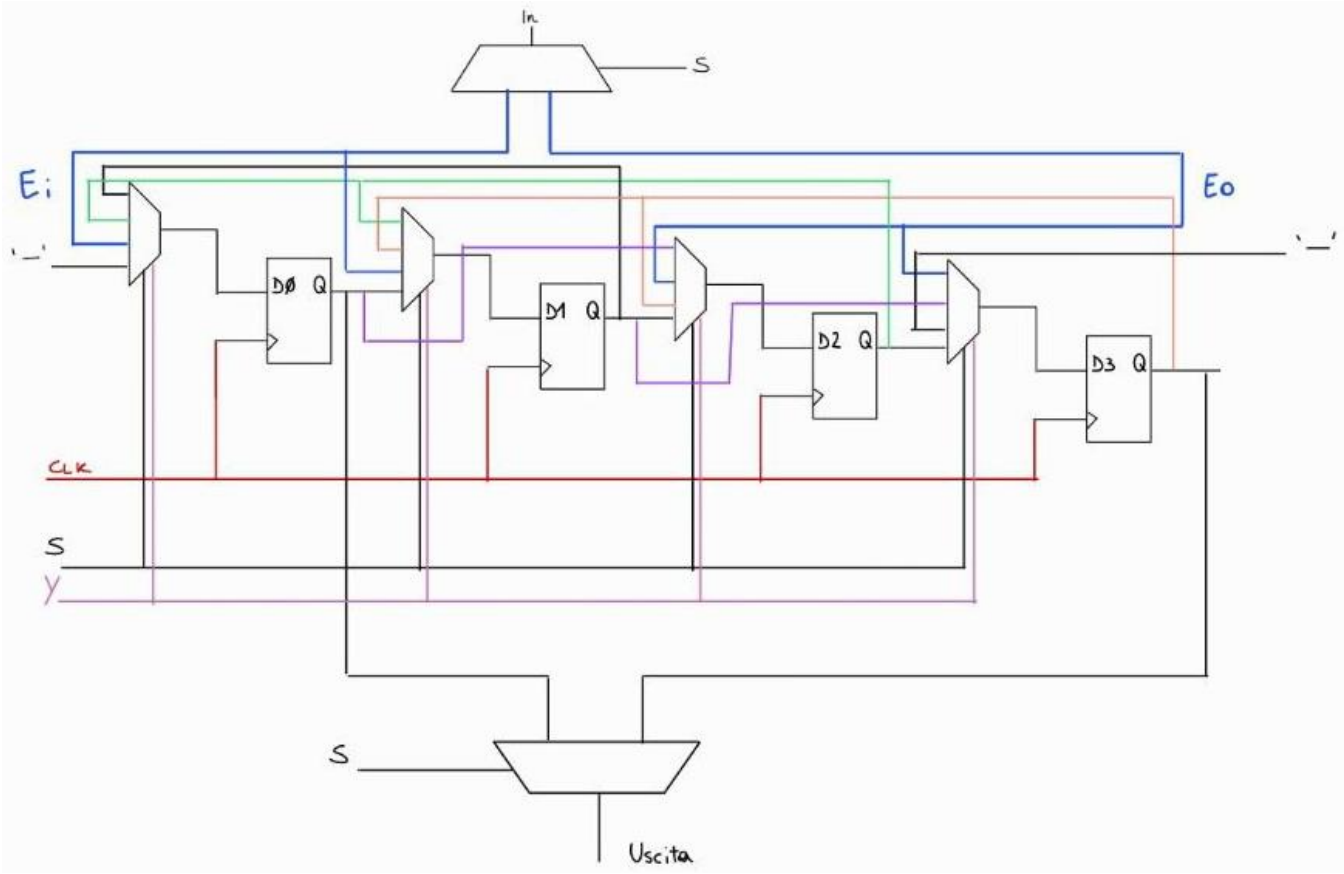
Nel caso in cui Y=1, un ingresso '-' andrà ai flip-flop non coinvolti dallo shift.

Viceversa, il multiplexer a valle del sistema porta in uscita l'uscita del flip-flop all'estrema sinistra per S=0 o l'uscita del flip-flop all'estrema destra per S=1.

Questi due elementi sono fondamentali per la descrizione strutturale dell'architettura.

Il numero N di bit del registro è stato definito come *generic*, l'architettura riportata nella seguente figura presenta il funzionamento con N=4 bit.

Per ogni bit è necessario istanziare un MUX 4:1, i cui segnali di selezione sono S e Y, e un flip-flop di tipo D, il quale riceve in ingresso l'uscita del MUX precedente e il clock.



Implementazione

L'entity dello shift register prevedere un numero di bit N generic, di default posto a 8, un segnale di clock, reset e dataIn in ingresso insieme ai segnali S e Y che definiscono il comportamento del sistema come esposto in precedenza.

Segue l'implementazione dell'architettura strutturale, ottenuta istanziando i singoli componenti: flip-flop di tipo D, multiplexer e demultiplexer, opportunamente connessi tramite segnali temporanei.

```vhdl
-- Entity definition
entity shift_register_N_bit is
    generic(
        N : integer := 8  -- Dimensione del registro
    );
    port(
        CLK, RST, dataIN    : in std_logic;   -- Ingressi
        Y                   : in std_logic;   -- Y = 0 (1 posizione), Y = 1 (2 posizioni)
        S                   : in std_logic;   -- S = 0 (sx <- dx), S = 1 (sx -> dx)
        dataOUT             : out std_logic   -- Uscita
    );
    -- SY = 00 shift a sinistra di 1 posizione
    -- SY = 01 shift a sinistra di 2 posizioni
    -- SY = 10 shift a destra di 1 posizione
    -- SY = 11 shift a destra di 2 posizioni

end shift_register_N_bit;

----Architecture Structural
architecture Structural of shift_register_N_bit is

    component FlipFlop is
        port(
            CLK     : in std_logic;
            RST     : in std_logic;
            D       : in std_logic;
            Q       : out std_logic
        );
    end component;

    component mux_4_1 is
        port(
            dataIN  : in std_logic_vector (3 downto 0);
            s0      : in std_logic;
            s1      : in std_logic;
            dataOUT : out std_logic
        );
    end component;

    component mux_2_1 is
        port(
            dataIN0 : in std_logic;
            dataIN1 : in std_logic;
            s       : in std_logic;
            dataOUT : out std_logic
        );
    end component;

    component demux_1_2 is
        port(
            dataIN     : in std_logic;
            s          : in std_logic;
```

```
75              dataOUT0     : out std_logic;
76              dataOUT1     : out std_logic
77          );
78      end component;
79
80      signal mux4OUT      : std_logic_vector(N - 1 downto 0);
81      signal FFOUT        : std_logic_vector(N - 1 downto 0);
82      signal demux2OUT    : std_logic_vector(1 downto 0);
83
84
85      begin
86
87          NFlipFlop: for i in 0 to N - 1 generate
88              FF: FlipFlop
89              port map(
90                  CLK => CLK,
91                  RST => RST,
92                  D   => mux4OUT(i),
93                  Q   => FFOUT(i)
94              );
95          end generate;
96
97          DEMUX_IN: demux_1_2
98          port map(
99              dataIN      => dataIN,
100             s           => S,
101             dataOUT0    => demux2OUT(0),
102             dataOUT1    => demux2OUT(1)
103         );
104
105         MUX_OUT: mux_2_1
106         port map(
107             dataIN0    => FFOUT(N-1),
108             dataIN1    => FFOUT(0),
109             s          => S,
110             dataOUT    => dataOUT
111         );
112
113         MUX_sx0: mux_4_1
114         port map(
115             dataIN(0) => FFOUT(N-2),
116             dataIN(1) => FFOUT(N-3),
117             dataIN(2) => demux2OUT(1),
118             dataIN(3) => '-',
119             s1 => S,
120             s0 => Y,
121             dataOUT => mux4OUT(N-1)
122         );
123
124         MUX_sx1: mux_4_1
125         port map(
126             dataIN(0) => FFOUT(N-3),
```

```
127             dataIN(1) => FFOUT(N-4),
128             dataIN(2) => FFOUT(N-1),
129             dataIN(3) => demux2OUT(1),
130             s1 => S,
131             s0 => Y,
132             dataOUT => mux4OUT(N-2)
133         );
134 
135         MUX_dx0: mux_4_1
136         port map(
137             dataIN(0) => demux2OUT(0),
138             dataIN(1) => '-',
139             dataIN(2) => FFOUT(1),
140             dataIN(3) => FFOUT(2),
141             s1 => S,
142             s0 => Y,
143             dataOUT => mux4OUT(0)
144         );
145 
146         MUX_dx1: mux_4_1
147         port map(
148             dataIN(0) => FFOUT(0),
149             dataIN(1) => demux2OUT(0),
150             dataIN(2) => FFOUT(2),
151             dataIN(3) => FFOUT(3),
152             s1 => S,
153             s0 => Y,
154             dataOUT => mux4OUT(1)
155         );
156 
157         NMUX: for i in 2 to N - 3 generate
158             MUX4: mux_4_1
159             port map(
160                 dataIN(0) => FFOUT(i-1),
161                 dataIN(1) => FFOUT(i-2),
162                 dataIN(2) => FFOUT(i+1),
163                 dataIN(3) => FFOUT(i+2),
164                 s1 => S,
165                 s0 => Y,
166                 dataOUT => mux4OUT(i)
167             );
168         end generate;
169 end Structural;
```

Dove il singolo Flip Flop è stato descritto nel modo seguente

```vhdl
entity FlipFlop is
    Port (
        CLK     : in std_logic;
        RST     : in std_logic;
        D       : in std_logic;
        Q       : out std_logic
   );
end FlipFlop;

architecture behavioural of FlipFlop is
    begin
        FlipFlop: process (CLK, RST)
        begin
            if (RST = '1') then
                Q <= '0';
            elsif (rising_edge(CLK)) then
                Q <= D;
            end if;
        end process;
    end behavioural;
```

Abbiamo, inoltre, implementato l'architettura *behavioral*, definendo un process sensibile al clock e con reset sincrono. Grazie a un registro a N bit '*tmp*', abbiamo descritto il comportamento del registro a scorrimento prima del caso S=0 e, successivamente, per S=1, collegando, infine, *tmp* all'uscita complessiva del sistema.

```vhdl
--Architecture behavioural
architecture archi of shift_register_N_bit is
    signal tmp: std_logic_vector(N-1 downto 0);  -- Registro a N bit
begin

    process (CLK)
    begin
        if (rising_edge(CLK)) then
            if (RST = '1') then
                tmp <= (others => '0');  -- Reset del registro

            elsif (S = '0') then  -- shift verso sinistra (sx <- dx)
                if (Y = '0') then  -- shift di 1 posizione
                    tmp(0) <= dataIN;
                    for i in N-1 downto 1 loop
                        tmp(i) <= tmp(i-1);
                    end loop;

                elsif (Y = '1') then  -- shift di 2 posizioni
                    tmp(0) <= '-';
                    tmp(1) <= dataIN;
                    for i in N-1 downto 2 loop
                        tmp(i) <= tmp(i-2);
                    end loop;
                end if;
```

```
196
197             elsif (S = '1') then  -- shift verso destra (sx -> dx)
198                 if (Y = '0') then  -- shift di 1 posizione
199                     for i in 0 to N-2 loop
200                         tmp(i) <= tmp(i+1);
201                     end loop;
202                     tmp(N-1) <= dataIN;
203
204                 elsif (Y = '1') then  -- shift di 2 posizioni
205                     for i in 0 to N-3 loop
206                         tmp(i) <= tmp(i+2);
207                     end loop;
208                     tmp(N-2) <= dataIN;
209                     tmp(N-1) <= '-';
210                 end if;
211             end if;
212         end if;
213     end process;
214
215     -- Assegnamento dell'uscita dataOUT
216     with S select
217         dataOUT <=  tmp(N-1)    when '0',  -- Direzione sinistra
218                     tmp(0)      when '1',  -- Direzione destra
219                     '-'         when others;
220
221 end archi;
```

Simulazione

Di seguito è riportato il codice testbench per la simulazione del registro a scorrimento in ogni modalità di funzionamento.

La forma d'onda del clock descritta parte dopo 50ns e la simulazione viene eseguita finché il clock è attivo.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity shift_register_N_bit_TB is
-- Nessuna porta in un testbench
end shift_register_N_bit_TB;

architecture Behavioral of shift_register_N_bit_TB is

    component shift_register_N_bit is
        Generic (
            N : integer := 8
        );
        Port (
            CLK     : in  STD_LOGIC;
            RST     : in  STD_LOGIC;
            Y       : in  STD_LOGIC;
            S       : in  STD_LOGIC;
            dataIN  : in  STD_LOGIC;
            dataOUT : out STD_LOGIC
        );
    end component;

    constant N : integer := 8;

    signal CLK      : STD_LOGIC := '0';
    signal RST      : STD_LOGIC := '0';
    signal Y        : STD_LOGIC := '0';
    signal S        : STD_LOGIC := '0';
    signal dataIN   : STD_LOGIC := '0';
    signal dataOUT  : STD_LOGIC;
    signal stop_the_clock: boolean := false;

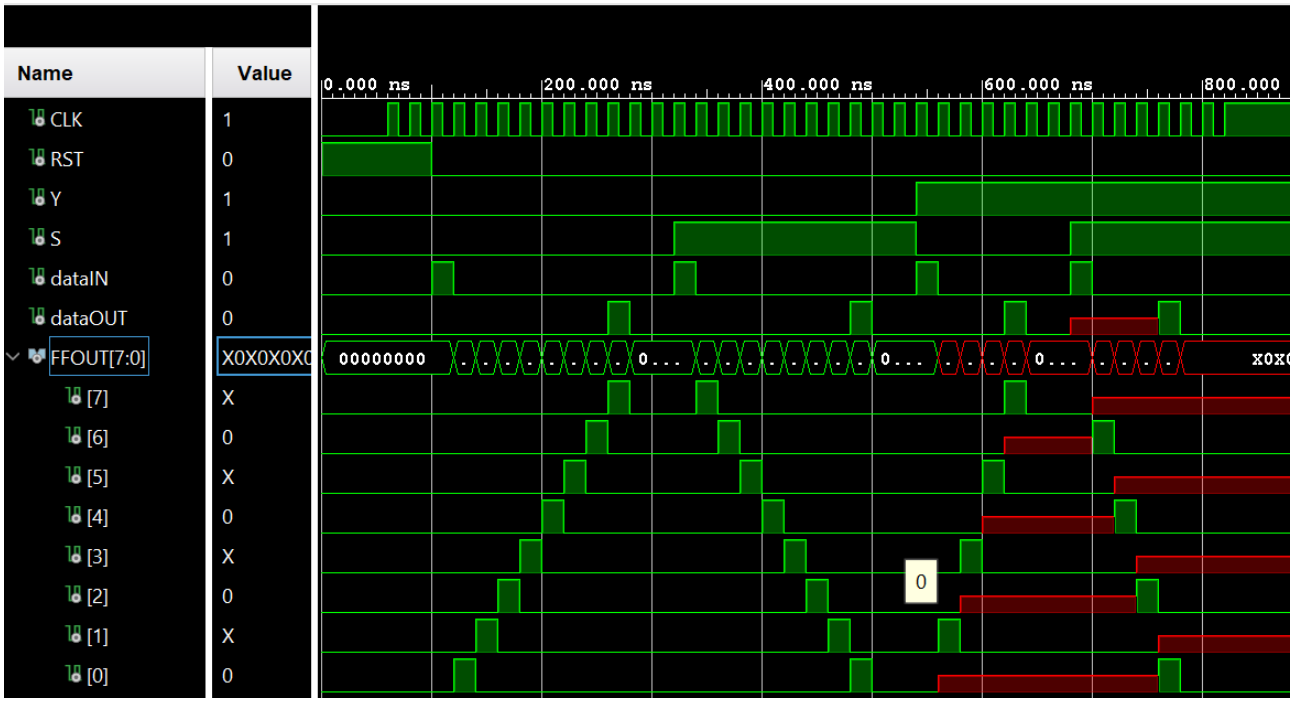
begin

    uut: shift_register_N_bit
        Generic map (
            N => N
        )
        Port map (
            CLK     => CLK,
            RST     => RST,
            Y       => Y,
            S       => S,
            dataIN  => dataIN,
            dataOUT => dataOUT
        );

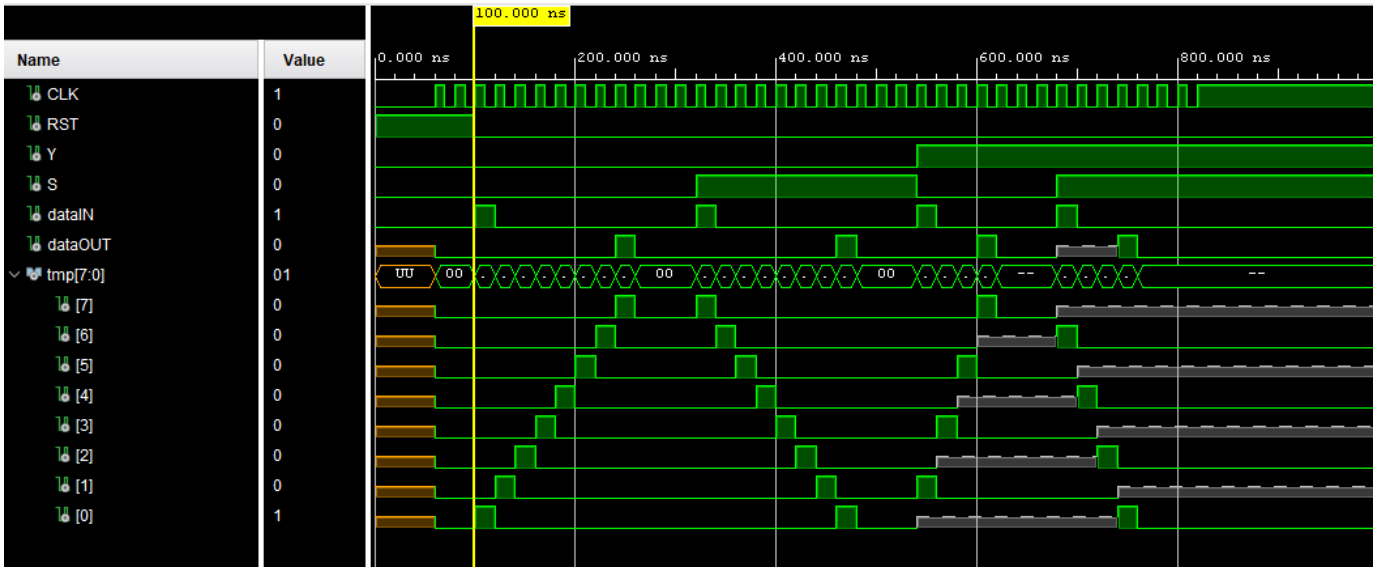
    CLK_process: process
    begin
        wait for 50 ns;
        while not stop_the_clock loop
```

```
53              CLK <= '0';
54              wait for 10 ns;
55              CLK <= '1';
56              wait for 10 ns;
57          end loop;
58          wait;
59      end process;
60
61      stim_proc: process
62      begin
63          -- Reset del registro
64          RST <= '1';
65          wait for 100 ns;
66          RST <= '0';
67
68          -- Stimolo: Shift verso sinistra di 1 posizione (Y = 0, S = 0)
69          S <= '0';
70          Y <= '0';
71          dataIN <= '1';
72          wait for 20 ns;
73          dataIN <= '0';
74          wait for 200 ns;
75
76          -- Stimolo: Shift verso destra di 1 posizione (Y = 0, S = 1)
77          S <= '1';
78          Y <= '0';
79          dataIN <= '1';
80          wait for 20 ns;
81          dataIN <= '0';
82          wait for 200 ns;
83
84          -- Stimolo: Shift verso sinistra di 2 posizioni (Y = 1, S = 0)
85          S <= '0';
86          Y <= '1';
87          dataIN <= '1';
88          wait for 20 ns;
89          dataIN <= '0';
90          wait for 120 ns;
91
92          -- Stimolo: Shift verso destra di 2 posizioni (Y = 1, S = 1)
93          S <= '1';
94          Y <= '1';
95          dataIN <= '1';
96          wait for 20 ns;
97          dataIN <= '0';
98          wait for 120 ns;
99          -- Fine della simulazione
100         stop_the_clock <= true;
101         assert false report "Test completato con successo" severity note;
102         wait;
103     end process;
104 end Behavioral;
```

Risultato Simulazione STRUTTURALE:



Risultato Simulazione COMPORTAMENTALE:



Esercizio 5: Cronometro

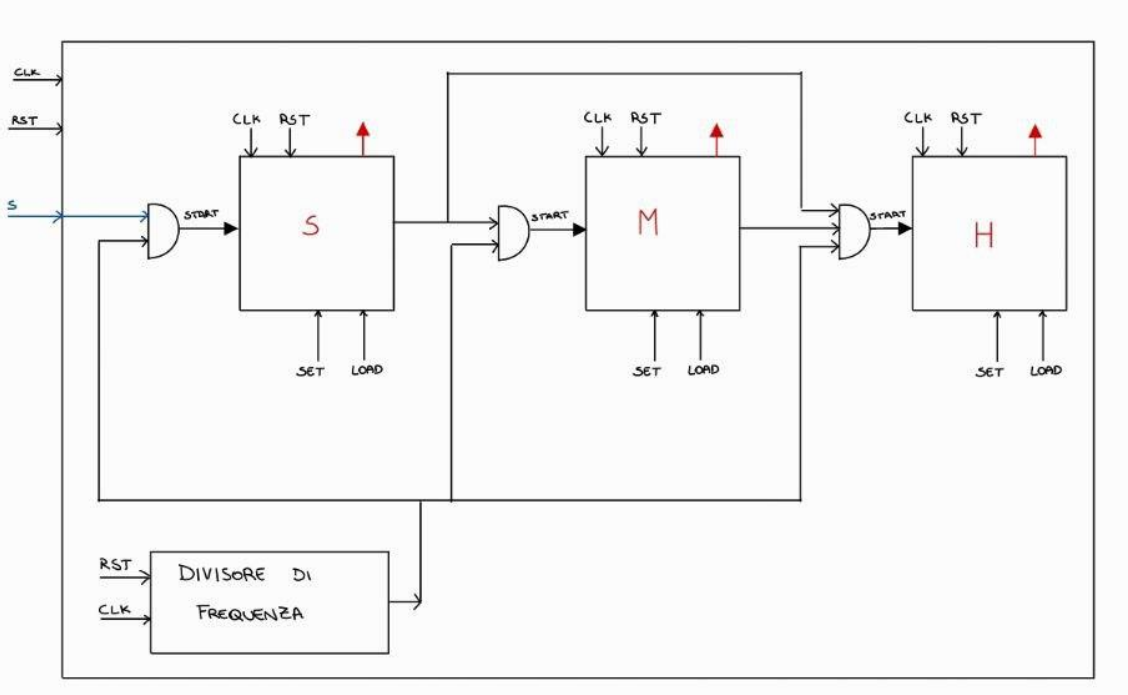
Esercizio 5.1

Progettare, implementare in VHDL e testare mediante simulazione un cronometro, in grado di scandire secondi, minuti e ore a partire da una base dei tempi prefissata (es. si consideri il clock a disposizione sulla board). Il progetto deve prevedere la possibilità di inizializzare il cronometro con un valore iniziale, sempre espresso in termini di ore, minuti e secondi, mediante un opportuno ingresso di set, e deve prevedere un ingresso di reset per azzerare il tempo.

Il componente deve essere realizzato utilizzando un approccio strutturale, collegando opportunamente dei contatori secondo uno schema a scelta.

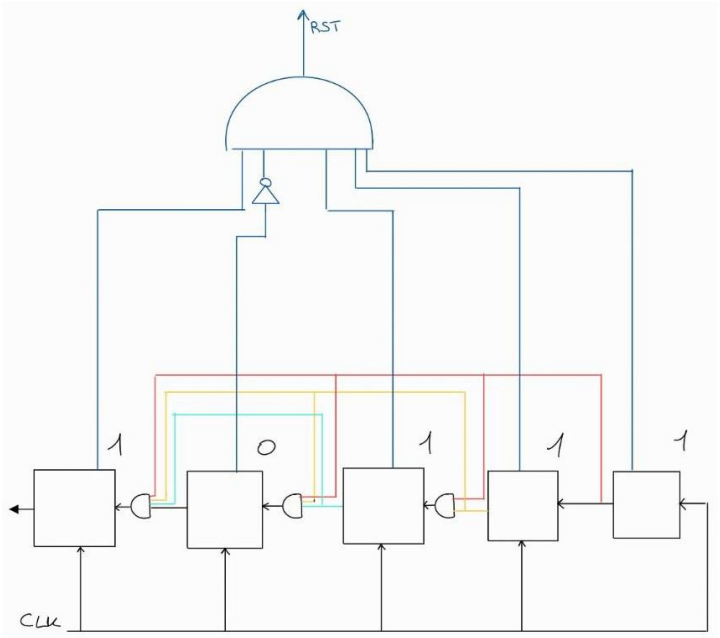
Progetto e architettura

Il cronometro è stato progettato collegando opportunamente tre contatori: 2 moduli 60 per secondi e minuti e un modulo 24 per le ore. Ad ognuno di essi sono stati forniti in ingresso dei segnali di load e set per la possibilità di inizializzare il cronometro con un valore iniziale e un segnale di reset per azzerare il tempo.

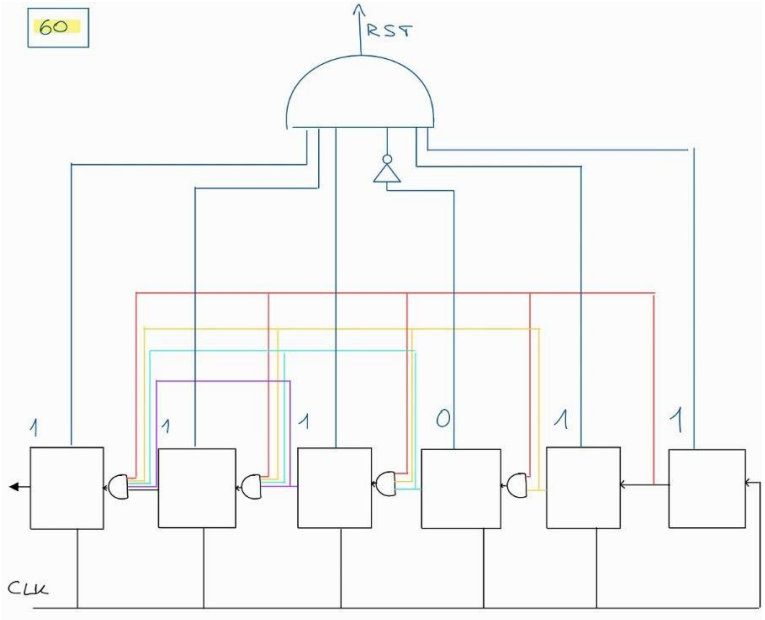


In ingresso a ciascun contatore vi è l'uscita di un divisore di frequenza, fondamentale per adattare il clock della board NEXYS-A7-100T-Master (frequenza di 100MHz) alla frequenza desiderata di 500Hz, in AND con l'uscita del precedente contatore o con lo start nel caso del contatore dei secondi.

È stato progettato il contatore modulo 24 mediante composizione di 5 flip-flop (contatori modulo 2). Un tale contatore è in grado, però, di contare fino a 32 quindi abbiamo ottenuto il contatore modulo 24 per riduzione, alzando un segnale di reset in corrispondenza del 23 (in binario 10111).



Allo stesso modo, il contatore modulo 60 è stato progettato mediante composizione di 6 flip-flop (contatori modulo 2). Un tale contatore è in grado, però, di contare fino a 64 quindi abbiamo ottenuto il contatore modulo 60 per riduzione, alzando un segnale di reset in corrispondenza del 59 (in binario 111011).



Implementazione

Il cronometro è stato implementato in maniera strutturale, collegando in parallelo i tre contatori sopra descritti. Al contatore dei secondi viene fornito un segnale di abilitazione pari alla AND tra il segnale di *start* e l'uscita del divisore di frequenza.

```
entity Crono is
    Port (
        CLK:        in std_logic; --clock
        RST:        in std_logic; --reset
        Secload:       in std_logic; --segnale di set dei secondi
        Minload:       in std_logic; --segnale di set dei minuti
        Hrload:       in std_logic; --segnale di set delle ore
        SecToset:   in std_logic_vector(5 downto 0); --secondi da settare
        MinToset:   in std_logic_vector(5 downto 0); --minuti da settare
        HrToset:    in std_logic_vector(4 downto 0); --ore da settare
        SecOUT:     out std_logic_vector(5 downto 0); --uscita conteggio secondi
        MinOUT:     out std_logic_vector(5 downto 0); --uscita conteggio minuti
        HrOUT:      out std_logic_vector(4 downto 0); --uscita conteggio ore
        start : in std_logic
         );
end Crono;

architecture structural of Crono is

    component clock_filter is
     generic(
            CLKIN_freq : integer := 100000000; --clock board 100MHz
            CLKOUT_freq : integer := 500       --frequenza desiderata
                );
    Port (

          clock_in : in  STD_LOGIC;
          reset : in STD_LOGIC;
          clock_out : out  STD_LOGIC -- attenzione: non è un vero clock ma un impulso che sarà usato come enable
    );
    end component;

    component ContMod60 is
        port(
            CLK:        in std_logic;
            RST:        in std_logic;
            count:      in std_logic;
            load:       in std_logic;
            dataToset:  in std_logic_vector(5 downto 0);
            dataOUT:    out std_logic_vector(5 downto 0);
            Umax:       out std_logic
        );
    end component ContMod60;

    component ContMod24 is
        port(
            CLK:        in std_logic;
            RST:        in std_logic;
            count:      in std_logic;
            load:       in std_logic;
            dataToset:  in std_logic_vector(4 downto 0);
```

```
56              dataOUT:    out std_logic_vector(4 downto 0)
57          );
58      end component ContMod24;
59
60      signal MinClock: std_logic;
61      signal Hrclock: std_logic;
62      signal Umaxsec: std_logic;
63      signal Umaxmin: std_logic;
64      signal SecClock: std_logic;
65      signal abilita: std_logic :='0';
66
67  begin
68
69      abilita <= SecClock AND start;
70
71      Divisore_frequenza: clock_filter
72          generic map(
73              CLKIN_freq => 100000000,
74              CLKOUT_freq => 1
75  --            CLKOUT_freq => 5000000
76          )
77          port map(
78              clock_in => CLK,
79              reset => RST,
80              clock_out => SecClock
81          );
82
83      Counter_sec: ContMod60
84          port map(
85              CLK => CLK,
86              RST => RST,
87              count => abilita,
88              load => Secload,
89              dataToset => SecToset,
90              dataOUT => SecOUT,
91              Umax => Umaxsec
92          );
93
94      Counter_min: ContMod60
95          port map(
96              CLK => CLK,
97              RST => RST,
98              count => MinClock,
99              load => Minload,
100             dataToset => MinToset,
101             dataOUT => MinOUT,
102             Umax => Umaxmin
103         );
104
105     Counter_Hr: ContMod24
106         port map(
107             CLK => CLK,
108             RST => RST,
109             count => HrClock,
110             load => Hrload,
111             dataToset => HrToset,
112             dataOUT => HrOUT
113         );
114
115     --collego in parallelo
116     MinClock <= Umaxsec and CLK;
117     HrClock <= Umaxsec and Umaxmin and CLK;
118
119 end ;
```

Riportiamo l'implementazione del contatore modulo 24, utilizzato per contare le ore.

```
15  entity ContMod24 is
16      Port (
17          CLK:        in std_logic; --clock
18          RST:        in std_logic; --reset
19          count:      in std_logic;
20          load:       in std_logic; --segnale di set
21          dataToset:  in std_logic_vector(4 downto 0); --bit da settare
22          dataOUT:    out std_logic_vector(4 downto 0) --uscita conteggio
23       );
24  end ContMod24;
25
26
27  architecture structural of ContMod24 is
28
29      component ContMod2 is
30          port(
31              CLK:        in std_logic;
32              RST:        in std_logic;
33              count:      in std_logic;
34              reset_count:in std_logic;
35              load:       in std_logic;
36              dataToset:  in std_logic;
37              dataOUT:    out std_logic
38          );
39      end component ContMod2;
41      signal ContOUT: std_logic_vector(4 downto 0); --appoggio uscita
42      signal ContIN:  std_logic_vector(4 downto 0); --appoggio entrata
43      signal reset_temp: std_logic;  --appoggio reset per realizzare mod.24
44
45  begin
46
47      --genero i contatori
48      Parallel_ContMod2: for i in 0 to 4 generate
49          C: ContMod2
50              port map(
51                  CLK         => CLK,
52                  RST         => RST,
53                  count       => ContIN(i),
54                  reset_count => reset_temp,
55                  load        => load,
56                  dataToset   => dataToset(i),
57                  dataOUT     => ContOUT(i)
58              );
59      end generate Parallel_ContMod2;
60
61      -- collego i contatori in parallelo
62      ContIN(0) <= count;
63      ContIN(1) <= ContOUT(0) and count;  --entrata del 2°
64      ContIN(2) <= ContOUT(1) and ContOUT(0) and count;  --entrata del 3°
65      ContIN(3) <= ContOUT(2) and ContOUT(1) and ContOUT(0) and count; --entrata del 4°
```

```
66      ContIN(4) <= ContOUT(3) and ContOUT(2) and ContOUT(1) and ContOUT(0) and count; --entrata del 5°
67
68      --reset conteggio sincrono alzo il segnale quando conta 23
69      reset_temp <= (ContOUT(4) and not ContOUT(3) and ContOUT(2) and
70                     ContOUT(1) and ContOUT(0)) and count;
71
72      -- collego l'uscita del modulo 24 concatenando le singole uscite
73      dataOUT <= ContOUT(4) & ContOUT(3) & ContOUT(2) & ContOUT(1) & ContOUT(0);
74
75  end ;
```

E l'implementazione del contatore modulo 60 per secondi e minuti.

```
 6  entity ContMod60 is
 7      Port (
 8          CLK:        in std_logic; --clock
 9          RST:        in std_logic; --reset asincrono
10          count:      in std_logic;
11          load:       in std_logic; --segnale di load
12          dataToset:  in std_logic_vector(5 downto 0); --bit da settare
13          dataOUT:    out std_logic_vector(5 downto 0); --uscita conteggio
14          Umax:       out std_logic --abilitazione prossimo contatore
15       );
16  end ContMod60;
17
18
19  architecture structural of ContMod60 is
20
21      component ContMod2 is
22          port(
23              CLK:        in std_logic;
24              RST:        in std_logic;
25              count:      in std_logic;
26              reset_count:in std_logic;
27              load:       in std_logic;
28              dataToset:  in std_logic;
29              dataOUT:    out std_logic
30          );
31      end component ContMod2;
32
33      signal ContOUT: std_logic_vector(5 downto 0); --appoggio uscita
34      signal ContIN:  std_logic_vector(5 downto 0); --appoggio entrata
35      signal reset_temp: std_logic;  --appoggio reset per realizzare mod.60
36
37  begin
38
39      --genero i contatori
40      Parallel_ContMod2: for i in 0 to 5 generate
41          C: ContMod2
42              port map(
43                  CLK         => CLK,
44                  RST         => RST,
45                  count       => ContIN(i),
46                  reset_count => reset_temp,
47                  load        => load,
48                  dataToset   => dataToset(i),
49                  dataOUT     => ContOUT(i)
50              );
51      end generate Parallel_ContMod2;
52
53      -- collego i contatori in parallelo
54      ContIN(0) <= count; --entrata del 1°
55      ContIN(1) <= ContOUT(0) and count;  --entrata del 2°
```

```
56      ContIN(2) <= ContOUT(1) and ContOUT(0) and count;  --entrata del 3°
57      ContIN(3) <= ContOUT(2) and ContOUT(1) and ContOUT(0) and count; --entrata del 4°
58      ContIN(4) <= ContOUT(3) and ContOUT(2) and ContOUT(1) and ContOUT(0) and count; --entrata del 5°
59      ContIN(5) <= ContOUT(4) and ContOUT(3) and ContOUT(2) and ContOUT(1) and ContOUT(0) and count; --entrata del 6°
60
61       --reset conteggio sincrono alzo il segnale quando conta 59
62      reset_temp <= (ContOUT(5) and ContOUT(4) and ContOUT(3) and not ContOUT(2)
63                     and ContOUT(1) and ContOUT(0)) and count;
64
65      --quando conta 59 attivo il conteggio del prossimo orologio
66      Umax <= (ContOUT(5) and ContOUT(4) and ContOUT(3) and not ContOUT(2)
67                     and ContOUT(1) and ContOUT(0)) and count;
68
69      -- collego l'uscita
70      dataOUT <= ContOUT;
71
72  end ;
```

Simulazione

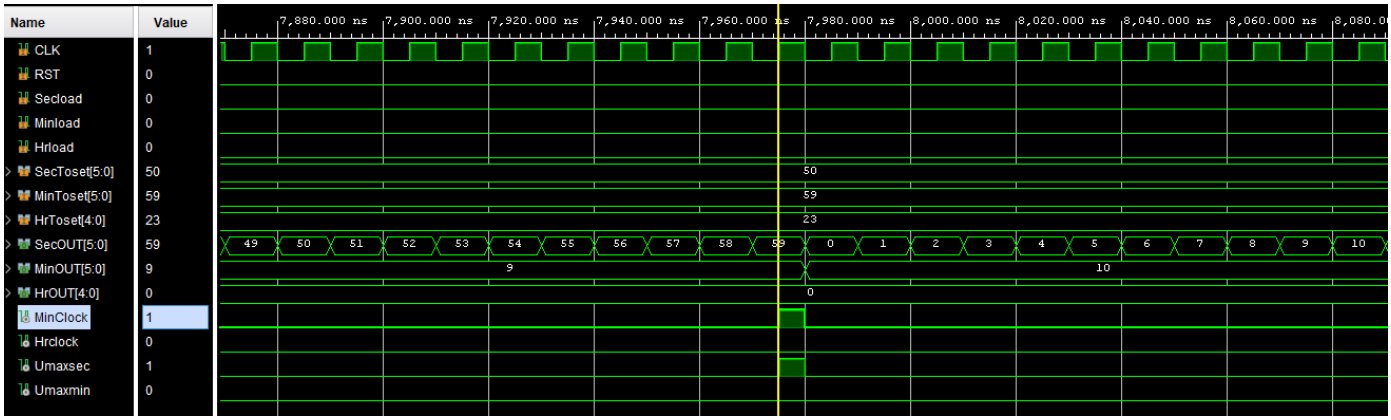
Di seguito è riportato il codice testbench per la simulazione del cronometro.

```
26  ENTITY Crono_tb IS
27  END Crono_tb;
28
29  ARCHITECTURE behavior OF Crono_tb IS
30
31      -- Component under test (CUT)
32      COMPONENT Crono
33          PORT (
34              CLK:        IN std_logic;
35              RST:        IN std_logic;
36              Secload:    IN std_logic;
37              Minload:    IN std_logic;
38              Hrload:     IN std_logic;
39              SecToset:   IN std_logic_vector(5 downto 0);
40              MinToset:   IN std_logic_vector(5 downto 0);
41              HrToset:    IN std_logic_vector(4 downto 0);
42              SecOUT:     OUT std_logic_vector(5 downto 0);
43              MinOUT:     OUT std_logic_vector(5 downto 0);
44              HrOUT:      OUT std_logic_vector(4 downto 0)
45          );
46      END COMPONENT;
47
48      -- Signals for testbench
49      SIGNAL CLK_tb:      std_logic := '0';
50      SIGNAL RST_tb:      std_logic := '0';
```

```
51      SIGNAL Secload_tb:  std_logic := '0';
52      SIGNAL Minload_tb:  std_logic := '0';
53      SIGNAL Hrload_tb:   std_logic := '0';
54      SIGNAL SecToset_tb: std_logic_vector(5 downto 0) := (others => '0');
55      SIGNAL MinToset_tb: std_logic_vector(5 downto 0) := (others => '0');
56      SIGNAL HrToset_tb:  std_logic_vector(4 downto 0) := (others => '0');
57      SIGNAL SecOUT_tb:   std_logic_vector(5 downto 0);
58      SIGNAL MinOUT_tb:   std_logic_vector(5 downto 0);
59      SIGNAL HrOUT_tb:    std_logic_vector(4 downto 0);
60
61      -- Clock period (simuliamo un clock rapido per velocizzare il test)
62      CONSTANT CLK_period : time := 10 ns;
63
64  BEGIN
65
66      -- Instanziazione del componente Crono
67      uut: Crono
68          PORT MAP (
69              CLK       => CLK_tb,
70              RST       => RST_tb,
71              Secload   => Secload_tb,
72              Minload   => Minload_tb,
73              Hrload    => Hrload_tb,
74              SecToset  => SecToset_tb,
75              MinToset  => MinToset_tb,
76              HrToset   => HrToset_tb,
77              SecOUT    => SecOUT_tb,
78              MinOUT    => MinOUT_tb,
79              HrOUT     => HrOUT_tb
80          );
81
82      -- Clock process
83      clk_process: PROCESS
84      BEGIN
85          WHILE NOW < 10000 ns LOOP  -- Simulazione fino a 10.000 ns
86              CLK_tb <= '0';
87              WAIT FOR CLK_period / 2;
88              CLK_tb <= '1';
89              WAIT FOR CLK_period / 2;
90          END LOOP;
91          WAIT;
92      END PROCESS;
93
94      -- Stimulus process
95      stim_proc: PROCESS
96      BEGIN
97          -- **1 RESET iniziale**
98          RST_tb <= '1';
99          WAIT FOR 20 ns;
100         RST tb <= '0';
```

```
101         WAIT FOR 20 ns;
102 
103         -- **2 Conteggio naturale fino a 23:59:59 ? 00:00:00**
104         WAIT FOR 600 ns; -- Aspettiamo il passaggio del tempo
105 
106         -- **3 Settaggio manuale: 23:45:50**
107         Secload_tb <= '1';
108         SecToset_tb <= "110010";  -- 50 in binario
109         WAIT FOR 10 ns;
110         Secload_tb <= '0';
111 
112         Minload_tb <= '1';
113         MinToset_tb <= "101101";  -- 45 in binario
114         WAIT FOR 10 ns;
115         Minload_tb <= '0';
116 
117         Hrload_tb <= '1';
118         HrToset_tb <= "10111";    -- 23 in binario
119         WAIT FOR 10 ns;
120         Hrload_tb <= '0';
121 
122         -- **4 Aspettiamo il passaggio 23:59:59 ? 00:00:00**
123         WAIT FOR 600 ns;
125       -- **5 Reset del sistema**
126       RST_tb <= '1';
127       WAIT FOR 10 ns;
128       RST_tb <= '0';
129 
130       WAIT FOR 600 ns;
131 
132       Secload_tb <= '1';
133       SecToset_tb <= "110010";  -- 50 in binario
134       WAIT FOR 10 ns;
135       Secload_tb <= '0';
136 
137       Minload_tb <= '1';
138       MinToset_tb <= "111011";  -- 59 in binario
139       WAIT FOR 10 ns;
140       Minload_tb <= '0';
141 
142       Hrload_tb <= '1';
143       HrToset_tb <= "10111";    -- 23 in binario
144       WAIT FOR 10 ns;
145       Hrload_tb <= '0';
146 
147       -- **6 Verifica che il conteggio riparta da 00:00:00**
148       WAIT FOR 600 ns;
149 
150         -- Fine test
```

Riportiamo, infine, i risultati della simulazione.

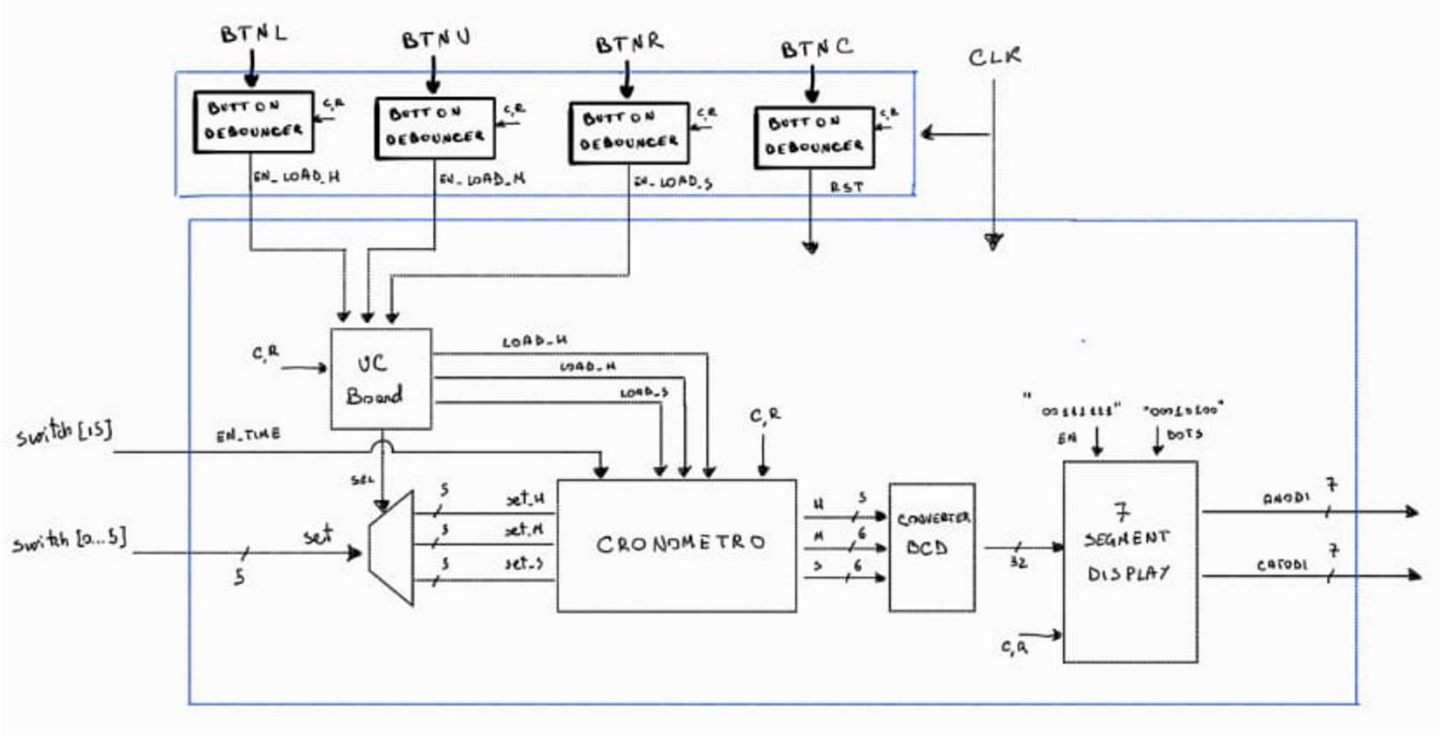


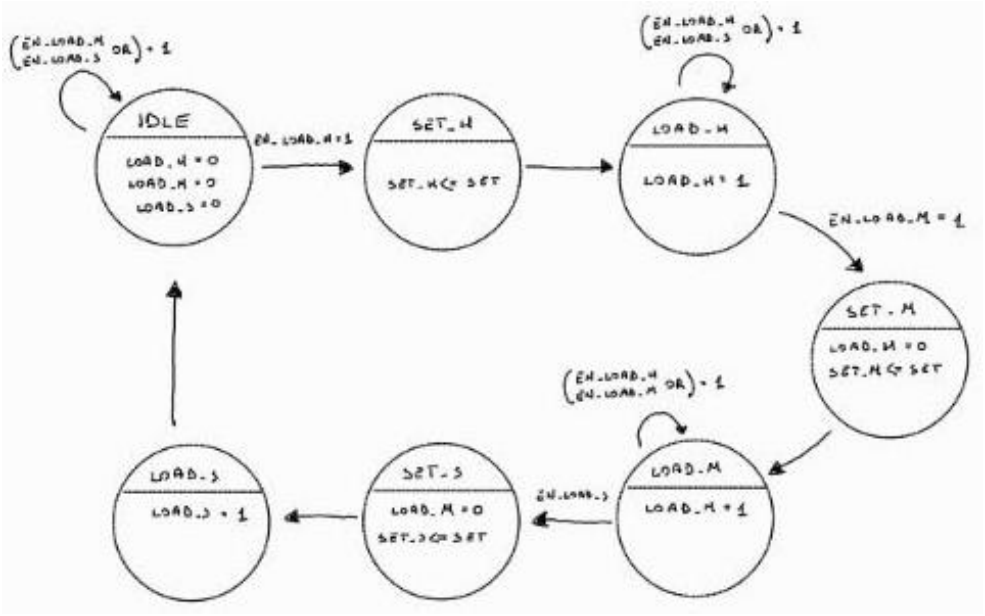
Esercizio 5.2

Sintetizzare ed implementare su board il componente sviluppato al punto precedente, utilizzando i display a 7 segmenti per la visualizzazione dell'orario (o una combinazione di display e led nel caso in cui i display a disposizione siano in numero inferiore a quello necessario), gli switch per l'immissione dell'orario iniziale e due bottoni, uno per il set dell'orario e uno per il reset. Si utilizzi una codifica a scelta dello studente per la visualizzazione dell'orario sui display (esadecimale o decimale).

Sintesi su board di sviluppo

L'esercizio è stato progettato come nel seguente schema a blocchi e il comportamento per l'acquisisizione sequenziale degli operandi è descritto nel successivo automa a stati finiti.





Di seguito mostriamo l'implementazione dell'unità operativa composta dal display, da un convertitore e dal cronometro stesso.

```
6  entity UO is
7      Port (
8          clk : in STD_LOGIC;
9          rst : in STD_LOGIC;
10         en_time : in STD_LOGIC;
11         load_sec : in STD_LOGIC;
12         load_min : in STD_LOGIC;
13         load_hours : in STD_LOGIC;
14         set_sec : in STD_LOGIC_VECTOR(5 downto 0);
15         set_min : in STD_LOGIC_VECTOR(5 downto 0);
16         set_hours : in STD_LOGIC_VECTOR(4 downto 0);
17         anodes_out : out STD_LOGIC_VECTOR (7 downto 0);
18         cathodes_out : out STD_LOGIC_VECTOR(7 downto 0));
19 end UO;
20
21 architecture structural of UO is
22     component Crono
23     Port (
24         CLK:        in std_logic;
25         RST:        in std_logic;
26         Secload:    in std_logic;
27         Minload:    in std_logic;
28         Hrload:     in std_logic;
29         SecToset:   in std_logic_vector(5 downto 0);
30         MinToset:   in std_logic_vector(5 downto 0);
```

```
31          HrToset:    in std_logic_vector(4 downto 0);
32          SecOUT:     out std_logic_vector(5 downto 0);
33          MinOUT:     out std_logic_vector(5 downto 0);
34          HrOUT:      out std_logic_vector(4 downto 0);
35          start:      in std_logic
36           );
37      end component;
38
39      component display_seven_segments
40      Generic(
41          CLKIN_freq : integer := 100000000;
42          CLKOUT_freq : integer := 500
43      );
44      Port ( CLK : in  STD_LOGIC;
45             RST : in  STD_LOGIC;
46             VALUE : in  STD_LOGIC_VECTOR (31 downto 0) :=(others => '0');
47             ENABLE : in  STD_LOGIC_VECTOR (7 downto 0);
48             DOTS : in  STD_LOGIC_VECTOR (7 downto 0);
49             ANODES : out  STD_LOGIC_VECTOR (7 downto 0);
50             CATHODES : out  STD_LOGIC_VECTOR (7 downto 0));
51      end component;
52
53      component BCDConverter is
54          port(
55              x : in std_logic_vector(5 downto 0);
56              y : out std_logic_vector(7 downto 0));
57      end component;
58
59      SIGNAL temp_second : STD_LOGIC_VECTOR(5 downto 0);
60      SIGNAL temp_min : STD_LOGIC_VECTOR(5 downto 0);
61      SIGNAL temp_hours : STD_LOGIC_VECTOR(5 downto 0);
62      SIGNAL temp_hours_out : STD_LOGIC_VECTOR(4 downto 0);
63      SIGNAL orario_BCD : STD_LOGIC_VECTOR(31 downto 0);
64
65
66  begin
67
68  temp_hours <= "0" & temp_hours_out;
69
70  Cronometro: Crono
71      Port map (
72          CLK => CLK,
73          RST => rst,
74          Secload => load_sec,
75          Minload => load_min,
76          Hrload => load_hours,
77          SecToset => set_sec,
78          MinToset => set_min,
79          HrToset => set_hours,
80          SecOUT => temp_second,
```

```
81          MinOUT => temp_min,
82          HrOUT => temp_hours_out,
83          start => en_time
84      );
85
86  converter_sec: BCDConverter
87      Port map (
88          x => temp_second,
89          y => orario_BCD(7 downto 0)
90      );
91
92  converter_min: BCDConverter
93      Port map (
94          x => temp_min,
95          y => orario_BCD(15 downto 8)
96      );
97
98  converter_hours: BCDConverter
99      Port map (
100         x => temp_hours,
101         y => orario_BCD(23 downto 16)
102     );
103
104 Display: display_seven_segments
105     Generic map(
106         CLKIN_freq => 100000000,
107         CLKOUT_freq => 500
108     )
109     Port map(
110         CLK => CLK,
111         RST => rst,
112         VALUE => orario_BCD,
113         ENABLE => "00111111",
114         DOTS => "00010100",
115         ANODES => anodes_out,
116         CATHODES => cathodes_out
117     );
118
119 end structural;
```

Nel modulo che descrive il sistema complessivo abbiamo quindi inserito un process che modella l'automa a stati finiti per l'acquisizione degli operandi e la parte operativa precedentemente descritta.

```
6  entity sistema is
7      Port(
8          clk : in STD_LOGIC;
9          rst : in STD_LOGIC;
10         en_time : in STD_LOGIC;
11         load_sec : in STD_LOGIC;
12         load_min : in STD_LOGIC;
13         load_hours : in STD_LOGIC;
14         set : in STD_LOGIC_VECTOR(5 downto 0);
15         anodes_out : out STD_LOGIC_VECTOR (7 downto 0);
16         cathodes_out : out STD_LOGIC_VECTOR(7 downto 0));
17 end sistema;
18
19 architecture Behavioral of sistema is
20
21
22 component UO is
23     Port (
24         clk : in STD_LOGIC;
25         rst : in STD_LOGIC;
26         en_time : in STD_LOGIC;
27         load_sec : in STD_LOGIC;
28         load_min : in STD_LOGIC;
29         load_hours : in STD_LOGIC;
30         set_sec : in STD_LOGIC_VECTOR(5 downto 0);
31         set_min : in STD_LOGIC_VECTOR(5 downto 0);
32         set_hours : in STD_LOGIC_VECTOR(4 downto 0);
33         anodes_out : out STD_LOGIC_VECTOR (7 downto 0);
34         cathodes_out : out STD_LOGIC_VECTOR(7 downto 0));
35 end component;
36
37     --stati e comandi della UC
38     type stato is (IDLE, SET_S, SET_M, SET_H, LOAD_H, LOAD_M, LOAD_S);
39     signal stato_corrente : stato := idle;
40     signal stato_prossimo : stato := stato_corrente;
41
42     SIGNAL temp_set_hours : STD_LOGIC_VECTOR(4 downto 0);
43     SIGNAL temp_set_min : STD_LOGIC_VECTOR(5 downto 0);
44     SIGNAL temp_set_sec : STD_LOGIC_VECTOR(5 downto 0);
45     SIGNAL temp_load_sec : std_logic := '0';
46     SIGNAL temp_load_min : std_logic := '0';
47     SIGNAL temp_load_hours : std_logic := '0';
48
49
50 begin
51
52     --unità operativa
53     crono_display : UO
54         Port map(
55             clk => clk,
```

```
56              rst => rst,
57              en_time => en_time,
58              load_sec => temp_load_sec,
59              load_min => temp_load_min,
60              load_hours => temp_load_hours,
61              set_sec => temp_set_sec,
62              set_min => temp_set_min,
63              set_hours => temp_set_hours,
64              anodes_out => anodes_out,
65              cathodes_out => cathodes_out
66          );
67
68      --unità di controllo
69      mem : process (clk, rst)
70          begin
71              if (rst = '1') then
72                 stato_corrente <= Idle;
73
74              elsif (rising_edge(clk)) then
75                  stato_corrente <= stato_prossimo;
76              end if;
77          end process;
78
79      comb: process(stato_corrente, load_sec, load_min, load_hours)
80          begin
82           case stato_corrente is
83               when IDLE =>
84                   temp_load_sec <= '0';
85                   temp_load_min <= '0';
86                   temp_load_hours <= '0';
87
88                   if (load_hours = '1') then
89                       stato_prossimo <= SET_H;
90                   else
91                       stato_prossimo <= IDLE;
92                   end if;
93
94               when SET_H  =>
95                   temp_set_hours <= set(4 downto 0);
96                   stato_prossimo <= LOAD_H;
97
98               when LOAD_H  =>
99                   temp_load_hours <= '1';
100
101                  if (load_min = '1') then
102                      stato_prossimo <= SET_M;
103                  else
104                      stato_prossimo <= LOAD_H;
105                  end if;
```

```
107             when SET_M  =>
108                 temp_load_hours <= '0';
109                 temp_set_min <= set;
110                 stato_prossimo <= LOAD_M;
111
112             when LOAD_M  =>
113                 temp_load_min <= '1';
114
115                 if (load_sec = '1') then
116                     stato_prossimo <= SET_S;
117                 else
118                     stato_prossimo <= LOAD_M;
119                 end if;
120
121             when SET_S  =>
122                 temp_load_min <= '0';
123                 temp_set_sec <= set;
124                 stato_prossimo <= LOAD_S;
125
126             when LOAD_S  =>
127                 temp_load_sec <= '1';
128                 stato_prossimo <= IDLE;
129
130             when OTHERS =>
131                 stato_prossimo <= IDLE;
132
133         end case;
134
135 end process;
136
137
138
139 end Behavioral;
```

Inoltre, è stata implementata un'interfaccia in cui sono stati istanziati i ButtonDebouncer per collegare la pressione dei bottoni della board ai segnali.

```
entity interfaccia is
    Port(
        clk : in STD_LOGIC;
        rst : in STD_LOGIC;
        load_sec : in STD_LOGIC;
        load_min : in STD_LOGIC;
        load_hours : in STD_LOGIC;
        cleared_reset : out STD_LOGIC;
        cleared_load_sec : out STD_LOGIC;
        cleared_load_min : out STD_LOGIC;
        cleared_load_hours : out STD_LOGIC
    );
end interfaccia;

architecture structural of interfaccia is

    component ButtonDebouncer
        generic (
            CLK_period: integer := 10;
            btn_noise_time: integer := 10000000
        );
        Port (
            RST : in STD_LOGIC;
            CLK : in STD_LOGIC;
            BTN : in STD_LOGIC;

            CLEARED_BTN : out STD_LOGIC
        );
    end component;

begin

    BTN_reset : ButtonDebouncer
        Generic map(
            CLK_period => 10,
            btn_noise_time => 10000000
        )
        Port map( RST => '0',
            CLK => CLK,
            BTN => RST,
            CLEARED_BTN => CLEARED_RESET
        );

    BTN_loadsec : ButtonDebouncer
        Generic map(
            CLK_period => 10,
            btn_noise_time => 10000000
        )
        Port map( RST =>'0',
            CLK => CLK,
            BTN => LOAD_SEC,
```

```
            CLEARED_BTN => CLEARED_LOAD_SEC
        );

    BTN_loadmin : ButtonDebouncer
        Generic map(
            CLK_period => 10,
            btn_noise_time => 10000000
        )
        Port map( RST => '0',
            CLK => CLK,
            BTN => LOAD_MIN,
            CLEARED_BTN => CLEARED_LOAD_MIN
        );

    BTN_loadhours : ButtonDebouncer
        Generic map(
            CLK_period => 10,
            btn_noise_time => 10000000
        )
        Port map( RST => '0',
            CLK => CLK,
            BTN => LOAD_HOURS,
            CLEARED_BTN => CLEARED_LOAD_HOURS
        );
```

Il sistema complessivo è stato implementato con un'architettura strutturale composta da *sistema* e *interfaccia* visti poc'anzi.

```
entity crono_on_board is
    Port(
        clk : in STD_LOGIC;
        rst : in STD_LOGIC;
        start_stop : in STD_LOGIC;
        load_sec : in STD_LOGIC;
        load_min : in STD_LOGIC;
        load_hours : in STD_LOGIC;
        set : in STD_LOGIC_VECTOR(5 downto 0);
        anodes_out : out STD_LOGIC_VECTOR (7 downto 0);
        cathodes_out : out STD_LOGIC_VECTOR(7 downto 0)
        );
end crono_on_board;

architecture structural of crono_on_board is

    component sistema is
        Port(
            clk : in STD_LOGIC;
            rst : in STD_LOGIC;
            en_time : in STD_LOGIC;
            load_sec : in STD_LOGIC;
            load_min : in STD_LOGIC;
            load_hours : in STD_LOGIC;
            set : in STD_LOGIC_VECTOR(5 downto 0);
```

```
            anodes_out : out STD_LOGIC_VECTOR (7 downto 0);
            cathodes_out : out STD_LOGIC_VECTOR(7 downto 0)
        );
    end component;

    component interfaccia is
        Port(
            clk : in STD_LOGIC;
            rst : in STD_LOGIC;
            load_sec : in STD_LOGIC;
            load_min : in STD_LOGIC;
            load_hours : in STD_LOGIC;
            cleared_reset : out STD_LOGIC;
            cleared_load_sec : out STD_LOGIC;
            cleared_load_min : out STD_LOGIC;
            cleared_load_hours : out STD_LOGIC
        );
    end component;

    SIGNAL temp_cleared_reset : STD_LOGIC;
    SIGNAL temp_cleared_load_sec : STD_LOGIC;
    SIGNAL temp_cleared_load_min : STD_LOGIC;
    SIGNAL temp_cleared_load_hours : STD_LOGIC;

begin

    BTN_interface: interfaccia
       Port map(
           clk => clk,
           rst => rst,
           load_sec => load_sec,
           load_min => load_min,
           load_hours => load_hours,
           cleared_reset => temp_cleared_reset,
           cleared_load_sec => temp_cleared_load_sec,
           cleared_load_min => temp_cleared_load_min,
           cleared_load_hours => temp_cleared_load_hours
       );

   crono_system : sistema
       Port map(
           clk => clk,
           rst => temp_cleared_reset,
           en_time => start_stop,
           load_sec => temp_cleared_load_sec,
           load_min => temp_cleared_load_min,
           load_hours => temp_cleared_load_hours,
           set => set,
           anodes_out => anodes_out,
           cathodes_out => cathodes_out
       );
```

Il file dei constraints della board NEXYS-A7-100T-Master è stato modificato come in figura.

```
## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clk }]; #IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { clk }];


##Switches
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { set[0] }]; #IO_L24N_T3_RS0_15 Sch=sw[0]
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { set[1] }]; #IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { set[2] }]; #IO_L6N_T0_D08_VREF_14 Sch=sw[2]
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { set[3] }]; #IO_L13N_T2_MRCC_14 Sch=sw[3]
set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { set[4] }]; #IO_L12N_T1_MRCC_14 Sch=sw[4]
set_property -dict { PACKAGE_PIN T18   IOSTANDARD LVCMOS33 } [get_ports { set[5] }]; #IO_L7N_T1_D10_14 Sch=sw[5]
```

Si utilizzano i primi 6 switches della board per il set dell'orario.

```
#set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { SW[14] }]; #IO_L19N_T3_A09_D25_VREF_14 Sch=
set_property -dict { PACKAGE_PIN V10   IOSTANDARD LVCMOS33 } [get_ports { start_stop }]; #IO_L21P_T3_DQS_14 Sch=sw[15]

## LEDs
#set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { LED[0] }]; #IO_L18P_T2_A24_15 Sch=led[0]
```

E l'ultimo switch come start/stop del cronometro.

```
##7 segment display
set_property -dict { PACKAGE_PIN T10   IOSTANDARD LVCMOS33 } [get_ports { cathodes_out[0] }]; #IO_L24N_T3_A00_D16_14 Sch=ca
set_property -dict { PACKAGE_PIN R10   IOSTANDARD LVCMOS33 } [get_ports { cathodes_out[1] }]; #IO_25_14 Sch=cb
set_property -dict { PACKAGE_PIN K16   IOSTANDARD LVCMOS33 } [get_ports { cathodes_out[2] }]; #IO_25_15 Sch=cc
set_property -dict { PACKAGE_PIN K13   IOSTANDARD LVCMOS33 } [get_ports { cathodes_out[3] }]; #IO_L17P_T2_A26_15 Sch=cd
set_property -dict { PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { cathodes_out[4] }]; #IO_L13P_T2_MRCC_14 Sch=ce
set_property -dict { PACKAGE_PIN T11   IOSTANDARD LVCMOS33 } [get_ports { cathodes_out[5] }]; #IO_L19P_T3_A10_D26_14 Sch=cf
set_property -dict { PACKAGE_PIN L18   IOSTANDARD LVCMOS33 } [get_ports { cathodes_out[6] }]; #IO_L4P_T0_D04_14 Sch=cg
set_property -dict { PACKAGE_PIN H15   IOSTANDARD LVCMOS33 } [get_ports { cathodes_out[7] }]; #IO_L19N_T3_A21_VREF_15 Sch=dp
set_property -dict { PACKAGE_PIN J17   IOSTANDARD LVCMOS33 } [get_ports { anodes_out[0] }]; #IO_L23P_T3_FOE_B_15 Sch=an[0]
set_property -dict { PACKAGE_PIN J18   IOSTANDARD LVCMOS33 } [get_ports { anodes_out[1] }]; #IO_L23N_T3_FWE_B_15 Sch=an[1]
set_property -dict { PACKAGE_PIN T9    IOSTANDARD LVCMOS33 } [get_ports { anodes_out[2] }]; #IO_L24P_T3_A01_D17_14 Sch=an[2]
set_property -dict { PACKAGE_PIN J14   IOSTANDARD LVCMOS33 } [get_ports { anodes_out[3] }]; #IO_L19P_T3_A22_15 Sch=an[3]
set_property -dict { PACKAGE_PIN P14   IOSTANDARD LVCMOS33 } [get_ports { anodes_out[4] }]; #IO_L8N_T1_D12_14 Sch=an[4]
set_property -dict { PACKAGE_PIN T14   IOSTANDARD LVCMOS33 } [get_ports { anodes_out[5] }]; #IO_L14P_T2_SRCC_14 Sch=an[5]
set_property -dict { PACKAGE_PIN K2    IOSTANDARD LVCMOS33 } [get_ports { anodes_out[6] }]; #IO_L23P_T3_35 Sch=an[6]
set_property -dict { PACKAGE_PIN U13   IOSTANDARD LVCMOS33 } [get_ports { anodes_out[7] }]; #IO_L23N_T3_A02_D18_14 Sch=an[7]

##Buttons
#set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { CPU_RESETN }]; #IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resetn
set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { rst }]; #IO_L9P_T1_DQS_14 Sch=btnc
set_property -dict { PACKAGE_PIN M18   IOSTANDARD LVCMOS33 } [get_ports { load_min }]; #IO_L4N_T0_D05_14 Sch=btnu
set_property -dict { PACKAGE_PIN P17   IOSTANDARD LVCMOS33 } [get_ports { load_hours }]; #IO_L12P_T1_MRCC_14 Sch=btnl
set_property -dict { PACKAGE_PIN M17   IOSTANDARD LVCMOS33 } [get_ports { load_sec }]; #IO_L10N_T1_D15_14 Sch=btnr
#set_property -dict { PACKAGE_PIN P18   IOSTANDARD LVCMOS33 } [get_ports { BTND }]; #IO_L9N_T1_DQS_D13_14 Sch=btnd
```

Si utilizza il display a 7 segmenti e quattro bottoni:

- Il centrale per il reset;
- Quello superiore per il load dei minuti;
- Quello di sinistra per il load dei secondi;
- Quello di destra per il load dei secondi.

Esercizio 6: Sistema di lettura-elaborazione-scrittura PO_PC

Esercizio 6.1

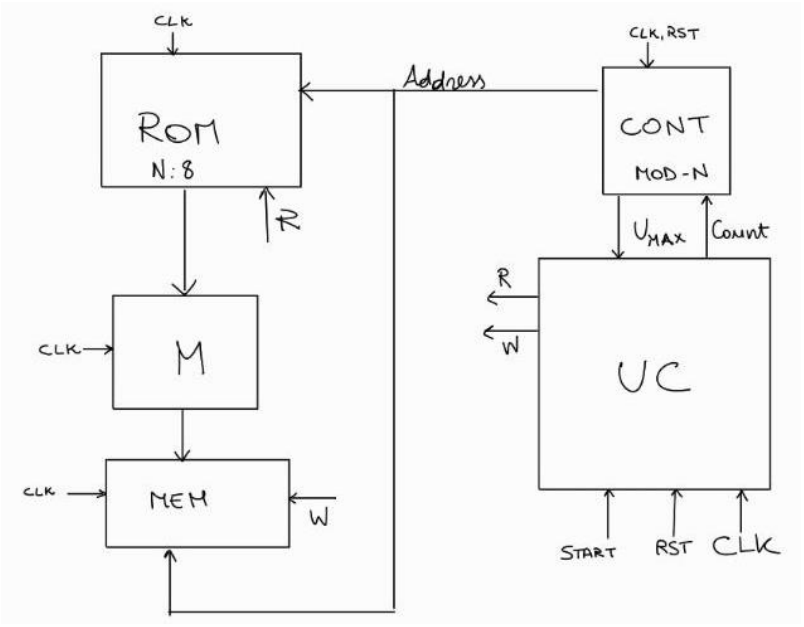
Progettare, implementare in VHDL e verificare mediante simulazione un sistema dotato di una memoria ROM di N locazioni da 8 bit ciascuna, una macchina combinatoria M in grado di trasformare (secondo una funzione a scelta dello studente) la stringa di 8 bit letta dalla ROM in una stringa di 4 bit, e una memoria MEM di N locazioni che memorizza la stringa in output da M.

Il sistema si avvia in corrispondenza di un segnale di START che viene fornito esternamente. Una volta avviato, tramite un'apposita unità di controllo che gestisce la tempificazione del sistema, viene scandita una locazione alla volta della ROM e viene scritta la corrispondente locazione di MEM. Gli indirizzi di memoria sono forniti da un contatore. Le memorie ROM e MEM hanno rispettivamente un read e un write sincrono.

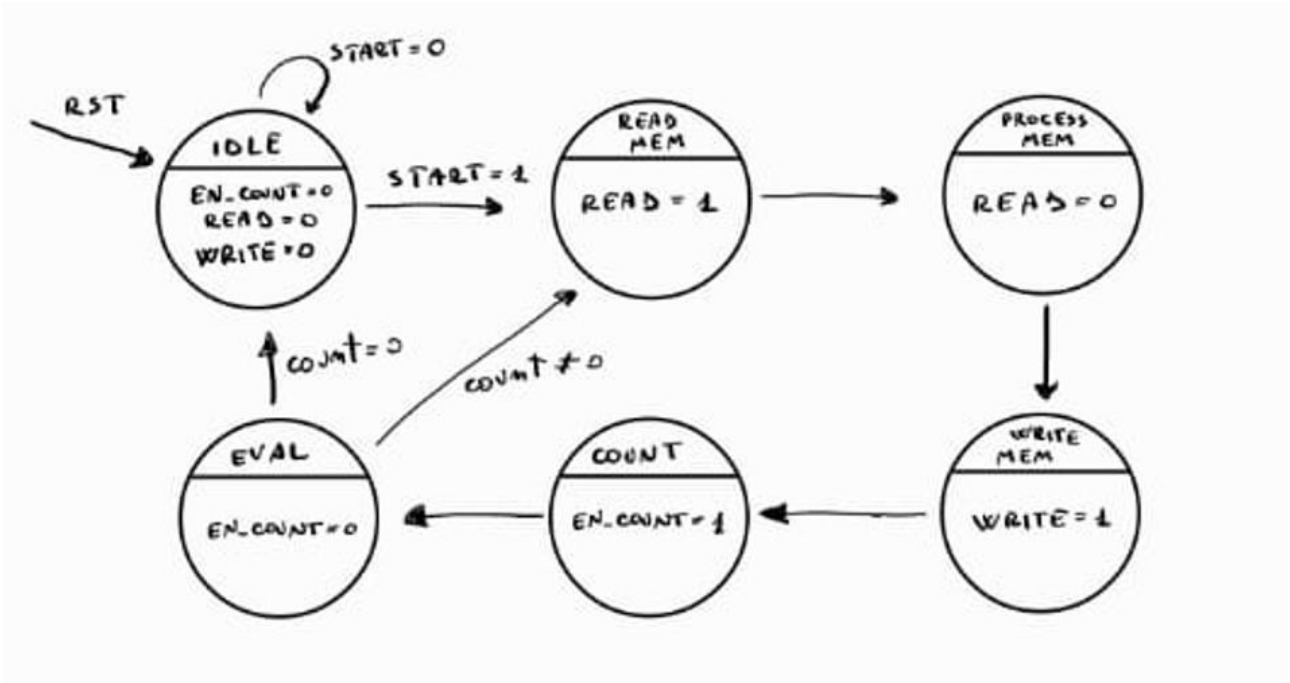
Progetto e architettura

Il sistema progettato prevede un'unità operativa composta da una ROM di N locazioni da 8 bit ciascuna, un contatore modulo N per il suo indirizzamento ed una memoria MEM di N locazioni. L'unità di controllo, invece, riceve in ingresso *start, clock, reset* e il segnale di conteggio massimo del contatore. Essa abilita il contatore e i segnali di *Read e Write* (sincroni) per le memorie.

Abbiamo progettato il sistema mediante diagramma a blocchi, diviso in UC e UO.



Riportiamo in figura l' ASF che descrive il comportamento della parte di controllo.



Implementazione

Il sistema è stato implementato tramite un'architettura strutturale costituita dalla parte di controllo PC e dalla parte operativa PO.

```
8  entity PO_PC_system is
9      Generic (
10         N : integer := 16 -- Numero di locazioni di memoria (modificabile)
11     );
12     Port (
13         CLK:            in std_logic;
14         RST:            in std_logic;
15         Start       :   in std_logic
16     );
17 end PO_PC_system;
18
19 architecture structural of PO_PC_system is
20
21     component PO is
22         Generic (
23             N : integer := 64 -- Numero di locazioni di memoria (modificabile)
24         );
25         Port (
26             CLK:        in std_logic;
27             RST:        in std_logic;
28             Read:       in std_logic;
29             Write:      in std_logic;
30             en_count:   in std_logic;
31             counter:    out std_logic_vector (integer(ceil(log2(real(N))))-1 downto 0)
32         );
33     end component;
34
35     component PC is
36         Generic (
37             N : integer := 64 -- Numero di locazioni di memoria (modificabile)
38         );
39         Port (
40             CLK:        in std_logic;
41             RST:        in std_logic;
42             Start:      in std_logic;
43             counter:    in std_logic_vector (integer(ceil(log2(real(N))))-1 downto 0);
44             Read:       out std_logic;
45             Write:      out std_logic;
46             en_count:   out std_logic
47         );
48     end component;
49
50     signal Read_temp : std_logic := '0';
51     signal Write_temp : std_logic := '0';
52     signal en_count_temp : std_logic := '0';
53     signal counter_temp : std_logic_vector (integer(ceil(log2(real(N))))-1 downto 0);
54
55 begin
56
57     Parte_operativa: PO
```

```
58          generic map (
59              N => N
60          )
61          port map(
62              CLK => CLK,
63              RST => RST,
64              Read => Read_temp,
65              Write => Write_temp,
66              en_count => en_count_temp,
67              counter => counter_temp
68          );
69
70      Parte_controllo: PC
71          generic map (
72              N => N
73          )
74          port map(
75              CLK => CLK,
76              RST => RST,
77              Start => Start,
78              Read => Read_temp,
79              Write => Write_temp,
80              en_count => en_count_temp,
81              counter => counter_temp
82          );
```

La Parte Operativa, implementata in modo strutturale, è costituita dal sistema ROM + macchina M, un contatore e una memoria MEM.

```
 9  entity PO is
10      Generic (
11          N : integer := 64 -- Numero di locazioni di memoria (modificabile)
12      );
13      Port (
14          CLK:        in std_logic;
15          RST:        in std_logic;
16          Read:       in std_logic;
17          Write:      in std_logic;
18          en_count:   in std_logic;
19          counter:    out std_logic_vector (integer(ceil(log2(real(N))))-1 downto 0)
20    );
21  end PO;
22
23  architecture structural of PO is
24
25      component ROM_M is
26          Generic (
27              N : integer := 64 -- Numero di locazioni di memoria (modificabile)
28          );
29          Port (
30              CLK     : in std_logic;
31              Address : in std_logic_vector (integer(ceil(log2(real(N))))-1 downto 0);
32              Read    : in std_logic;
33              dataOUT : out std_logic_vector (3 downto 0)
```

```
34          );
35      end component;
36
37      component ContMod_N is
38          Generic (
39              N : integer := 64 -- Modulo del contatore
40          );
41          Port (
42              CLK:         in std_logic;  -- clock
43              RST:         in std_logic;  -- reset asincrono
44              en_count:    in std_logic; --segnale di conteggio
45              dataOUT:     out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0) -- uscita di conteggio
46          );
47      end component;
48
49      component MEM is
50          Generic (
51              N : integer := 64 -- Numero di locazioni di memoria (modificabile)
52          );
53          Port (
54              CLK     : in std_logic;
55              Address : in std_logic_vector (integer(ceil(log2(real(N))))-1 downto 0);
56              Write   : in std_logic;
57              dataIN  : in std_logic_vector (3 downto 0)
58          );
59       end component;
60
61       signal Address_temp: std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0);
62       signal data_temp: std_logic_vector(3 downto 0);
63
64       begin
65
66       counter <= Address_temp;
67
68       Contatore: ContMod_N
69           generic map (
70               N => N)
71           port map(
72               CLK => CLK,
73               RST => RST,
74               en_count => en_count,
75               dataOUT => Address_temp);
76
77       ROM_sys: ROM_M
78           generic map (
79               N => N)
80           port map(
81               CLK => CLK,
82               Address => Address_temp,
```

```
83              Read    => Read,
84              dataOUT => data_temp);
85
86      Memory: MEM
87          generic map (
88              N => N)
89          port map(
90              CLK => CLK,
91              Address => Address_temp,
92              Write   => Write,
93              dataIN  => data_temp);
94
95  end ;
```

La Parte di Controllo implementa l'ASF descritto in fase di progettazione, secondo il modello di Moore, con due process.

```
 8  entity PC is
 9      Generic (
10          N : integer := 64 -- Numero di locazioni di memoria
11      );
12      Port (
13          CLK:        in std_logic;
14          RST:        in std_logic;
15          Start:      in std_logic;
16          counter:    in std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0);
17          Read:       out std_logic;
18          Write:      out std_logic;
19          en_count:   out std_logic
20      );
21  end PC;
22
23
24  -- modellato come automa di Moore
25  architecture Behavioral of PC is
26
27      type stato is (Idle, ReadMem, WaitMem, WriteMem, Count, Eval);
28      signal stato_corrente : stato := idle;
29      signal stato_prossimo : stato := stato_corrente;
30
31      begin
32          updateStatus: process (stato_corrente, start, counter)
```

```
33          begin
34              Case stato_corrente is
35                  when Idle =>
36                      en_count <= '0';
37                      Read <= '0';
38                      Write <= '0';
39
40                      if (start = '1') then
41                          stato_prossimo <= ReadMem;
42                      else
43                          stato_prossimo <= idle;
44                      end if;
45
46                  when ReadMem =>
47                      Read <= '1';
48                      stato_prossimo <= WaitMem;
49
50                  when WaitMem =>
51                      Read <= '0';
52                      stato_prossimo <= WriteMem ;
53
54                  when WriteMem =>
55                      Write <= '1';
56                      stato_prossimo <= Count;
57
58                  when Count =>
59                      Write <= '0';
60                      en_count <= '1';
61                      stato_prossimo <= Eval;
62
63                  when Eval =>
64                      en_count <= '0';
65                      if (counter = std_logic_vector(to_unsigned(0, counter'length)))then
66                          stato_prossimo <= idle;
67                      else
68                          stato_prossimo <= ReadMem;
69                  end if;
70
71                  when others =>
72                      stato_prossimo <= Idle;
73              end case;
74          end process;
75
76          mem : process (CLK, RST)
77          begin
78              if (RST = '1') then
79                 stato_corrente <= Idle;
80
81              elsif (rising_edge(CLK)) then
82                  stato_corrente <= stato_prossimo;
83              end if;
84          end process;
85
86  end;
```

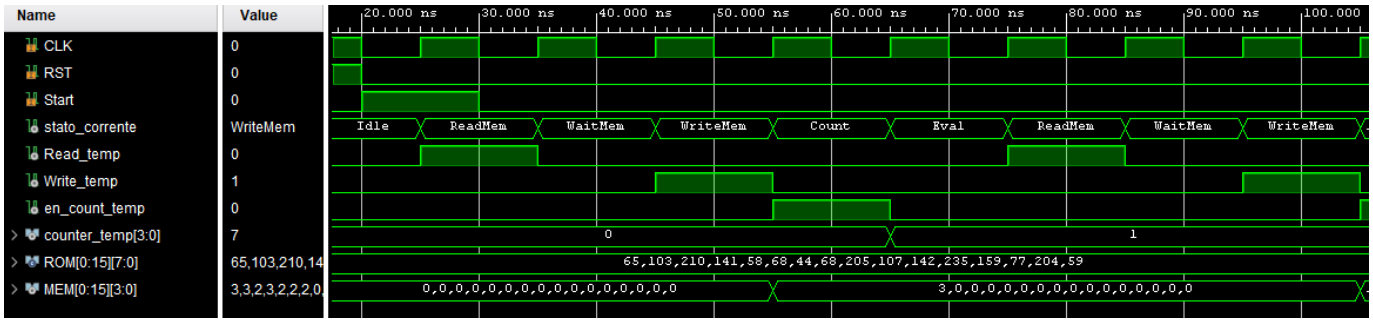
Simulazione

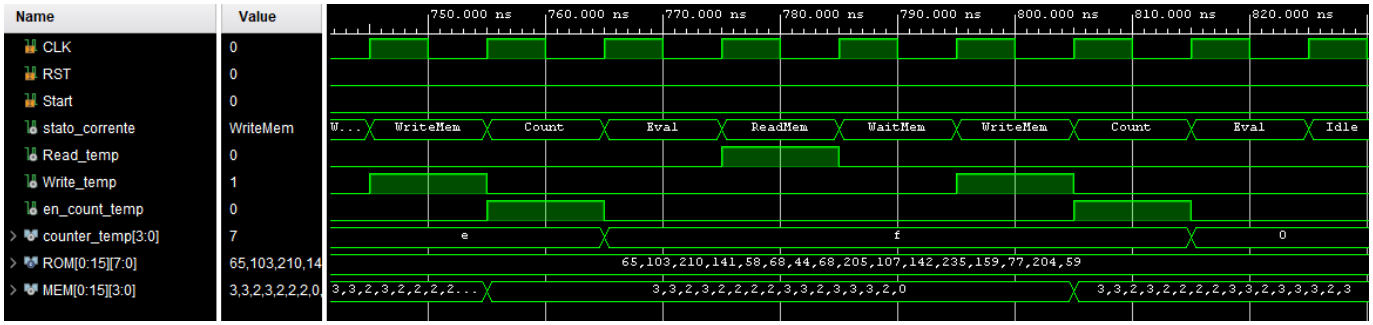
Segue il testbench implementato per la simulazione del sistema.

```
23      entity System_TB is
24      end System_TB;
25
26      architecture Behavioral of System_TB is
27          -- Segnali per il testbench
28          signal CLK : std_logic := '0';
29          signal RST : std_logic := '0';
30          signal Start : std_logic := '0';
31
32          -- Costante per il periodo del clock
33          constant CLK_PERIOD : time := 10 ns;
34
35          -- Istanza del sistema da testare
36          component System is
37              Generic (
38                  N : integer := 64 -- Numero di locazioni di memoria (modificabile)
39              );
40              Port (
41                  CLK : in std_logic;
42                  RST : in std_logic;
43                  Start : in std_logic
44              );
45          end component;
46
47      begin
48          -- Istanza del sistema
49          UUT: System
50              Generic Map (
51                  N => 64 -- Valore di default per il test
52              )
53              Port Map (
54                  CLK => CLK,
55                  RST => RST,
56                  Start => Start
57              );
58
59          -- Generazione del clock
60          CLK_process: process
61          begin
62              CLK <= '0';
63              wait for CLK_PERIOD / 2;
64              CLK <= '1';
65              wait for CLK_PERIOD / 2;
66          end process;
67
68          -- Processo di test
69          test_process: process
70          begin
71              -- Inizializzazione
72              RST <= '1'; -- Attiva il reset
73              Start <= '0';
```

```
74          wait for CLK_PERIOD * 2;
75
76          -- Disattiva il reset e avvia il sistema
77          RST <= '0';
78          Start <= '1';
79          wait for 1000 ns;
80
81          -- Simula un ciclo di lavoro
82          Start <= '0';
83          wait for CLK_PERIOD * 20;
84
85          -- Fine del test
86          wait;
87      end process;
88
89  end Behavioral;
```

Di seguito sono riportati i risultati ottenuti.



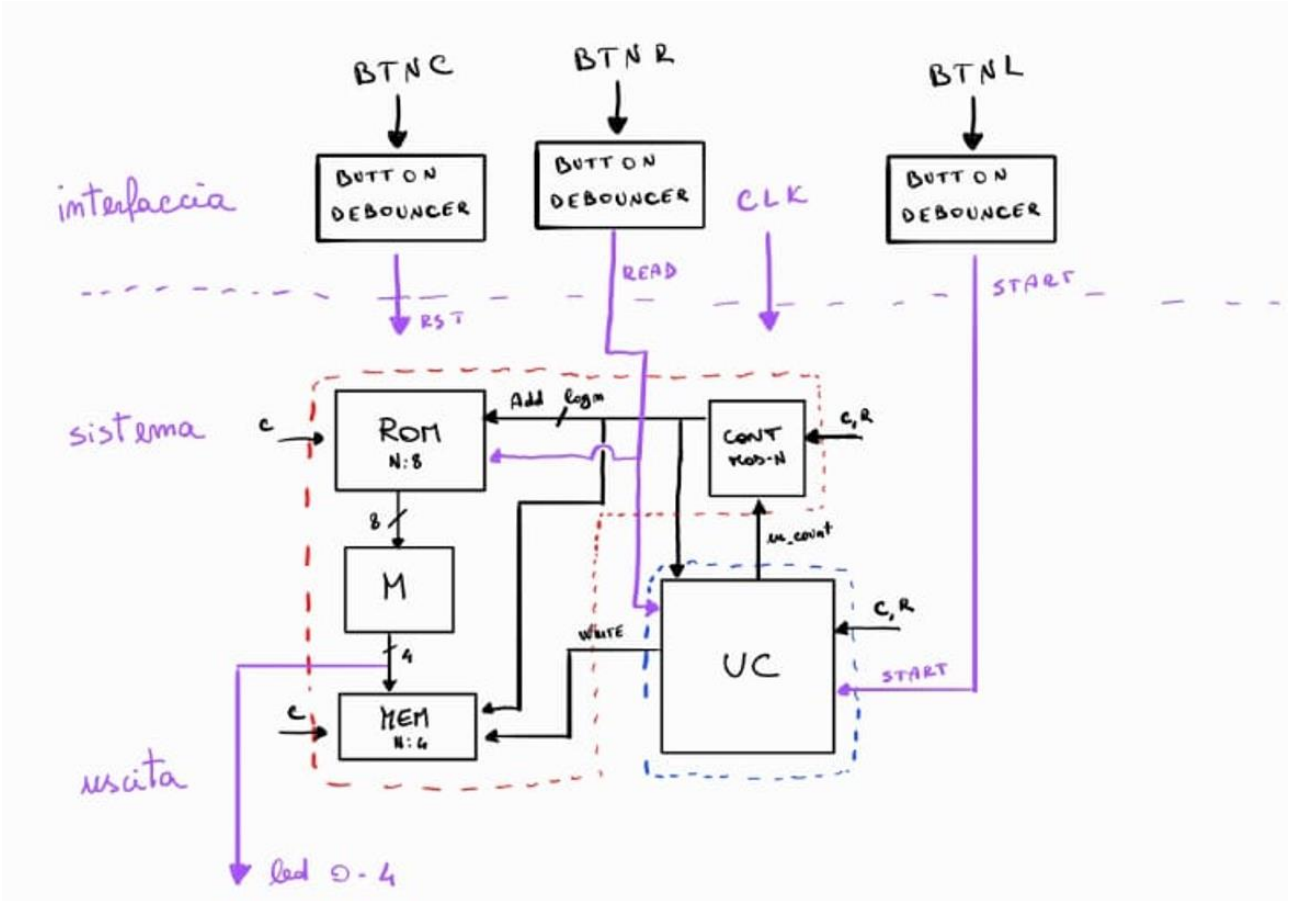


Esercizio 6.2

Sintetizzare ed implementare su board il componente sviluppato al punto precedente, utilizzando due bottoni per i segnali di read e reset rispettivamente e i led per la visualizzazione delle uscite della macchina istante per istante.

Sintesi su board di sviluppo

Per svolgere l'esercizio su board, abbiamo modificato l'architettura come segue



Istanziando quindi dei ButtonDebouncer per poter utlizzare i bottoni.

```
6  entity BTN_Interface is
7      Port(
8          clk : in std_logic;
9          Input_rst : in std_logic;
10         Input_start: in std_logic;
11         Input_Read : in std_logic;
12         cleared_rst : out std_logic;
13         cleared_start: out std_logic;
14         cleared_Read: out std_logic
15
16     );
17 end BTN_Interface;
18
19 architecture structural of BTN_Interface is
20
21     component ButtonDebouncer
22         generic (
23             CLK_period: integer := 10;
24             btn_noise_time: integer := 10000000
25         );
26         Port (
27             RST : in std_logic;
28             CLK : in std_logic;
29             BTN : in std_logic;
30             CLEARED_BTN : out std_logic
31         );
32     end component;
33
34 begin
35
36     BTN_reset : ButtonDebouncer
37         Generic map(
38             CLK_period => 10,
39             btn_noise_time => 10000000
40         )
41         Port map( RST => '0',
42             CLK => clk,
43             BTN => Input_rst,
44             CLEARED_BTN => cleared_rst
45         );
46
47     BTN_start : ButtonDebouncer
48         Generic map(
49             CLK_period => 10,
50             btn_noise_time => 10000000
51         )
52         Port map( RST => '0',
53             CLK => clk,
54             BTN => Input_start,
55             CLEARED_BTN => cleared_start
```

```
        );

    BTN_Read : ButtonDebouncer
        Generic map(
            CLK_period => 10,
            btn_noise_time => 10000000
        )
        Port map( RST =>'0',
            CLK => clk,
            BTN => Input_Read,
            CLEARED_BTN => cleared_Read
        );

end structural;
```

Mentre nel sistema PO_PC abbiamo associato l'uscita a 4 led. Di fatti, il file dei constraints della board NEXYS-A7-100T-Master è stato modificato come segue.

```
## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clk }]; #IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { clk }];
```

```
## LEDs
set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { led[0] }]; #IO_L18P_T2_A24_15 Sch=led[0]
set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { led[1] }]; #IO_L24P_T3_RS1_15 Sch=led[1]
set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { led[2] }]; #IO_L17N_T2_A25_15 Sch=led[2]
set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports { led[3] }]; #IO_L8P_T1_D11_14 Sch=led[3]
#set_property -dict { PACKAGE_PIN R18   IOSTANDARD LVCMOS33 } [get_ports { LED[4] }]; #IO_L7P_T1_D09_14 Sch=led[4]
#set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports { LED[5] }]; #IO_L18N_T2_A11_D27_14 Sch=led[5]
```

```
##Buttons
#set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { CPU_RESETN }]; #IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resetn
set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { Input_rst }]; #IO_L9P_T1_DQS_14 Sch=btnc
#set_property -dict { PACKAGE_PIN M18   IOSTANDARD LVCMOS33 } [get_ports { load_min }]; #IO_L4N_T0_D05_14 Sch=btnu
set_property -dict { PACKAGE_PIN P17   IOSTANDARD LVCMOS33 } [get_ports { Input_start }]; #IO_L12P_T1_MRCC_14 Sch=btnl
set_property -dict { PACKAGE_PIN M17   IOSTANDARD LVCMOS33 } [get_ports { Input_Read }]; #IO_L10N_T1_D15_14 Sch=btnr
#set_property -dict { PACKAGE_PIN P18   IOSTANDARD LVCMOS33 } [get_ports { BTND }]; #IO_L9N_T1_DQS_D13_14 Sch=btnd


##Pmod Headers
```

Il bottone centrale è utilizzato come *reset*, quello di sinistra come *start* e quello di destra come segnale di *read*.

Timing analysis

Per effettuare la timing analysis abbiamo utilizzato l'apposito tool di report fornito da Vivado; le analisi sono state condotte a valle di sintesi e implementazione per avere delle stime più precise, dato che dopo l'implementazione sono disponibili anche i tempi di routing.
Con il periodo di clock di 10ns (di default), otteniamo uno slack di 5,015 ns.

```
## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clk }]; #IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { clk }];
```

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	5,015 ns	Worst Hold Slack (WHS):	0,126 ns	Worst Pulse Width Slack (WPWS):	4,500 ns
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	0,000 ns	Total Pulse Width Negative Slack (TPWS):	0,000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	313	Total Number of Endpoints:	313	Total Number of Endpoints:	117

All user specified timing constraints are met.

Per capire quale è la frequenza massima di funzionamento possiamo ridurre il periodo del primary clock fino a ottenere un WNS negativo.
Possiamo ottenere la frequenza limite come FMAX =1/(T−WNS).
Nel nostro caso, ciò succede per periodi di clock minori o uguali a 3,73 nano-secondi:

```
## Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clk }]; #IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 3.73 -waveform {0 1.865} [get_ports { clk }];
```

Con questo periodo otteniamo uno slack molto vicino allo zero mentre con periodi inferiori lo slack sarebbe risultato negativo. Per questo assumiamo il limite minimo del periodo a 3,73ns.

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	0,026 ns	Worst Hold Slack (WHS):	0,152 ns	Worst Pulse Width Slack (WPWS):	1,365 ns
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	0,000 ns	Total Pulse Width Negative Slack (TPWS):	0,000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	313	Total Number of Endpoints:	313	Total Number of Endpoints:	117

All user specified timing constraints are met.

Capitolo 3: Macchine Aritmetiche

Esercizio 7 – Moltiplicatore di Booth

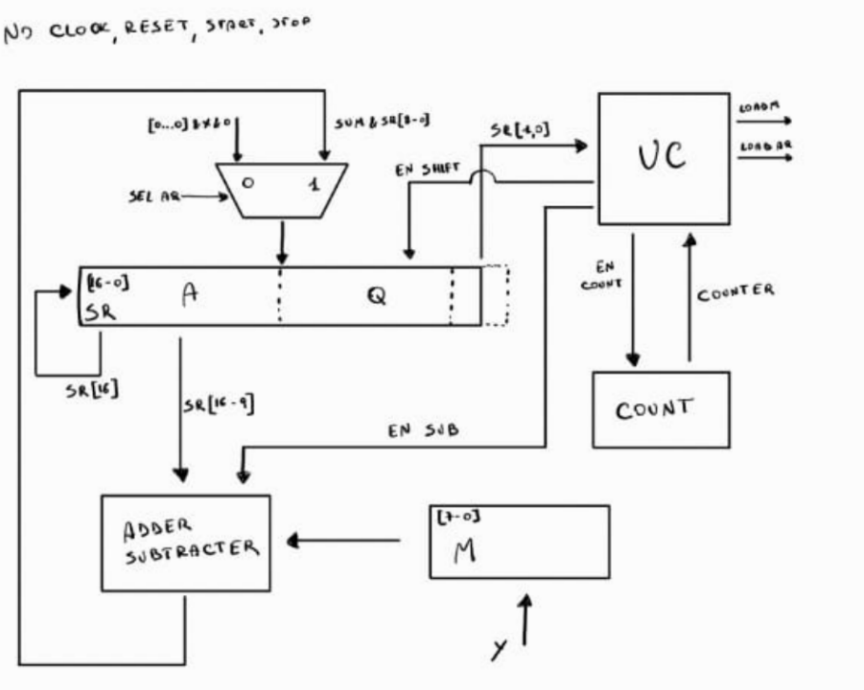
Il Moltiplicatore di Booth sfrutta la codifica di Booth per effettuare l'operazione tra un moltiplicando e un moltiplicatore appartenenti ai numeri relativi ed espressi binario in complemento a 2. Nell'algoritmo di Booth ad ogni passo si esaminano due bit adiacenti e, quando essi risultano avere lo stesso valore, si effettua solo uno shift, risparmiando una somma/sottrazione.

Esercizio 7.1

Progettare, implementare in VHDL e simulare una macchina moltiplicatore di Booth in grado di effettuare il prodotto di 2 stringhe A e B da 8 bit ciascuna.

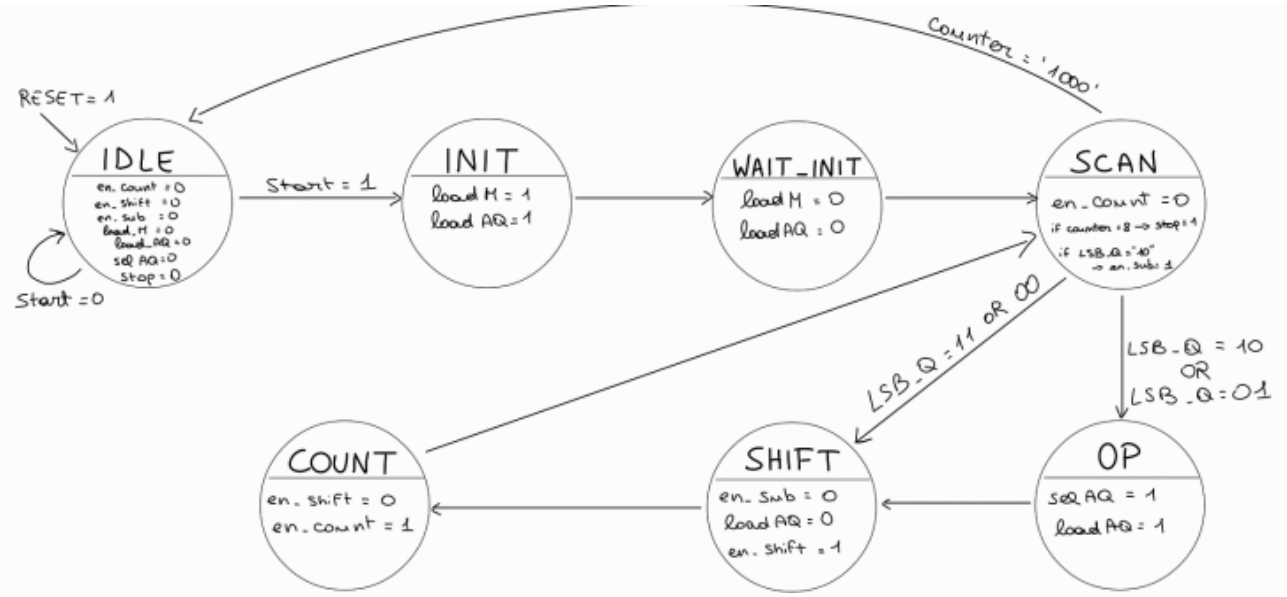
Progetto e architettura

L'architettura del moltiplicatore di Booth è schematizzata in figura.



Abbiamo distinto unità di controllo e unità operativa. In quest'ultima, abbiamo inserito un contatore modulo 8, un registro M, uno shift register per AQ da 17 bit, un multiplexer 2:1 e un addizionatore/sottrattore, composto dal parallel adder e da una porta xor.

Il comportamento della macchina può essere descritto mediante il seguente automa, composto da 7 stati: IDLE, INIT, WAIT_INIT_ SCAN, OP, SHIFT e COUNT.



Grazie al controllo che il moltiplicatore di Booth esegue sui due bit meno significativi del registro Q in SCAN, è possibile saltare l'addizionatore/sottrattore ed eseguire esclusivamente lo shift.

Implementazione

Segue l'implementazione del Moltiplicatore di Booth in VHDL.

```
26  entity booth is
27    Port (
28      clk, rst, start : in std_logic;
29      X,Y : in std_logic_vector(7 downto 0);
30      P : out std_logic_vector (15 downto 0);
31      stop: out std_logic
32    );
33  end booth;
34
35  architecture Structural of booth is
36      component unita_controllo is
37      Port(
38          clk, rst, start : in std_logic;
39          LSB_Q : in std_logic_vector (1 downto 0);
40          counter : in std_logic_vector (3 downto 0);
41          en_count, en_shift, en_sub, loadM, loadAQ, selAQ : out std_logic;
42          stop : out std_logic
43       );
44       end component;
45
46      component unita_operativa is
47      Port (
48          clk, rst : in std_logic;
49          X,Y : in std_logic_vector(7 downto 0);
50          en_count, en_shift, en_sub, loadM, loadAQ, selAQ: in std_logic;
51          counter : out std_logic_vector (3 downto 0);
52           P : out std_logic_vector (16 downto 0)
53      );
54      end component;
55
56      signal temp_LSBQ : std_logic_vector(1 downto 0) := (others => '0');
57      signal temp_counter : std_logic_vector(3 downto 0) := (others => '0');
58      signal temp_en_count : std_logic := '0';
59      signal temp_en_shift : std_logic := '0';
60      signal temp_en_sub : std_logic := '0';
61      signal temp_loadM : std_logic := '0';
62      signal temp_loadAQ : std_logic := '0';
63      signal temp_selAQ : std_logic := '0';
64      signal temp_P : std_logic_vector(16 downto 0) := (others => '0');
65
66      begin
67
68      temp_LSBQ <= temp_P(1 downto 0);
69      P <= temp_P (16 downto 1);
70
71      UC : unita_controllo
72          Port map (
73             clk => clk,
74             rst => rst,
75             start => start,
76             LSB_Q => temp_LSBQ,
77             counter => temp_counter,
```

```
            en_count => temp_en_count,
            en_shift => temp_en_shift,
            en_sub => temp_en_sub,
            loadM => temp_loadM,
            loadAQ => temp_loadAQ,
            selAQ => temp_selAQ,
            stop => stop
        );

     UO : unita_operativa
        Port map(
            clk => clk,
            rst => rst,
            X => X,
            Y => Y,
            en_count => temp_en_count,
            en_shift => temp_en_shift,
            en_sub => temp_en_sub,
            loadM => temp_loadM,
            loadAQ => temp_loadAQ,
            selAQ => temp_selAQ,
            counter => temp_counter,
            P => temp_P
    );
```

L'Unità Operativa è stata realizzata in maniera strutturale utilizzando uno shift register, un contatore, un addizionatore/sottrattore, un registro M e un mux 2:1.

```
entity unita_operativa is
    Port (
        clk, rst : in std_logic;
        X,Y : in std_logic_vector(7 downto 0);
        en_count, en_shift, en_sub, loadM, loadAQ, selAQ: in std_logic;
        counter : out std_logic_vector (3 downto 0);
        P : out std_logic_vector (16 downto 0)
    );
end unita_operativa;

architecture Structural of unita_operativa is
   --componenti
    component shift_register is
        Port (
            clk, rst, en_shift, load : in std_logic;
            parallel_in : in std_logic_vector (16 downto 0);
            serial_in : in std_logic :='0';
            parallel_out : out std_logic_vector (16 downto 0)
            );
    end component;

    component cont_8 is
        Port (
            clk, rst : in std_logic;
            en_cont : in std_logic;
            counter : out std_logic_vector (3 downto 0)
```

```
51          );
52      end component;
53
54      component M is
55          Port (
56              clk, rst, load : in std_logic;
57              data_in : in std_logic_vector(7 downto 0);
58              data_out : out std_logic_vector(7 downto 0)
59              );
60      end component;
61
62      component add_sub is
63          Port (
64              x , y : in std_logic_vector(7 downto 0);
65              en_sub : in std_logic;
66              cout : out std_logic;
67              z: out std_logic_vector (7 downto 0)
68          );
69      end component;
70
71      component mux_21 is
72      generic (width : integer range 0 to 32 := 8);
73         port(
74              x0, x1: in std_logic_vector(width-1 downto 0);
75              s: in std_logic;
76              y: out std_logic_vector(width-1 downto 0));
77      end component;
78
79    --signals
80    signal temp_M_out : std_logic_vector (7 downto 0) := (others => '0');
81    signal temp_add_sub_sum: std_logic_vector(7 downto 0);
82    signal temp_add_sub_riporto : std_logic := '0';
83    signal AQ_init : std_logic_vector (16 downto 0) := (others => '0');
84    signal AQ_sum_in: std_logic_vector (16 downto 0) := (others => '0');
85    signal temp_AQ_in : std_logic_vector (16 downto 0) := (others => '0');
86    signal temp_AQ_out: std_logic_vector (16 downto 0) := (others => '0');
87
88
89
90    signal op1 : std_logic_vector (8 downto 0) := (X & '0');
91
92  begin
93
94      AQ_init <= "00000000" & X & '0';
95      AQ_sum_in <= temp_add_sub_sum & temp_AQ_out(8 downto 0);
96      P <= temp_AQ_out;
97
98      --COUNTER
99      ContMod8 : cont_8
100         Port map(
101             clk => clk,
102             rst => rst,
```

```
103             en_cont => en_count,
104             counter => counter
105         );
106 
107     --REGISTRO M
108     RegM : M
109         Port map(
110             clk => clk,
111             rst => rst,
112             load => loadM,
113             data_in => Y,
114             data_out => temp_M_out
115         );
116 
117     --ADDER - SUBTRACTER
118     Adder : add_sub
119         Port map(
120             x => temp_AQ_out(16 downto 9),
121             y => temp_M_out,
122             en_sub => en_sub,
123             z => temp_add_sub_sum,
124             cout => temp_add_sub_riporto
125         );
126 
127     Mux : mux_21
128         generic map(
129             width => 17
130         )
131         port map(
132             x0 => AQ_init,
133             x1 => AQ_sum_in,
134             s => selAQ,
135             y => temp_AQ_in
136         );
137 
138     --REGISTRO AQ
139     AQ : shift_register
140         Port map (
141             clk => clk,
142             rst => rst,
143             en_shift => en_shift,
144             load => loadAQ,
145             parallel_in => temp_AQ_in,
146             serial_in => temp_AQ_out(16),
147             parallel_out => temp_AQ_out
148         );
149 
150 end Structural;
```

Il comportamento del Moltiplicatore di Booth, che abbiamo già modellato come ASF in precedenza, è stato definito invece nell'unità di controllo, che regola le transizioni tra i possibili stati della macchina a seconda degli ingressi, fornendo di volta in volta in uscita i segnali di controllo necessari.

```
13  entity unita_controllo is
14      Port(
15          clk, rst, start : in std_logic;
16          LSB_Q : in std_logic_vector (1 downto 0);
17          counter : in std_logic_vector (3 downto 0);
18          en_count, en_shift, en_sub, loadM, loadAQ, selAQ : out std_logic;
19          stop : out std_logic
20       );
21  end unita_controllo;
22
23  architecture Behavioral of unita_controllo is
24      type stato is (IDLE, INIT, WAITINIT, SCAN, OP, SHIFT, COUNT);
25      signal stato_corrente : stato := IDLE;
26      signal stato_prossimo : stato := stato_corrente;
27
28      begin
29
30      mem_stato: process (clk, rst)
31      begin
32          if (rising_edge(clk)) then
33              if (rst = '1') then
34                  stato_corrente <= idle;
35              else
36                  stato_corrente <= stato_prossimo;
37              end if;
38          end if;
39      end process;
40
41      comb: process (stato_corrente, start, counter)
42      begin
43
44          case stato_corrente is
45              when IDLE =>
46                  en_count <= '0';
47                  en_shift <= '0';
48                  en_sub <= '0';
49                  loadM <= '0';
50                  loadAQ <= '0';
51                  selAQ <= '0';
52                  stop <= '0';
53
54                  if (start='0') then
55                      stato_prossimo <= IDLE;
56
57                  elsif (start='1') then
58                      stato_prossimo <= INIT;
59                  end if;
60
61              when INIT  =>
62                  loadM <= '1';
63                  loadAQ <= '1';
64                  stato_prossimo <= scan;
```

```
65                  stato_prossimo <= waitINIT;
66
67              when WAITINIT =>
68                  loadM <= '0';
69                  loadAQ <= '0';
70                  stato_prossimo <= SCAN;
71
72              when SCAN =>
73
74                  en_count <= '0';
75
76                  if(counter = "1000")then
77                      stop <= '1';
78                      stato_prossimo <= IDLE;
79
80                  elsif (LSB_Q = "11" OR LSB_Q = "00") then
81                      stato_prossimo <= SHIFT;
82
83                  elsif (LSB_Q = "10") then
84                      en_sub <= '1';
85                      stato_prossimo <= OP;
86
87                  elsif (LSB_Q="01") then
88                      stato_prossimo <= OP;
89                  -- qui è appena stata eseguita la somma o la sottrazione
90                  end if;
91
92              when OP =>
93                  selAQ <= '1';
94                  loadAQ <= '1';
95                  stato_prossimo <= SHIFT;
96
97              when SHIFT =>
98                  en_sub <= '0';
99                  loadAQ <= '0';
100                 en_shift <= '1';
101                 stato_prossimo <= COUNT;
102
103             when COUNT =>
104                 en_shift <= '0';
105                 en_count <= '1';
106                 stato_prossimo <= SCAN;
107
108             when OTHERS =>
109                 stato_prossimo <= IDLE;
110
111         end case;
112     end process;
113
114 end Behavioral;
```

Simulazione

Segue il codice del testbench per il Moltiplicatore di Booth.

```
14  entity booth_tb is
15  end booth_tb;
16
17  architecture behavioural of booth_tb is
18
19      -- Dichiarazione del componente da testare
20      component booth is
21          Port (
22              clk, rst, start : in std_logic;
23              X, Y : in std_logic_vector(7 downto 0);
24              P : out std_logic_vector (15 downto 0);
25              stop: out std_logic
26          );
27      end component;
28
29      -- Segnali per i test
30      signal clk, rst, start : std_logic;
31      signal X, Y : std_logic_vector(7 downto 0);
32      signal P : std_logic_vector(15 downto 0);
33      signal stop : std_logic;
34
35      constant clk_period : time := 20 ns;
36      signal end_sim : std_logic := '0';
37
38  begin

41      uut: booth port map(
42          clk => clk,
43          rst => rst,
44          start => start,
45          X => X,
46          Y => Y,
47          P => P,
48          stop => stop
49      );
50
51      -- Generazione del clock
52      clk_process : process
53      begin
54          while (end_sim = '0') loop
55              clk <= '1';
56              wait for clk_period / 2;
57              clk <= '0';
58              wait for clk_period / 2;
59          end loop;
60          wait;
61      end process;
62
63      -- Stimoli per il testbench
64      sim: process
65      begin
66          wait for 100 ns;
```

```
        -- Reset
        rst <= '1';
        wait for 20 ns;
        rst <= '0';

        -- ------------------- Operazione 1:  5 * -3 = -15 (FFF1)
        X <= "00000101"; -- 5
        Y <= "11111101"; -- -3

        wait for 40 ns;
        start <= '1';
        wait for 20 ns;
        start <= '0';
        wait for 700 ns;

        -- Reset
        rst <= '1';
        wait for 20 ns;
        rst <= '0';
        wait for 20 ns;

        -- ------------------- Operazione 2:  -5 * 3 = -15 (FFF1)
        X <= "11111011"; -- -5
        Y <= "00000011"; -- 3

        wait for 40 ns;
```

```
         start <= '1';
         wait for 20 ns;
         start <= '0';
         wait for 600 ns;

         -- Reset
         rst <= '1';
         wait for 20 ns;
         rst <= '0';
         wait for 20 ns;

         -- ------------------- Operazione 3:  -3 * 3 = -9 (FFF7)
         X <= "11111101"; -- -3
         Y <= "00000011"; -- 3

         wait for 40 ns;
         start <= '1';
         wait for 20 ns;
         start <= '0';
         wait for 600 ns;

         -- Reset
         rst <= '1';
         wait for 20 ns;
         rst <= '0';
         wait for 20 ns;
```

```
        -- ------------------- Operazione 4:  -3 * -5 = 15 (000F)
        X <= "11111101"; -- -3
        Y <= "11111011"; -- -5

        wait for 40 ns;
        start <= '1';
        wait for 20 ns;
        start <= '0';
        wait for 600 ns;

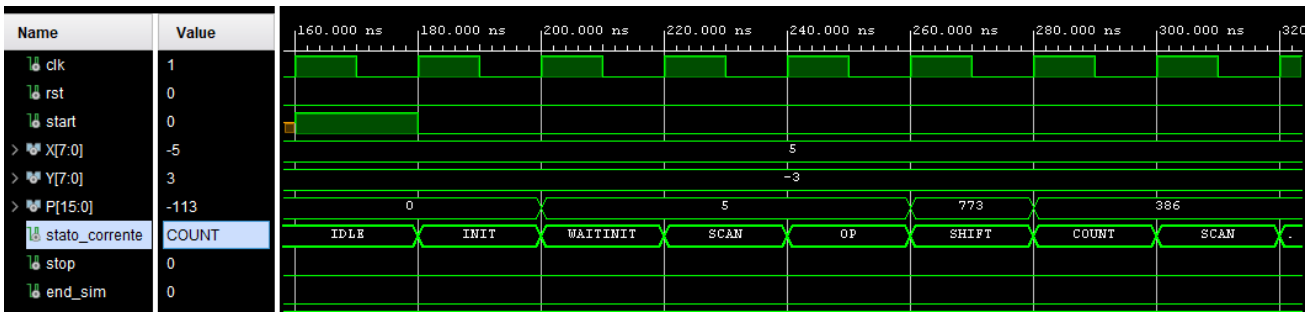
        -- Reset
        rst <= '1';
        wait for 20 ns;
        rst <= '0';
        wait for 20 ns;

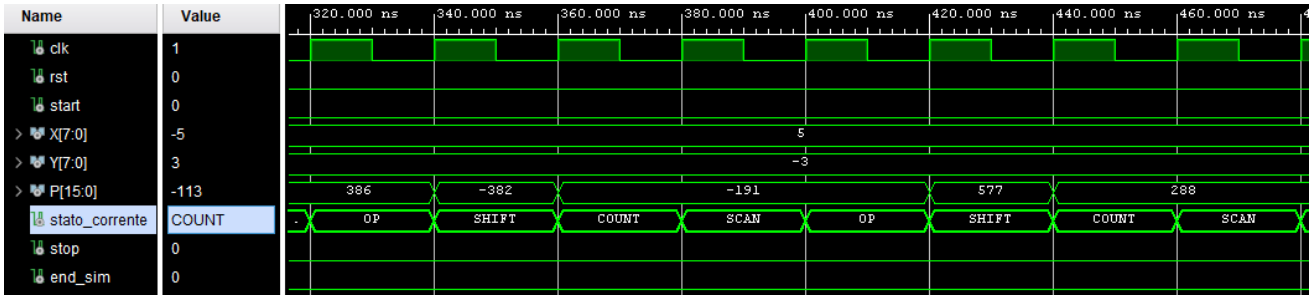
        -- ------------------- Operazione 5:  -128 * -128 = 16384 (4000)
        X <= "10000000"; -- -128
        Y <= "10000000"; -- -128

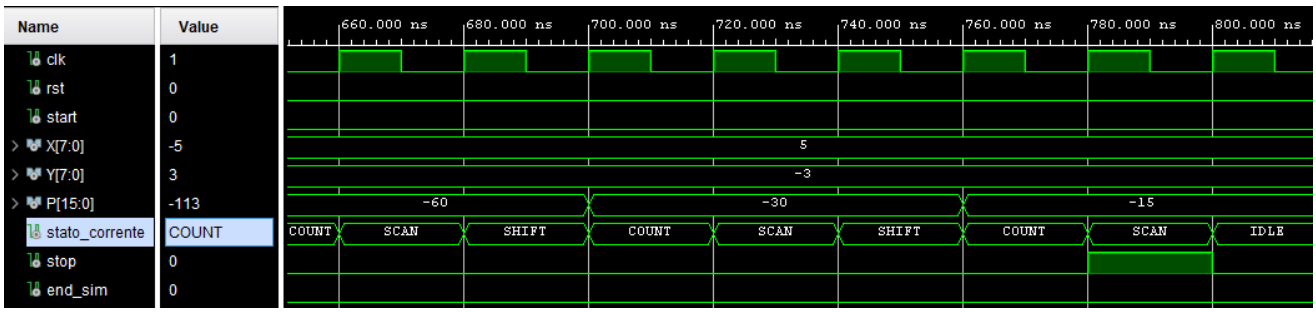
        wait for 40 ns;
        start <= '1';
        wait for 20 ns;
        start <= '0';
        wait for 600 ns;
```

```
147          -- Fine della simulazione
148          end_sim <= '1';
149          wait;
150      end process;
151
152  end behavioural;
153
```

Risultati della simulazione per un esempio di moltiplicazione:







Dopo 760 ns dal prelevamento degli operandi X e Y, P riporta il risultato -3x5=-15.

Esercizio 7.2

Sintetizzare il moltiplicatore implementato al punto 7.1 su FPGA e testarlo mediante l'utilizzo dei dispositivi di input/output (switch, bottoni, led, display) presenti sulla board di sviluppo in dotazione. La modalità di utilizzo degli stessi è a completa discrezione degli studenti.

Sintesi su board di sviluppo

Per implementare il moltiplicatore di Booth su board abbiamo utilizzato tre bottoni della board NEXYS-A7-100T-Master per il reset, start e segnale di conteggio.

Per comodità abbiamo istanziato i ButtonDebouncer necessari all'interno di una ButtonInterface strutturale.

```
6  entity BTN_Interface is
7      Port(
8          clk : in std_logic;
9          Input_rst : in std_logic;
10         Input_start: in std_logic;
11         Input_count: in std_logic;
12         cleared_rst : out std_logic;
13         cleared_start: out std_logic;
14         cleared_count: out std_logic
15     );
16 end BTN_Interface;
17
18 architecture structural of BTN_Interface is
19
20     component ButtonDebouncer
21         generic (
22             CLK_period: integer := 10;
23             btn_noise_time: integer := 10000000
24         );
25         Port (
26             RST : in std_logic;
27             CLK : in std_logic;
28             BTN : in std_logic;
29             CLEARED_BTN : out std_logic
30         );
31     end component;
32
33 begin
34
35     BTN_reset : ButtonDebouncer
36         Generic map(
37             CLK_period => 10,
38             btn_noise_time => 10000000
39         )
40         Port map( RST => '0',
41             CLK => clk,
42             BTN => Input_rst,
43             CLEARED_BTN => cleared_rst
44         );
45
46     BTN_start : ButtonDebouncer
47         Generic map(
48             CLK_period => 10,
49             btn_noise_time => 10000000
50         )
51         Port map( RST => '0',
52             CLK => clk,
53             BTN => Input_start,
54             CLEARED_BTN => cleared_start
55         );
```

```
56 
57      BTN_visual : ButtonDebouncer
58          Generic map(
59              CLK_period => 10,
60              btn_noise_time => 10000000
61          )
62          Port map( RST => '0',
63              CLK => clk,
64              BTN => Input_count,
65              CLEARED_BTN => cleared_count
66          );
67 
68  end structural;
```

Utilizziamo questa interfaccia nel top-module per collegare i bottoni ai segnali di start, reset e count al moltiplicatore di Booth progettato in precedenza.

```
 7  entity Booth_on_board is
 8    Port (
 9      clk, rst, start, count : in std_logic;
10      X,Y : in std_logic_vector(7 downto 0);
11      LED : out  std_logic_vector (15 downto 0)
12    );
13  end Booth_on_board;
14 
15  architecture board of Booth_on_board is
16 
17      component BTN_Interface is
18          Port(
19              clk : in std_logic;
20              Input_rst : in std_logic;
21              Input_start: in std_logic;
22              Input_count: in std_logic;
23              cleared_rst : out std_logic;
24              cleared_start: out std_logic;
25              cleared_count: out std_logic
26          );
27      end component;
28 
29      component booth is
30          Port (
31              clk, rst, start, count : in std_logic;
32               X,Y : in std_logic_vector(7 downto 0);
33               P : out std_logic_vector (15 downto 0);
34               stop: out std_logic
35       );
36       end component;
37 
38 
39       signal temp_cleared_rst: std_logic;
40       signal temp_cleared_start: std_logic;
41       signal temp_cleared_count: std_logic;
42       signal temp_stop: std_logic;
43       signal temp_P: std_logic_vector(15 downto 0) := (others => '0');
44       signal temp_X: std_logic_vector(7 downto 0);
45       signal temp_Y: std_logic_vector(7 downto 0);
46 
47 
48       begin
```

```
54      interfaccia: BTN_Interface
55          Port map(
56              clk => clk,
57              Input_rst => rst,
58              Input_start => start,
59              Input_count => count,
60              cleared_rst => temp_cleared_rst,
61              cleared_start => temp_cleared_start,
62              cleared_count => temp_cleared_count
63          );
77          sistema: booth
78          Port map(
79              clk => clk,
80              rst => rst,
81              start => start,
82              count => count,
83              X => X,
84              Y => Y,
85              P => temp_P,
86              stop => temp_stop
87          );

89  --      Assegna il valore a LED solo quando stop è alto
90      process(clk)
91      begin
92          if rising_edge(clk) then
93              if(temp_cleared_rst = '1') then
94                  LED <= "0000000010101010";
95              elsif (temp_stop = '1') then
96                      LED <= temp_P;
97                  elsif (start = '1') then
98                      LED <= "0000000000000000";
99
100             end if;
101         end if;
102     end process;
103
104 end board;
```

Ad ogni pressione del pulsante connesso a count, si esegue uno shift del registro Q; quando si eseguono 8 shift, si ottiene il risultato del prodotto.
Il prodotto finale sarà quindi visualizzato sui 16 led della board.

Capitolo 4: Handshaking

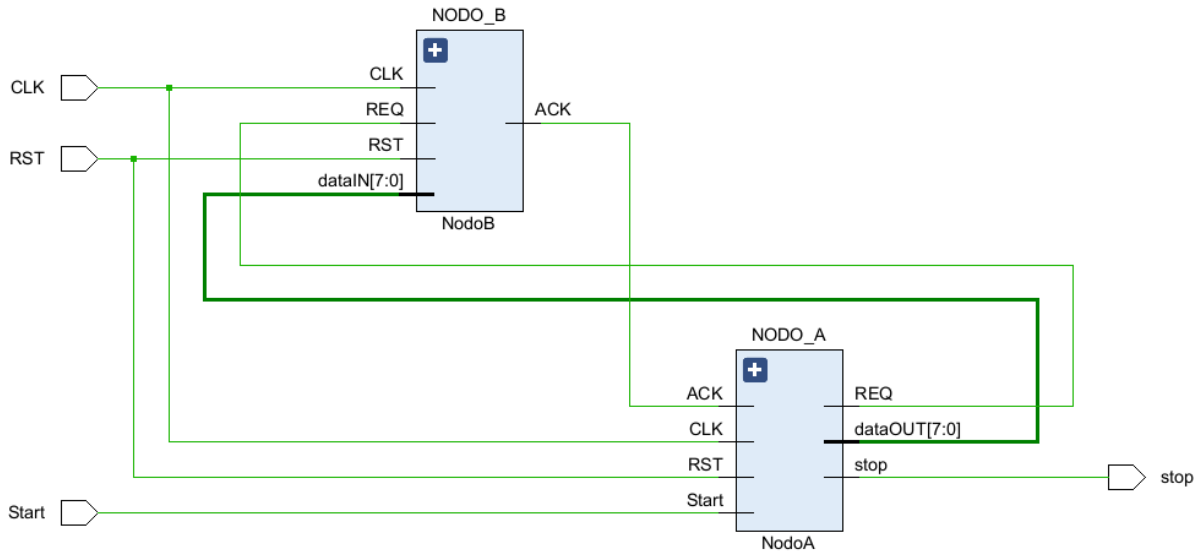
Esercizio 8 – comunicazione con handshaking

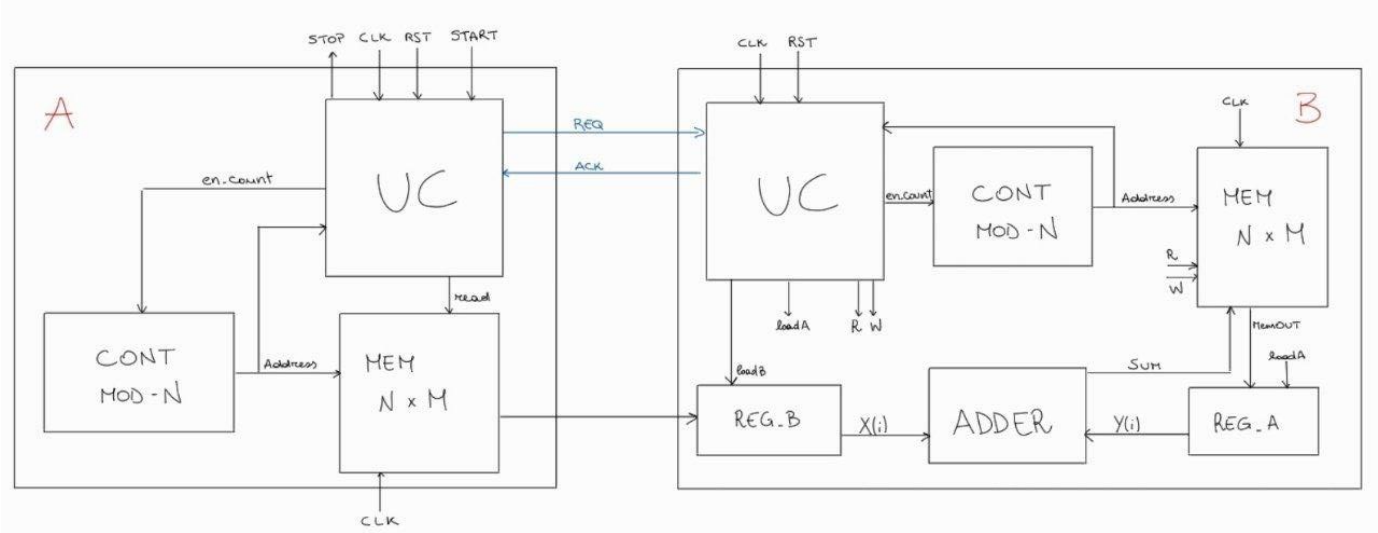
Esercizio 8.1

Progettare, implementare in VHDL e testare mediante simulazione un sistema composto da 2 nodi, A e B, che comunicano mediante un protocollo di handshaking. Il nodo A e il nodo B possiedono entrambi una memoria interna in cui sono memorizzate N stringhe di M bit, denominate X(i) e Y(i) rispettivamente (i=0,..,N-1). Il nodo A trasmette a B ciascuna stringa X(i) utilizzando un protocollo di handshaking; B, ricevuta la stringa X(i), calcola S(i)=X(i)+Y(i) e immagazzina la somma in opportune locazioni della propria memoria interna. Per il progetto è possibile considerare una implementazione di tipo comportamentale per effettuare la somma, mentre è necessario prevedere esplicitamente un componente contatore sia nel sistema A sia nel sistema B per scandire la trasmissione/ricezione delle stringhe e per terminare la comunicazione.

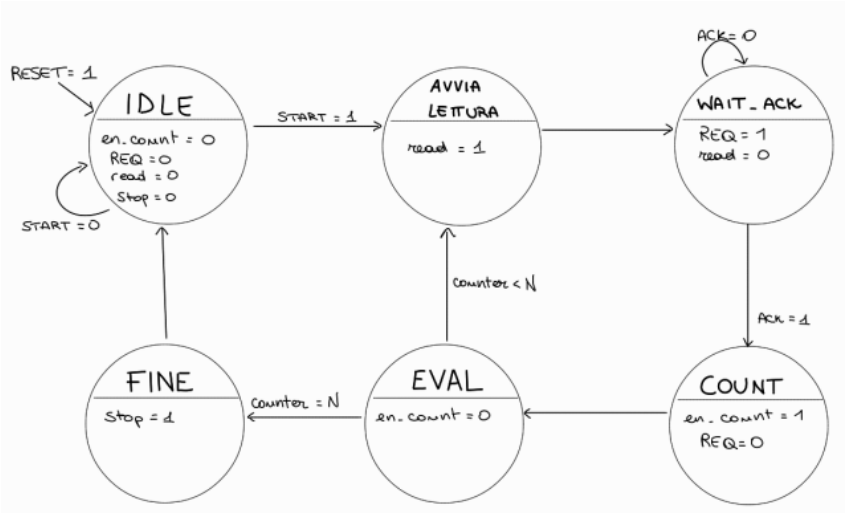
Progetto e architettura

Il nodo A effettua la trasmissione dei dati una volta ricevuto il segnale di *start* mentre B è il ricevitore. La gestione dei singoli componenti è affidata a due unità di controllo differenti, realizzate come due macchine sequenziali.

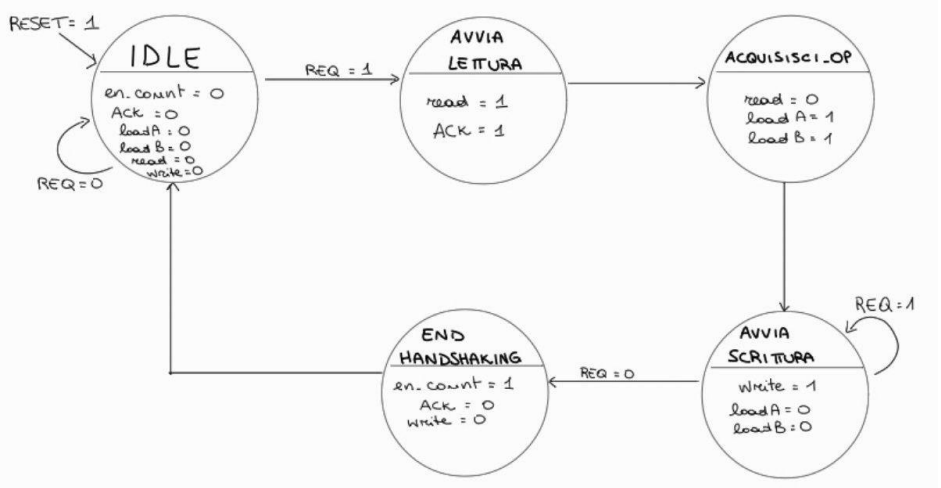




Il comportamento del nodo A è descritto mediante un automa costituito da 6 stati.



Mentre il nodo B evolve come mostrato nella seguente figura.



Il protocollo di comunicazione prescelto prevede che A alzi REQ dopo aver prelevato il dato dalla ROM ed attende che B alzi ACK per abbassare la sua REQ, mentre B alza l'ACK non appena vede REQ alto, memorizza il dato nel registro e abbassa ACK solo dopo aver visto REQ basso. Si tratta, dunque, di un protocollo di handshaking semplice.

Implementazione

Il sistema è implementato mediante architettura strutturale composta da nodo A e nodo B.

```
9  entity sistema is
10     Generic (
11         N : integer := 16; -- Numero di locazioni di memoria (modificabile)
12         M : integer := 8   -- Larghezza in bit di ogni locazione
13     );
14     Port (
15         CLK:        in std_logic;
16         RST:        in std_logic;
17         Start:      in std_logic;
18         stop:       out std_logic
19      );
20 end sistema;
21
22 architecture structural_sistema of sistema is
23
24     component NodoA is
25         Generic (
26             N : integer := 16; -- Numero di locazioni di memoria (modificabile)
27             M : integer := 8   -- Larghezza in bit di ogni locazione
28         );
29         Port (
30             CLK:        in std_logic;
31             RST:        in std_logic;
32             Start:      in std_logic;
33             ACK:        in std_logic;
34             stop:       out std_logic;

35            REQ:        out std_logic;
36            dataOUT :   out std_logic_vector(M-1 downto 0)
37        );
38     end component;
39
40     component NodoB is
41         Generic (
42             N : integer := 16; -- Numero di locazioni di memoria (modificabile)
43             M : integer := 8   -- Larghezza in bit di ogni locazione
44         );
45         Port (
46             CLK:        in std_logic;
47             RST:        in std_logic;
48             REQ:        in std_logic;
49             dataIN :    in std_logic_vector(M-1 downto 0);
50             ACK:        out std_logic
51         );
52     end component;
53
54     signal REQ_temp : std_logic := '0';
55     signal ACK_temp : std_logic := '0';
56     signal data_temp : std_logic_vector(M-1 downto 0) := (others => '0');
57
58
59
60 begin
```

```
62      NODO_A: NodoA
63          generic map (
64              N => N,
65              M => M
66          )
67          port map(
68              CLK => CLK,
69              RST => RST,
70              start => start,
71              ACK => ACK_temp,
72              stop => stop,
73              REQ => REQ_temp,
74              dataOUT => data_temp
75          );
76
77       NODO_B: NodoB
78          generic map (
79              N => N,
80              M => M
81          )
82          port map(
83              CLK => CLK,
84              RST => RST,
85              REQ => REQ_temp,
86              dataIN => data_temp,
87              ACK => ACK temp
```

Implementazione strutturale del Nodo A, suddiviso in unità operativa e di controllo:

```
9   entity NodoA is
10      Generic (
11          N : integer := 16; -- Numero di locazioni di memoria (modificabile)
12          M : integer := 8   -- Larghezza in bit di ogni locazione
13      );
14      Port (
15          CLK:        in std_logic;
16          RST:        in std_logic;
17          Start:      in std_logic;
18          ACK:        in std_logic;
19          stop:       out std_logic;
20          REQ:        out std_logic;
21          dataOUT :   out std_logic_vector(M-1 downto 0)
22       );
23  end NodoA;
24
25  architecture structuralNodoA of NodoA is
26
27      component UO_A is
28          Generic (
29              N : integer := 64; -- Numero di locazioni di memoria
30              M : integer := 8   -- Larghezza in bit di ogni locazione
31          );
32          Port (
33              clk, rst : in std_logic;
34              en_count, read : in std_logic;
```

```
            dataOUT : out std_logic_vector(M-1 downto 0);
            counter: out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0)
        );
    end component;

    component UC_A is
        Generic (
            N : integer := 64 -- Numero di locazioni di memoria
        );
        Port(
            clk, rst, start, ACK : in std_logic;
            counter : in std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0);
            en_count, read, REQ, stop: out std_logic
        );
    end component;

    signal Read_temp : std_logic := '0';
    signal counter_temp : std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0):= (others => '0');
    signal en_count_temp : std_logic := '0';


begin

    Parte_operativa: UO_A
        generic map (
            N => N,
            M => M
        )
        port map(
            CLK => CLK,
            RST => RST,
            en_count => en_count_temp,
            Read => Read_temp,
            dataOUT => dataOUT,
            counter => counter_temp
        );

    Parte_controllo: UC_A
        generic map (
            N => N
        )
        port map(
            CLK => CLK,
            RST => RST,
            Start => Start,
            ACK => ACK,
            counter => counter_temp,
            en_count => en_count_temp,
            Read => Read_temp,
            REQ => REQ,
            stop => stop
        );
```

Di seguito, la parte operativa di A che include una memoria e un contatore modulo N.

```vhdl
entity UO_A is
    Generic (
        N : integer := 64; -- Numero di locazioni di memoria
        M : integer := 8   -- Larghezza in bit di ogni locazione
    );
    Port (
        clk, rst : in std_logic;
        en_count, read : in std_logic;
        dataOUT : out std_logic_vector(M-1 downto 0);
        counter: out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0)
    );
end UO_A;

architecture structural_A of UO_A is

    component ContMod_N is
        Generic (
            N : integer := 64 -- Modulo del contatore
        );
        Port (
            CLK:         in std_logic;  -- clock
            RST:         in std_logic;  -- reset asincrono
            en_count:    in std_logic; --segnale di conteggio
            dataOUT:     out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0) -- uscita di conteggio
        );
    end component;

    component ROM is
    Generic (
        N : integer := 64; -- Numero di locazioni di memoria
        M : integer := 8   -- Larghezza in bit di ogni locazione
    );
    Port (
        CLK     : in std_logic;
        Address : in std_logic_vector (integer(ceil(log2(real(N))))-1 downto 0);
        Read    : in std_logic;
        dataOUT : out std_logic_vector (M-1 downto 0)
    );
    end component;

    signal Address_temp: std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0):= (others => '0');

    begin

    counter <= Address_temp;

    --COUNTER
    ContModN_A : ContMod_N
         Generic map(
            N => N
         )
        Port map(
            clk => clk,
            rst => rst,
            en_count => en_count,
            dataOUT => Address_temp
        );

    ROM_A : ROM
        Generic map(
            N => N,
            M => M
        )
        Port map(
            CLK => clk,
            Address => Address_temp,
            Read => read,
            dataOUT => dataOUT
        );

end Structural_A;
```

Mentre l'unità di controllo implementa l'automa a stati finiti di A.

```
15  entity UC_A is
16      Generic (
17          N : integer := 64 -- Numero di locazioni di memoria
18      );
19      Port(
20          clk, rst, start, ACK : in std_logic;
21          counter : in std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0);
22          en_count, read, REQ, stop: out std_logic
23       );
24  end UC_A;
25
26  architecture Behavioral of UC_A is
27      type stato is (IDLE, AVVIA_LETTURA, WAIT_ACK, COUNT, EVAL, FINE);
28      signal stato_corrente : stato := IDLE;
29      signal stato_prossimo : stato := stato_corrente;
30
31      begin
32
33      mem_stato: process (clk, rst)
34      begin
35          if (rising_edge(clk)) then
36              if (rst = '1') then
37                  stato_corrente <= idle;
38              else
39                  stato_corrente <= stato_prossimo;
40              end if;
41          end if;
42      end process;
43
44      comb: process (stato_corrente, start, counter, ACK)
45      begin
46
47          case stato_corrente is
48              when IDLE =>
49                  en_count <= '0';
50                  REQ <= '0';
51                  read <= '0';
52                  stop <= '0';
53
54                  if (start='0') then
55                      stato_prossimo <= IDLE;
56
57                  elsif (start='1') then
58                      stato_prossimo <= AVVIA_LETTURA;
59                  end if;
60
61              when AVVIA_LETTURA  =>
62                  read <= '1';
63                  stato_prossimo <= WAIT_ACK;
64
65              when WAIT_ACK =>
66                  REQ <= '1';
67                  read <= '0';
68
69                  if (ACK = '0') then
70                      stato_prossimo <= WAIT_ACK;
71
72                  elsif (ACK = '1') then
73                      stato_prossimo <= COUNT;
74                  end if;
75
76              when COUNT =>
77                  en_count <= '1';
78                  REQ <= '0';
79                  stato_prossimo <= EVAL;
80
81              when EVAL =>
82                  en_count <= '0';
83                  if (counter = std_logic_vector(to_unsigned(0, counter'length)))then
84                      stato_prossimo <= FINE;
85                  else
86                      stato_prossimo <= AVVIA_LETTURA;
87                  end if;
88
89              when FINE =>
90                  stop <= '1';
91                  stato_prossimo <= IDLE;
```

```
92
93              when OTHERS =>
94                  stato_prossimo <= IDLE;
95
96          end case;
97      end process;
98
99  end Behavioral;
```

Anche l'implementazione del Nodo B prevede unità di controllo e unità operativa:

```
9   entity NodoB is
10      Generic (
11          N : integer := 16; -- Numero di locazioni di memoria (modificabile)
12          M : integer := 8   -- Larghezza in bit di ogni locazione
13      );
14      Port (
15          CLK:        in std_logic;
16          RST:        in std_logic;
17          REQ:        in std_logic;
18          dataIN :    in std_logic_vector(M-1 downto 0);
19          ACK:        out std_logic
20
21       );
22  end NodoB;
23
24  architecture structuralNodoB of NodoB is
25
26      component UO_B is
27          Generic (
28              N : integer := 64; -- Numero di locazioni di memoria
29              M : integer := 8   -- Larghezza in bit di ogni locazione
30          );
31          Port (
32              clk, rst : in std_logic;
33              en_count, read, write, loadA, loadB : in std_logic;
34              dataIN : in std_logic_vector(M-1 downto 0);

35             counter: out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0)
36          );
37      end component;
38
39      component UC_B is
40          Generic (
41              N : integer := 64 -- Numero di locazioni di memoria
42          );
43          Port(
44              clk, rst, REQ : in std_logic;
45              counter : in std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0);
46              en_count, read, write, loadA, loadB, ACK: out std_logic
47          );
48      end component;
49
50      signal Read_temp : std_logic := '0';
51      signal Write_temp : std_logic := '0';
52      signal counter_temp : std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0):= (others => '0');
53      signal en_count_temp : std_logic := '0';
54      signal loadA_temp : std_logic := '0';
55      signal loadB_temp : std_logic := '0';
56
57
58  begin
59
60      Parte operativa: UO B
```

```
61          generic map (
62              N => N,
63              M => M
64          )
65          port map(
66              CLK => CLK,
67              RST => RST,
68              en_count => en_count_temp,
69              Read => Read_temp,
70              Write => Write_temp,
71              loadA => loadA_temp,
72              loadB => loadB_temp,
73              dataIN => dataIN,
74              counter => counter_temp
75          );
76
77      Parte_controllo: UC_B
78          generic map (
79              N => N
80          )
81          port map(
82              CLK => CLK,
83              RST => RST,
84              REQ => REQ,
85              counter => counter_temp,
86              en_count => en_count_temp,
87              Read => Read_temp,
88              Write => Write_temp,
89              loadA => loadA_temp,
90              loadB => loadB_temp,
91              ACK => ACK
92          );
93
94  end ;
```

L'unità operativa comprende 2 registri, una memoria MEM, un contatore modulo N e un adder.

```
7   entity UO_B is
8       Generic (
9           N : integer := 64; -- Numero di locazioni di memoria
10          M : integer := 8   -- Larghezza in bit di ogni locazione
11      );
12      Port (
13          clk, rst : in std_logic;
14          en_count, read, write, loadA, loadB : in std_logic;
15          dataIN : in std_logic_vector(M-1 downto 0);
16          counter: out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0)
17      );
18  end UO_B;
19
20  architecture structural_B of UO_B is
21
22      component ContMod_N is
23          Generic (
24              N : integer := 64 -- Modulo del contatore
25          );
26          Port (
27              CLK:         in std_logic;  -- clock
28              RST:         in std_logic;  -- reset asincrono
29              en_count:    in std_logic; --segnale di conteggio
30              dataOUT:     out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0) -- uscita di conteggio
31          );
32      end component;
```

```
    component MEM is
        Generic (
            N : integer := 64;  -- Numero di locazioni di memoria (modificabile)
            M : integer := 8    -- Numero di bit per locazione di memoria (modificabile)
        );
        Port (
            CLK     : in std_logic;
            Address : in std_logic_vector (integer(ceil(log2(real(N))))-1 downto 0);
            Write   : in std_logic;
            Read    : in std_logic;
            dataIN  : in std_logic_vector (M-1 downto 0);
            dataOUT : out std_logic_vector (M-1 downto 0)
        );
    end component;

    component Adder is
        Generic (
            M : integer := 8  -- Numero di bit (default: 8)
        );
        Port (
            A, B   : in  std_logic_vector(M-1 downto 0); -- Operandi M-bit
            SUM    : out std_logic_vector(M-1 downto 0)  -- Somma M-bit
        );
    end component;

    component REG_M is
```

```
        Generic (
            M : integer := 8
        );  -- Parametro per la larghezza del registro
        Port (
            dataIN  : in std_logic_vector(M-1 downto 0);
            clk, rst, load : in std_logic;
            dataOUT : out std_logic_vector(M-1 downto 0)
        );
    end component;

    signal temp_Address: std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0):= (others => '0');
    signal temp_MEM_out: std_logic_vector(M-1 downto 0):= (others => '0');
    signal temp_REG_A_out: std_logic_vector(M-1 downto 0):= (others => '0');
    signal temp_REG_B_out: std_logic_vector(M-1 downto 0):= (others => '0');
    signal temp_adder_out: std_logic_vector(M-1 downto 0):= (others => '0');

    begin

    counter <= temp_Address;

    --COUNTER
    ContModN_B : ContMod_N
         Generic map(
            N => N
         )
        Port map(
```

```
            Clk => clk,
            rst => rst,
            en_count => en_count,
            dataOUT => temp_Address
        );

    MEM_B : MEM
        Generic map(
            N => N,
            M => M
        )
        Port map(
            CLK => clk,
            Address => temp_Address,
            Write => write,
            Read => read,
            dataIN => temp_adder_out,
            dataOUT => temp_MEM_out
        );

    M_adder : Adder
        Generic map(
            M => M
        )
        Port map(
            A => temp_REG_A_out,
            B => temp_REG_B_out,
            SUM => temp_adder_out
        );

    REG_A : REG_M
        Generic map (
            M => M
        )
        Port map(
            dataIN => temp_MEM_out,
            clk => clk,
            rst => rst,
            load => loadA,
            dataOUT => temp_REG_A_out
        );

    REG_B : REG_M
        Generic map (
            M => M
        )
        Port map(
            dataIN => dataIN,
            clk => clk,
            rst => rst,
            load => loadB,
            dataOUT => temp_REG_B_out
```

Mentre l'unità di controllo implementa l'automa a stati finiti di B.

```
15  entity UC_B is
16      Generic (
17          N : integer := 64 -- Numero di locazioni di memoria
18      );
19      Port(
20          clk, rst, REQ : in std_logic;
21          counter : in std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0);
22          en_count, read, write, loadA, loadB, ACK: out std_logic
23       );
24  end UC_B;
25
26  architecture Behavioral of UC_B is
27      type stato is (IDLE, AVVIA_LETTURA, ACQUISISCI_OP, AVVIA_SCRITTURA, END_HANDSHAKING);
28      signal stato_corrente : stato := IDLE;
29      signal stato_prossimo : stato := stato_corrente;
30
31      begin
32
33      mem_stato: process (clk, rst)
34      begin
35          if (rising_edge(clk)) then
36              if (rst = '1') then
37                  stato_corrente <= idle;
38              else
39                  stato_corrente <= stato_prossimo;
40              end if;
41          end if;
42      end process;
43
44      comb: process (stato_corrente, counter, REQ)
45      begin
46
47          case stato_corrente is
48              when IDLE =>
49                  en_count <= '0';
50                  ACK <= '0';
51                  loadA <= '0';
52                  loadB <= '0';
53                  read <= '0';
54                  write <= '0';
55
56                  if (REQ = '0') then
57                      stato_prossimo <= IDLE;
58
59                  elsif (REQ = '1') then
60                      stato_prossimo <= AVVIA_LETTURA;
61                  end if;
62
63              when AVVIA_LETTURA  =>
64                  read <= '1';
65                  ACK <= '1';
66                  stato_prossimo <= ACQUISISCI_OP;
```

```
67
68              when ACQUISISCI_OP =>
69                  loadA <= '1';
70                  loadB <= '1';
71                  read <= '0';
72                  stato_prossimo <= AVVIA_SCRITTURA;
73
74              when AVVIA_SCRITTURA =>
75                  write <= '1';
76                  loadA <= '0';
77                  loadB <= '0';
78
79                  if (REQ = '1') then
80                      stato_prossimo <= AVVIA_SCRITTURA;
81
82                  elsif (REQ = '0') then
83                      stato_prossimo <= END_HANDSHAKING;
84                  end if;
85
86              when END_HANDSHAKING =>
87                  en_count <= '1';
88                  ACK <= '0';
89                  write <= '0';
90                  stato_prossimo <= IDLE;
91
92              when OTHERS =>
93                   stato_prossimo <= IDLE;
94
95          end case;
96      end process;
97
98  end Behavioral;
```

Simulazione

Abbiamo implementato un Testbench per la simulazione del sistema.

```
entity testbench_sistema is
end testbench_sistema;

architecture tb of testbench_sistema is

    constant CLK_PERIOD : time := 10 ns; -- Periodo del clock
    signal CLK   : std_logic := '0';
    signal RST   : std_logic := '0';
    signal Start : std_logic := '0';
    signal stop  : std_logic := '0';

    -- Instanza del DUT (Device Under Test)
    component sistema
        Generic (
            N : integer := 8;
            M : integer := 4
        );
        Port (
            CLK   : in std_logic;
            RST   : in std_logic;
            Start : in std_logic;
            stop  : out std_logic
        );
    end component;

begin
    -- Instanza del sistema
    DUT: sistema
        generic map (
            N => 8,
            M => 4
        )
        port map (
            CLK   => CLK,
            RST   => RST,
            Start => Start,
            stop  => stop
        );

    -- Processo per generare il clock
    process
    begin
        while now < 3000 ns loop -- Simulazione per 1000 ns
            CLK <= '0';
            wait for CLK_PERIOD / 2;
            CLK <= '1';
            wait for CLK_PERIOD / 2;
        end loop;
        wait;
    end process;

    -- Processo per generare i segnali di test

    process
    begin
        -- Reset iniziale
        RST <= '1';
        wait for 20 ns;
        RST <= '0';
        wait for 20 ns;

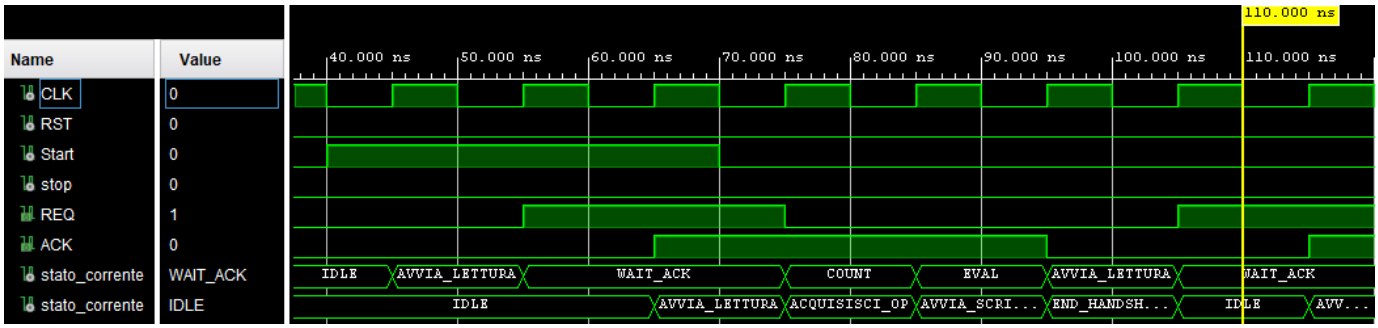
        -- Attivazione del segnale Start
        Start <= '1';
        wait for 30 ns;
        Start <= '0';

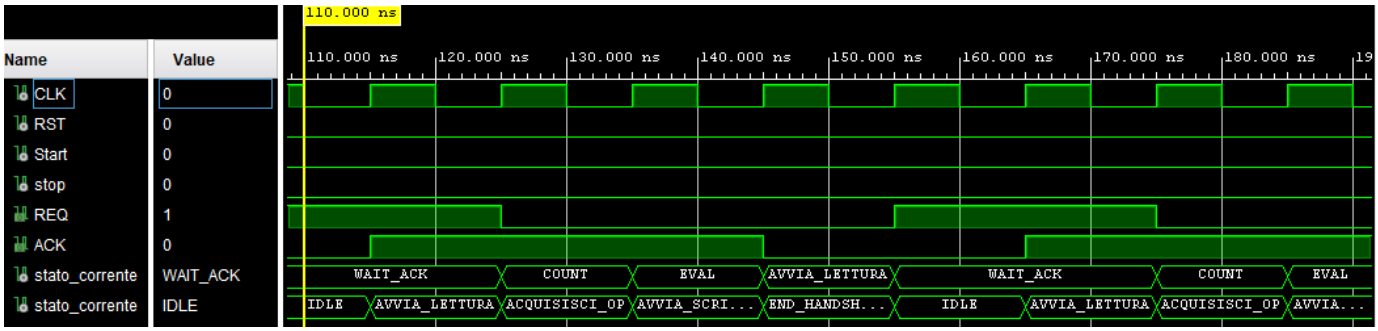
        -- Attendere il completamento dell'operazione
        wait for 3000 ns;

        -- Fine simulazione
        wait;
    end process;

end tb;
```

Riportiamo di seguito i risultati della simulazione.

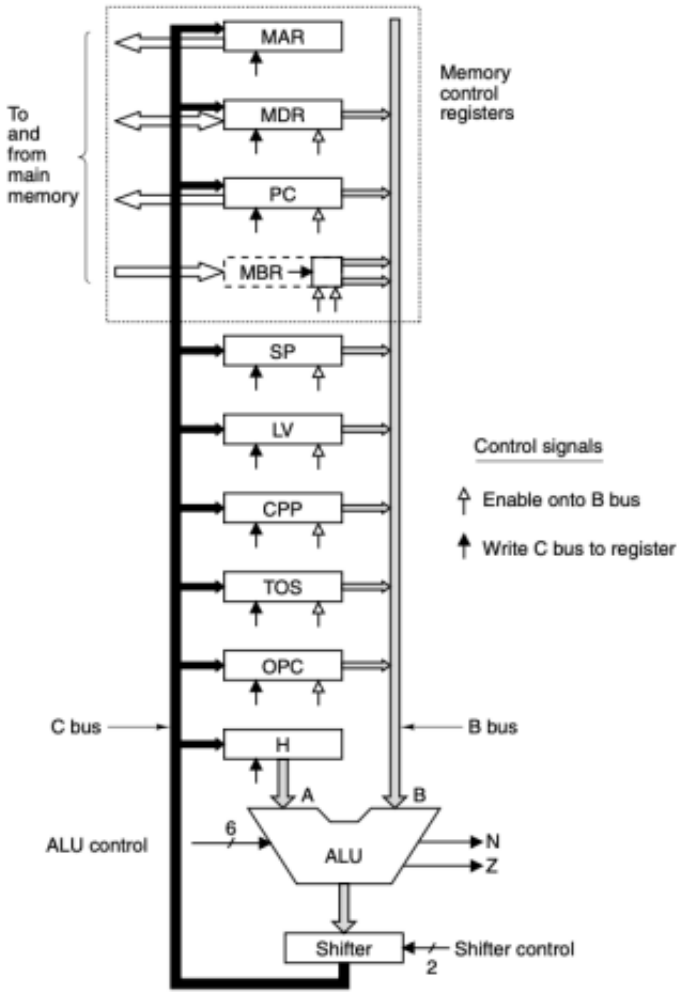




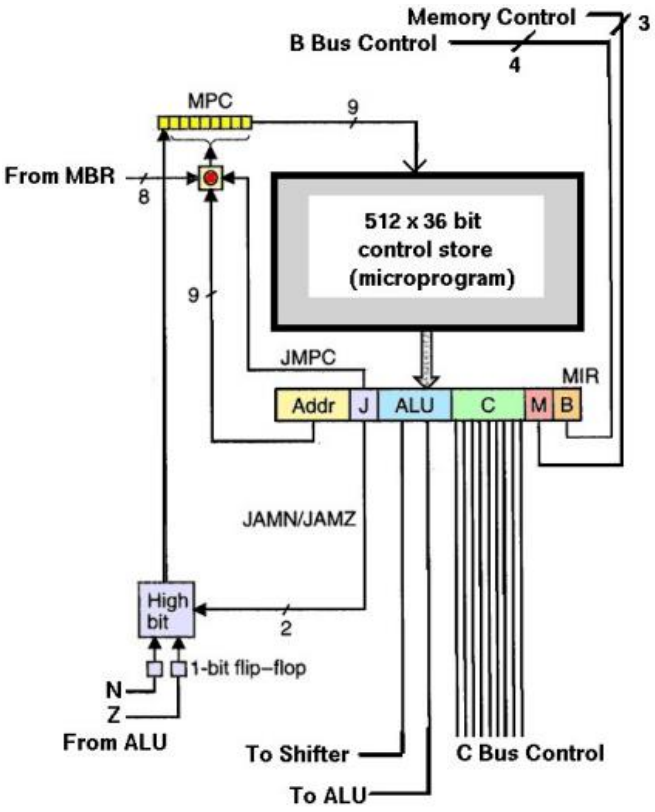
Capitolo 5: Processore

Il MIC-1 utilizza un modello di processore a stack, non fornendo nel modello di programmazione registri generali, ma supportando esclusivamente istruzioni con operandi provenienti dallo stack. In questo modo è possibile semplificare notevolmente l'instruction set del processore supportando esclusivamente operazioni con operandi impliciti, ottenendo però maggiori inefficienze causate da continui accessi in memoria.

Il datapath di questa architettura è composto da due bus fondamentali, il B bus connesso alle uscite dei registri e il C bus, che connette l'output della ALU all'ingresso dei registri.



La control unit è, invece, realizzata seguendo il modello microprogrammato: le sequenze di controllo sono memorizzate all'interno di una ROM 512x36 bit.

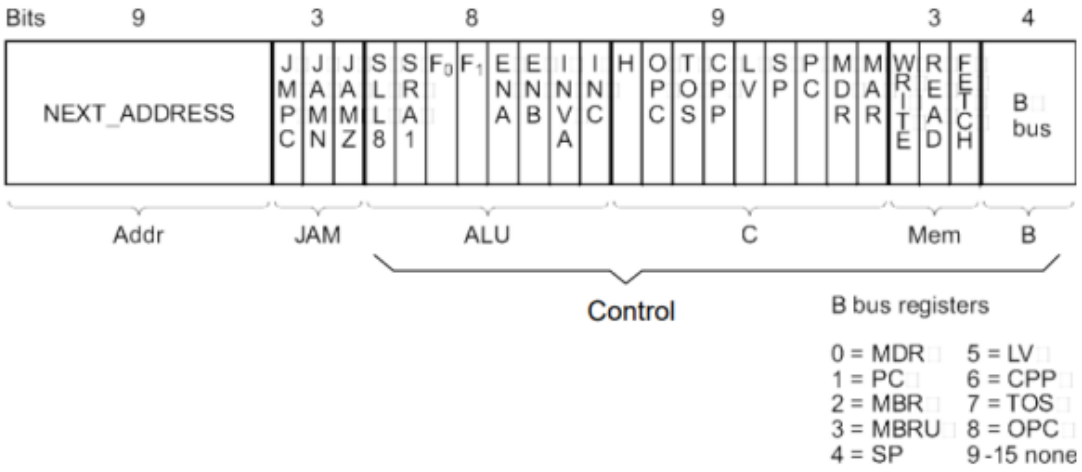


Per controllare il datapath del MIC-1 occorrono 29 segnali:

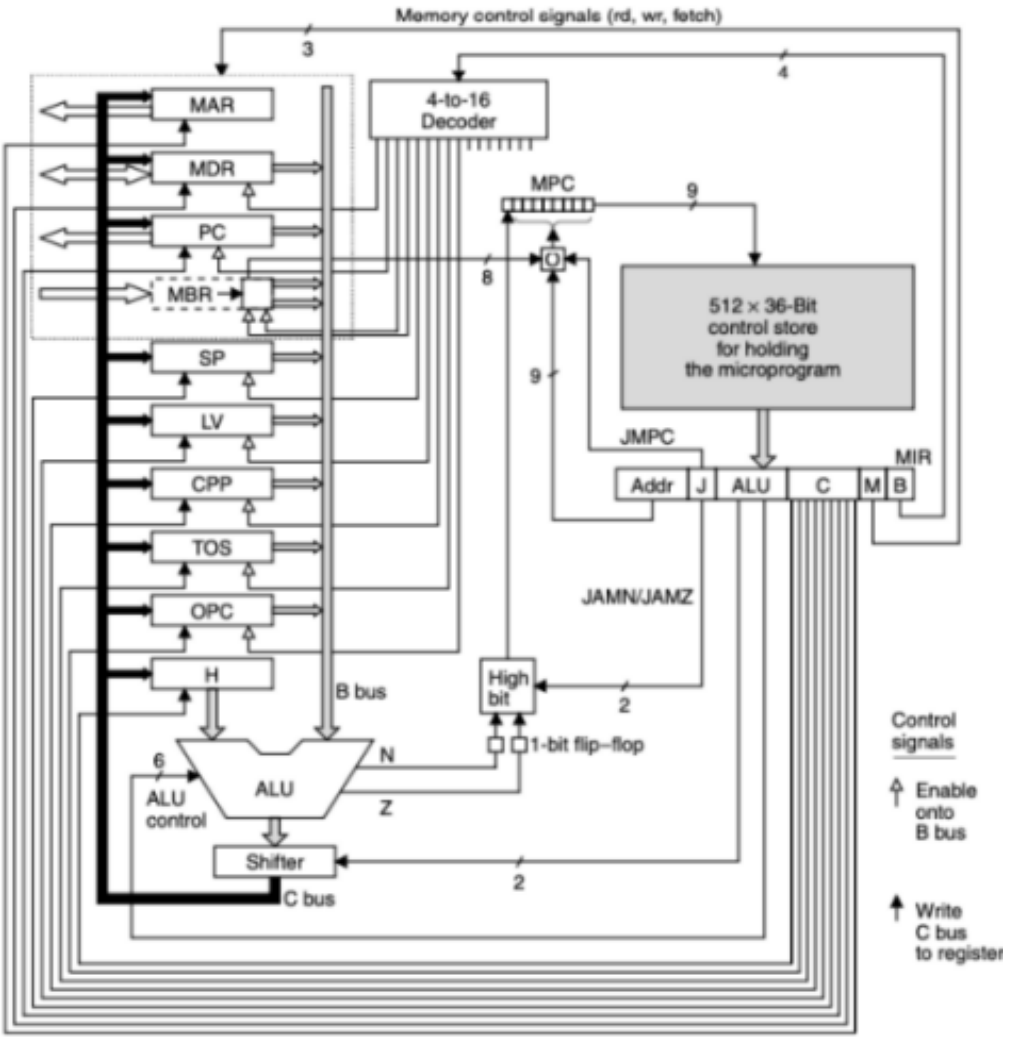
- 9 segnali per effettuare la selezione del bus C;
- 9 segnali per effettuare la selezione del bus B;
- 8 segnali per la gestione dell'ALU (in particolare 6 per l'ALU e 2 per lo shift register);
- 2 segnale di read/write per specificare se si vuole scrivere o leggere dalla memoria tramite la coppia MAR/MDR;
- 1 segnale per indicare il memory fetch del valore presente in PC/MBR:

Per ridurre il numero di bit di controllo (da 29 a 24) la nostra microarchitettura utilizza un decoder che con soli 4 bit è in grado di specificare quale dei 9 registri abilitare sul bus B.

Oltre a questi 24 segnali, si utilizzano 9 bit per l'indirizzo della prossima microistruzione e 3 bit per il JAM; perciò, ciascuna delle nostre microistruzioni avrà una lunghezza pari a 36 bit. Ovviamente solo un piccolo sottoinsieme delle 2^36 possibili microistruzioni verranno utilizzate dai microprogrammi.



Riportiamo, infine, l'architettura complessiva del MIC-1, comprendente data-path e unità di controllo.



Esercizio 9

A partire dall'implementazione fornita del processore operante secondo il modello IJVM,
a) si proceda all'analisi dell'architettura mediante simulazione e si approfondisca lo studio del suo funzionamento per due istruzioni a scelta,
b) si modifichi un codice operativo a scelta, documentando tutte le modifiche effettuate

Analisi BIPUSH

Nel seguente punto analizziamo nel dettaglio il funzionamento di alcune istruzioni IJVM. Consideriamo in primo luogo l'istruzione BIPUSH: questa istruzione esegue il push di un elemento in cima allo stack. È importante specificare che l'implementazione del microprogramma Mic-1 mantiene due utili invarianti, al termine dell'esecuzione dell'istruzione:

• il registro SP (Stack Pointer) contiene sempre l'indirizzo dell'elemento in testa allo stack;

• il registro TOS (Top of Stack) contiene sempre il valore dell'elemento in testa allo stack.

```
22
23 library ieee;
24 use ieee.std_logic_1164.all;
25 use ieee.numeric_std.all;
26
27 use work.common_defs.all;
28
29 --! Processor control store
30
31 --! The control store is a ROM used to store the processor microprogram.
32 entity control_store is
33   port (
34     --! Address of the desired word
35     address : in  ctrl_str_addr_type;
36     --! Content of the addressed word
37     word    : out ctrl_str_word_type
38     );
39 end entity control_store;
40
41 --! Dataflow architecture for the control store
42 architecture dataflow of control_store is
43
44   -- Constants
45   constant words : ctrl_str_type := (
46 --BEGIN_WORDS_ENTRY
47 0 => "00000100100000000000000000001001",
48 1 => "01011110000000000000000000001001",
49 2 => "00000000000000000000000000000000",
50 3 => "00000000000000000000000000000000",
51 4 => "00000000000000000000000000000000",
52 5 => "00000000000000000000000000000000",
53 6 => "00000011100000110101000001000000001",
54 7 => "00000100000000110101000001000010001",
55 8 => "00000100100000000000000000001001",
56 9 => "00000000010000110101000001000010001",
57 10 => "00000000000000000000000000000000",
58 11 => "00000000000000000000000000000000",
59 12 => "00000000000000000000000000000000",
60 13 => "00000000000000000000000000000000",
61 14 => "00000000000000000000000000000000",
62 15 => "00000000000000000000000000000000",
63 16 => "00001000100000110101000010010000100",
64 17 => "00000100100000011010100000100010001",
65 18 => "00000100100000010100001000010100010",
66 19 => "00000000000000000000000000000000",
67 20 => "00000000000000000000000000000000",
68 21 => "00001011000000010100100000000000101",
69 22 => "00001011100000111100000000010100011",
70 23 => "00001100000000110101000010010000100",
```

All’interno della control store, l’istruzione BIPUSH è all’indirizzo 0x10 cioè 16 in decimale (riga 63 in figura).

Nel caso della BIPUSH non sono necessari accessi in memoria, e l’istruzione può essere eseguita in tre cicli di clock, ossia in tre microistruzioni, le quali sono descritte di seguito in linguaggio mal:

1. bipush = 0x10:
2. SP = MAR = SP + 1
3. PC = PC + 1; fetch
4. MDR = TOS = MBR; wr; goto main

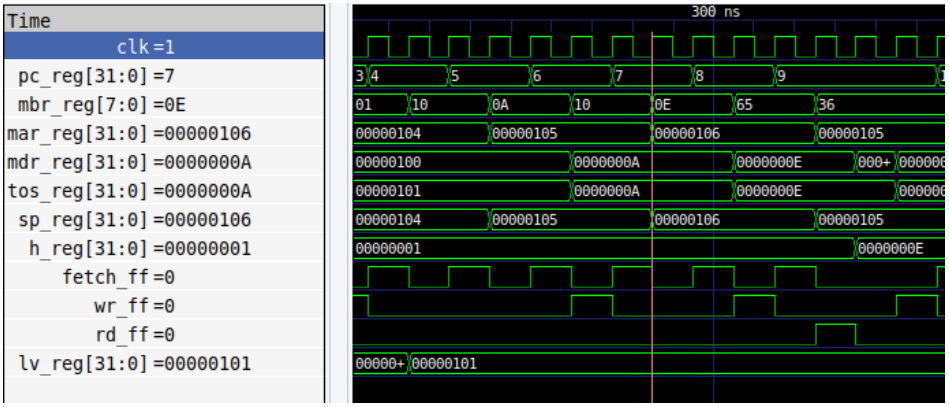
Nella riga 2) il contenuto di SP viene incrementato di 1 e scritto sia in SP che in MAR; SP punta ora alla posizione in cui dovrà essere scritto il valore da inserire in cima allo stack;

nella 3), il contenuto di PC viene incrementato e posto nuovamente all’interno di PC; viene effettuato il fetch per leggere la prossima istruzione.

nella 4), il dato da pushare, contenuto in MBR è posto sia in TOS che in MDR, viene poi effettuata una operazione di scrittura.

Questa istruzione ha una lunghezza complessiva di due byte: il primo corrisponde all’opcode del bipush, che, per quanto detto in precedenza corrisponde all’indirizzo della control-store dove è presente l’entry-point del microprogramma 0x10; il secondo, invece, è il valore da caricare sullo stack.

L’output della simulazione, visualizzata tramite GTKWave, è mostrato in figura. Effettuiamo un bipush per inserire sulla punta dello stack il valore 0XA e un secondo bipush per il valore 0XE.



Dopo 290ns, viene eseguita la prima operazione di bipush (0x10) e vengono aggiornati MAR e SP con il valore incrementato di SP, come riporta la prima microistruzione. Inoltre, notiamo come in questa fase MBR contenga il dato che si vuole porre in cima allo stack. Di Dopo l’incremento dello PC a 300ns, si procede alla fetch della prossima istruzione, seguita dall’aggiornamento di TOS, MDR con il valore presente in MBR. Infine viene asserito Wr.

Analisi SWAP

La seconda istruzione analizzata è la SWAP, la quale scambia i due elementi in cima allo stack.

```
92 => "00101110100000110110000010010100100",
93 => "00101111000000010100100000000000111",
94 => "00000100100000111111001000010100000",
95 => "00110000000000110110000000010100100",
96 => "00110000100000010100000000010000100",
97 => "00110001000000010100100000001000000",
98 => "00110001100000010100000000100000111",
99 => "00110010000000110110000000011000100",
```

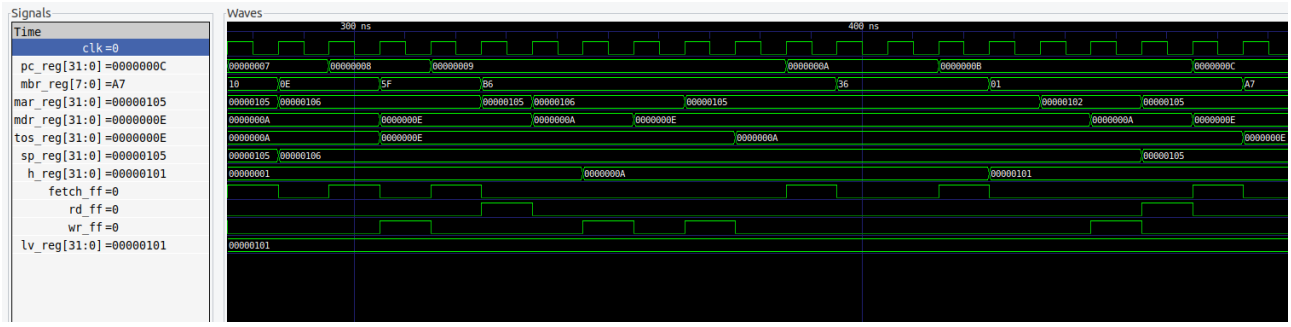
L'istruzione è memorizzata all'indirizzo 95 della control store. Il codice in linguaggio MAL è il seguente:

1. swap=0x5F:
2. MAR=SP-1; rd
3. MAR=SP
4. H=MDR; wr
5. MDR=TOS
6. MAR=SP-1; wr
7. TOS=H; goto main

Analizzando il codice riga per riga, supponiamo A in cima allo stack e B in seconda posizione.

2) MAR viene impostato all'indirizzo SP - 1, ovvero la seconda locazione di memoria. Rd significa che la memoria viene letta a questo indirizzo: il valore contenuto in SP – 1, B, verrà caricato in MDR.
3) MAR punta all'indirizzo di SP, cioè la locazione in cima allo stack..
4) Il valore B che si trova in MDR viene copiato nel registro temporaneo H e scritto in cima allo stack. Ora nella prima e seconda posizione dello stack è presente B.
5) Il valore attualmente in cima allo stack, A, presente in TOS, viene caricato in MDR
6) MAR viene impostato a SP - 1, cioè il secondo elemento dello stack. Wr indica che stiamo scrivendo in memoria il valore contenuto in MDR, ovvero A. Lo swap è stato effettuato.
7) TOS viene aggiornato con il valore precedentemente salvato in H, cioè B che ora è in cima. *goto main* indica che l'esecuzione torna al ciclo principale della CPU.

L'output della simulazione, visualizzata tramite GTKWave, è mostrato in figura.



Modifica del codice operativo

L'istruzione che abbiamo scelto di modificare è la IAND, memorizzata nella control store all'indirizzo 126 (decimale).

```
125 => "00000000000000000000000000000000",
126 => "00111111100000110110000010010100100",
127 => "01000000000000101100100000000000111",
128 => "01000000100000101100010000000000000",
129 => "00000100100000001100001000010010000",
130 => "00000000000000000000000000000000000",
131 => "00000000000000000000000000000000000",
```

La modifica apportata consiste nel negare entrambi gli operandi tra cui eseguiamo la AND alla riga 26: in questo modo il risultato non sarà una AND ma una **NOR**.

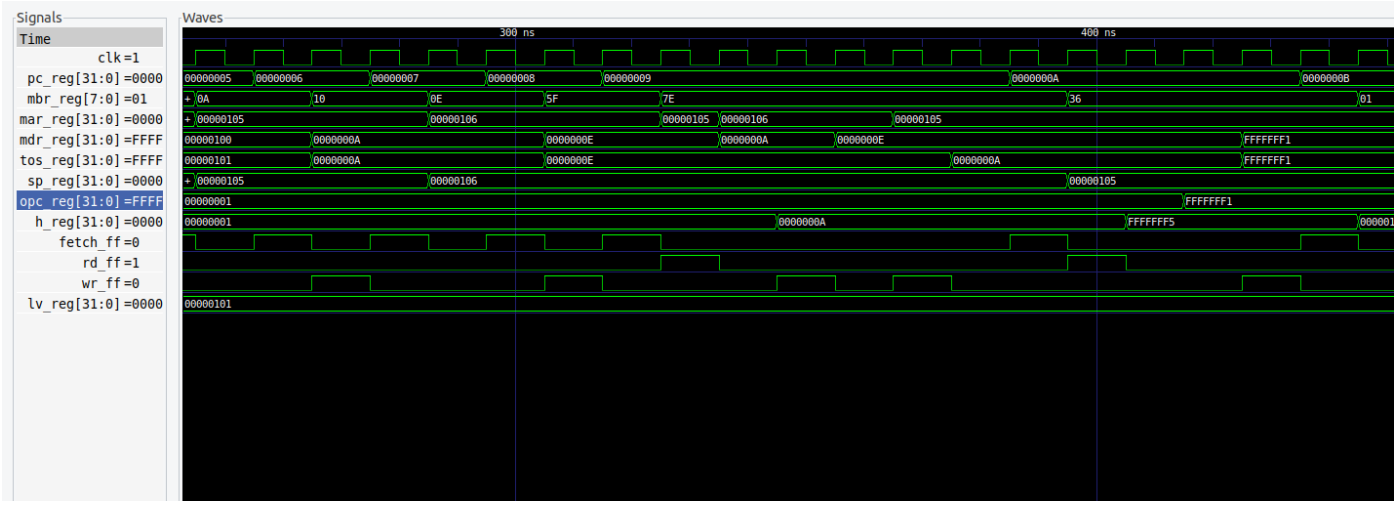
```
22 iand = 0x7E:
23          MAR = SP = SP - 1; rd
24          H = NOT TOS
25          OPC = NOT MDR
26          MDR = TOS = OPC AND H; wr; goto main
```

La IAND è memorizzata all'indirizzo 0x7E della control store ma negli indirizzi immediatamente seguenti non vi sono altre istruzioni, perciò è possibile aggiungere microistruzioni per negare gli operandi.

Il programma per lanciare la simulazione è stato modificato nel seguente modo:

```
 1 .main
 2 .var
 3 a
 4 .endvar
 5 BIPUSH 0xA
 6 BIPUSH 0xE
 7 SWAP
 8 IAND
 9 ISTORE a
10 HALT
11 .endmethod
```

L’output della simulazione è visualizzabile in figura, dove possiamo osservare che viene effettuata correttamente l’operazione di NOR.



Capitolo 6: Interfaccia Seriale

Esercizio 10

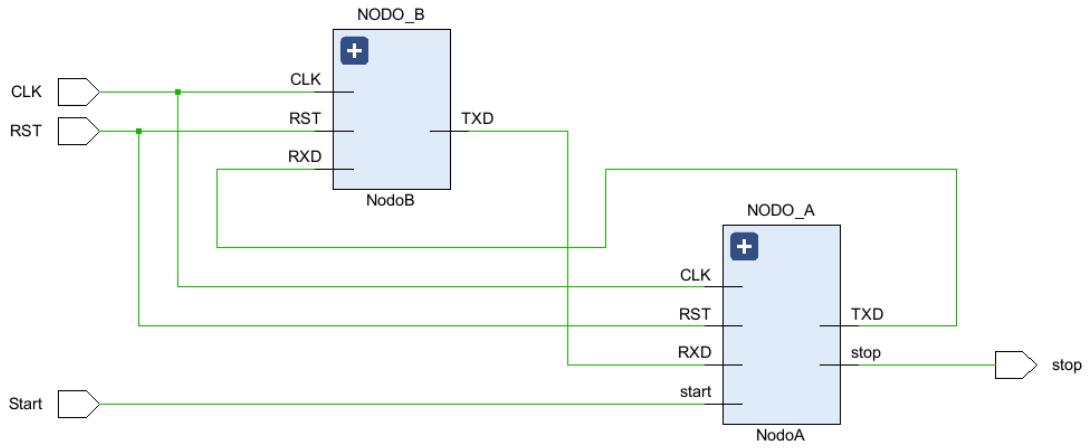
Partendo dall'implementazione fornita dalla Digilent di un dispositivo UART-RS232 (componente RS232RefComp.vhd), progettare, implementare e simulare in VHDL un sistema composto da 2 unità A e B che condividono lo stesso segnale di clock e comunicano tra loro mediante interfaccia seriale. Il sistema A contiene una ROM di 8 locazioni da 1 byte ciascuno, un contatore CONT_A per scandire le locazioni della ROM e una UART_A, mentre il sistema B contiene una memoria MEM di 8 locazioni da 1 byte ciascuno, un contatore CONT_B per scandire le locazioni della MEM e una UART_B. Quando un segnale WR viene asserito nell'unità A, viene prelevato un byte dalla ROM e inviato all'unità B, che dovrà riceverlo e salvarlo in MEM.

Progetto e architettura

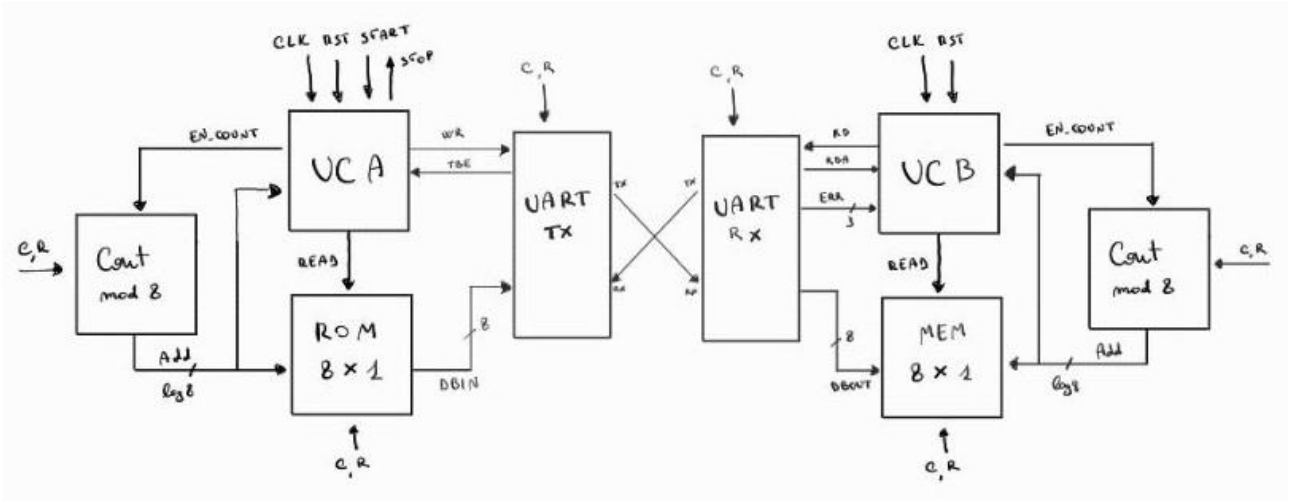
La UART (Universal Asynchronous Receiver Transmitter) è un dispositivo hardware che supporta la comunicazione seriale asincrona.

L'interfaccia UART consiste di 2 segnali TX e RX su cui viaggiano i dati trasmessi/ricevuti, più un segnale di GROUND, e prevede che la comunicazione avvenga secondo uno specifico protocollo che prevede una determinata struttura del frame trasmesso. Nel caso dell'UART, il pacchetto è composto da un bit di START, un byte di dati e un bit di STOP. Per garantire la corretta sincronizzazione, è comune che il ricevitore sovracampioni la linea, operando a una frequenza 8-16 volte superiore rispetto a quella del trasmettitore.

Il sistema include due nodi: A (trasmettitore) e B (ricevitore).

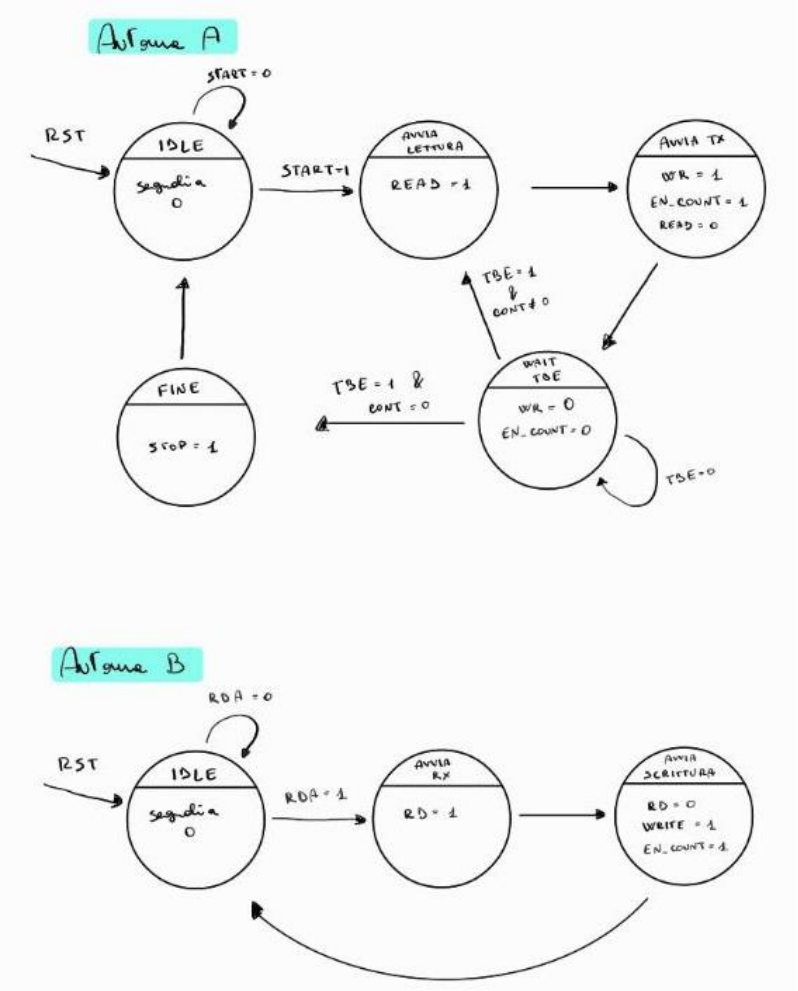


Complessivamente, il sistema si presenta come il seguente diagrammi a blocchi.



Ogni unità di controllo gestisce una UART, una ROM e un contatore modulo 8 per indirizzarla.

Il comportamento del nodo A e del nodo B è descritto dai seguenti automi.



Implementazione

Il nodo A è organizzato in parte di controllo e parte operativa, la quale include una ROM, un contatore e una UART. Di seguito riportiamo l'implementazione del nodo A.

```
entity NodoA is
    Generic (
        N : integer := 16; -- Numero di locazioni di memoria (modificabile)
        M : integer := 8   -- Larghezza in bit di ogni locazione
    );
    Port (
        CLK:        in std_logic;
        RST:        in std_logic;
        start:      in std_logic;
        RXD:        in std_logic;
        TXD :       out std_logic;
        stop:       out std_logic

     );
end NodoA;

architecture structuralNodoA of NodoA is

    component PO_A is
        Generic (
            N : integer := 64; -- Numero di locazioni di memoria
            M : integer := 8   -- Larghezza in bit di ogni locazione
        );
    Port (
            clk, rst : in std_logic;
            en_count, read : in std_logic;

            dataOUT : out std_logic;
            dataIN : in std_logic;
            WR : in std_logic;
            TBE : inout std_logic;
            counter: out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0)
        );
    end component;

    component UC_A is
        Generic (
            N : integer := 64 -- Numero di locazioni di memoria
        );
        Port(
            clk, rst, start : in std_logic;
            TBE : in std_logic;
            counter : in std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0);
            en_count, read, WR, stop: out std_logic
        );
    end component;

    signal Read_temp : std_logic := '0';
    signal counter_temp : std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0):= (others => '0');
    signal en_count_temp : std_logic := '0';
    signal WR_temp : std_logic := '0';
    signal TBE_temp : std_logic;
```

```
begin

    Parte_operativa: PO_A
        generic map (
            N => N,
            M => M
        )
        port map(
            CLK => CLK,
            RST => RST,
            en_count => en_count_temp,
            Read => Read_temp,
            dataOUT => TXD,
            dataIN => RXD,
            WR => WR_temp,
            TBE => TBE_temp,
            counter => counter_temp
        );

    Parte_controllo: UC_A
        generic map (
            N => N
        )
        port map(
            CLK => CLK,
            RST => RST,

           Start => Start,
           TBE => TBE_temp,
           counter => counter_temp,
           en_count => en_count_temp,
           Read => Read_temp,
           WR => WR_temp,
           stop => stop
        );

end ;
```

La parte operativa è stata implementata come nel seguente codice.

```
entity PO_A is
    Generic (
        N : integer := 64; -- Numero di locazioni di memoria
        M : integer := 8   -- Larghezza in bit di ogni locazione
    );
    Port (
        clk, rst : in std_logic;
        en_count, read : in std_logic;
        dataOUT : out std_logic;
        dataIN : in std_logic;
        WR : in std_logic;
        TBE : inout std_logic;
        counter: out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0)
    );
end PO_A;

architecture structural_A of PO_A is

    component ContMod_N is
        Generic (
            N : integer := 64 -- Modulo del contatore
        );
        Port (
            CLK:         in std_logic;  -- clock
            RST:         in std_logic;  -- reset asincrono
            en_count:    in std_logic; --segnale di conteggio
```

```
            dataOUT:     out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0) -- uscita di conteggio
        );
    end component;

    component ROM_N_M is
        Generic (
            N : integer := 64; -- Numero di locazioni di memoria
            M : integer := 8   -- Larghezza in bit di ogni locazione
        );
        Port (
            CLK     : in std_logic;
            Address : in std_logic_vector (integer(ceil(log2(real(N))))-1 downto 0);
            Read    : in std_logic;
            dataOUT : out std_logic_vector (M-1 downto 0)
        );
    end component;

    component Rs232RefComp is
    Port (
           TXD     : out std_logic     := '1';
           RXD     : in  std_logic;
           CLK     : in  std_logic;                    --Master Clock
           DBIN    : in  std_logic_vector (7 downto 0);--Data Bus in
           DBOUT : out std_logic_vector (7 downto 0);  --Data Bus out
           RDA : inout std_logic;                      --Read Data Available(1 quando il dato Ã¨ disponibile nel registro rdReg)
           TBE : inout std_logic   := '1';             --Transfer Bus Empty(1 quando il dato da inviare Ã¨ stato caricato nello shift reg.
           RD      : in  std_logic;                    --Read Strobe(se 1 significa "leggi" --> fa abbassare RDA)
           WR      : in  std_logic;                    --Write Strobe(se 1 significa "scrivi" --> fa abbassare TBE)
           PE      : out std_logic;                    --Parity Error Flag
           FE      : out std_logic;                    --Frame Error Flag
           OE      : out std_logic;                    --Overwrite Error Flag
           RST     : in  std_logic := '0');            --Master Reset
    end component;

    signal Address_temp: std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0):= (others => '0');
    signal ROM_dataOUT_temp: std_logic_vector (M-1 downto 0);
    signal DBOUT_temp: std_logic_vector (7 downto 0);
    signal RDA_temp: std_logic;
    signal RD_temp: std_logic;
    signal Error_temp: std_logic_vector(2 downto 0);

    begin

    counter <= Address_temp;

    --UART
    Uart_A : Rs232RefComp
        Port map(
            TXD => dataOUT,
            RXD => dataIN,
            CLK => clk,
            DBIN => ROM dataOUT temp,
            DBOUT => DBOUT_temp,
            RDA => RDA_temp,
            TBE => TBE,
            RD => RD_temp,
            WR => WR,
            PE => Error_temp(0),
            FE => Error_temp(1),
            OE => Error_temp(2),
            RST => rst
        );

    --COUNTER
    ContModN_A : ContMod_N
         Generic map(
            N => N
         )
        Port map(
            clk => clk,
            rst => rst,
            en_count => en_count,
            dataOUT => Address_temp
        );

    ROM_A : ROM_N_M
        Generic map(
            N => N,
```

```
111             M => M
112         )
113         Port map(
114             CLK => clk,
115             Address => Address_temp,
116             Read => read,
117             dataOUT => ROM_dataOUT_temp
118         );
119 
120 end Structural_A;
```

Mentre la parte di controllo implementa l'automa progettato con cinque stati e reset sincrono.

```vhdl
entity UC_A is
    Generic (
        N : integer := 64 -- Numero di locazioni di memoria
    );
    Port(
        clk, rst, start : in std_logic;
        TBE : in std_logic;
        counter : in std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0);
        en_count, read, WR, stop: out std_logic
     );
end UC_A;

architecture Behavioral of UC_A is
    type stato is (IDLE, AVVIA_LETTURA, WAIT_TBE, AVVIA_TX, FINE);
    signal stato_corrente : stato := IDLE;
    signal stato_prossimo : stato := stato_corrente;

    begin

    mem_stato: process (clk, rst)
    begin
        if (rising_edge(clk)) then
            if (rst = '1') then
                stato_corrente <= idle;
            else
                stato_corrente <= stato_prossimo;
            end if;
        end if;
    end process;

    comb: process (stato_corrente, start, counter, TBE)
    begin

        case stato_corrente is
            when IDLE =>
                en_count <= '0';
                WR <= '0';
                read <= '0';
                stop <= '0';

                if (start='0') then
                    stato_prossimo <= IDLE;

                elsif (start='1') then
                    stato_prossimo <= AVVIA_LETTURA;
                end if;

            when AVVIA_LETTURA  =>
                read <= '1';
                stato_prossimo <= AVVIA_TX;

            when AVVIA_TX =>
                en_count <= '1';
                WR <= '1';
                read <= '0';
                stato_prossimo <= WAIT_TBE;

            when WAIT_TBE =>
                WR <= '0';
                en_count <= '0';

                if (TBE = '0') then
                    stato_prossimo <= WAIT_TBE;

                elsif (TBE = '1' and counter /= std_logic_vector(to_unsigned(0, counter'length))) then
                    stato_prossimo <= AVVIA_LETTURA;

                elsif (TBE = '1' and counter = std_logic_vector(to_unsigned(0, counter'length))) then
                    stato_prossimo <= FINE;

                end if;

            when FINE =>
                stop <= '1';
                stato_prossimo <= IDLE;

            when OTHERS =>
                stato_prossimo <= IDLE;


        end case;
    end process;

end Behavioral;
```

Il nodo B allo stesso modo è organizzato in parte di controllo e parte operativa, la quale è costituita da una memoria MEM, un contatore e una UART.

```
entity NodoB is
    Generic (
        N : integer := 16; -- Numero di locazioni di memoria (modificabile)
        M : integer := 8   -- Larghezza in bit di ogni locazione
    );
    Port (
        CLK:        in std_logic;
        RST:        in std_logic;
        RXD :       in std_logic;
        TXD :       out std_logic

     );
end NodoB;

architecture structuralNodoB of NodoB is

    component PO_B is
        Generic (
            N : integer := 64; -- Numero di locazioni di memoria
            M : integer := 8   -- Larghezza in bit di ogni locazione
        );
        Port (
            clk, rst : in std_logic;
            en_count, write : in std_logic;
            RD: in std_logic;
            dataIN : in std_logic;

             dataOUT: out std_logic;
             RDA: inout std_logic;
             Error: out std_logic_vector(2 downto 0)
   --        counter: out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0)
        );
    end component;

    component UC_B is
        Generic (
            N : integer := 64 -- Numero di locazioni di memoria
        );
        Port(
            clk, rst: in std_logic;
            RDA: in std_logic;
            error : in std_logic_vector(2 downto 0);
            en_count, RD, write: out std_logic
        );
    end component;

    signal RD_temp : std_logic := '0';
    signal Write_temp : std_logic := '0';
    signal en_count_temp : std_logic := '0';
    signal Error_temp : std_logic_vector(2 downto 0) := (others => '0');
    signal RDA_temp : std_logic := '0';

begin

    Parte_operativa: PO_B
        generic map (
            N => N,
            M => M
        )
        port map(
            CLK => CLK,
            RST => RST,
            en_count => en_count_temp,
            write => Write_temp,
            RD => RD_temp,
            dataIN => RXD,
            dataOUT => TXD,
            RDA => RDA_temp,
            Error => Error_temp
        );

    Parte_controllo: UC_B
        generic map (
            N => N
        )
        port map(
            CLK => CLK,
            RST => RST,
```

```
            RDA => RDA_temp,
            Error => Error_temp,
            en_count => en_count_temp,
            RD => RD_temp,
            write => Write_temp
        );

end ;
```

La parte operativa è stata implementata come nel seguente codice.

```
entity PO_B is
    Generic (
        N : integer := 64; -- Numero di locazioni di memoria
        M : integer := 8   -- Larghezza in bit di ogni locazione
    );
    Port (
        clk, rst : in std_logic;
        en_count, write : in std_logic;
        RD: in std_logic;
        dataIN : in std_logic;
        dataOUT: out std_logic;
        RDA: inout std_logic;
        Error: out std_logic_vector(2 downto 0)
--        counter: out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0)
    );
end PO_B;

architecture structural_B of PO_B is

    component ContMod_N is
        Generic (
            N : integer := 64 -- Modulo del contatore
        );
        Port (
            CLK:         in std_logic;  -- clock
            RST:         in std_logic;  -- reset asincrono

            en_count:    in std_logic; --segnale di conteggio
            dataOUT:     out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0) -- uscita di conteggio
        );
    end component;

    component MEM is
        Generic (
            N : integer := 64;  -- Numero di locazioni di memoria (modificabile)
            M : integer := 8    -- Numero di bit per locazione di memoria (modificabile)
        );
        Port (
            CLK     : in std_logic;
            Address : in std_logic_vector (integer(ceil(log2(real(N))))-1 downto 0);
            Write   : in std_logic;
            Read    : in std_logic;
            dataIN  : in std_logic_vector (M-1 downto 0);
            dataOUT : out std_logic_vector (M-1 downto 0)
        );
    end component;

    component Rs232RefComp is
        Port (
            TXD     : out std_logic     := '1';
            RXD     : in  std_logic;
            CLK     : in  std_logic;                    --Master Clock
            DBIN    : in  std_logic_vector (7 downto 0);--Data Bus in
```

```
59              DBOUT : out std_logic_vector (7 downto 0);  --Data Bus out
60              RDA : inout std_logic;                      --Read Data Available(1 quando il dato Ã¨ disponibile nel registro rdReg)
61              TBE : inout std_logic   := '1';             --Transfer Bus Empty(1 quando il dato da inviare Ã¨ stato caricato nello shift reg
62              RD      : in  std_logic;                    --Read Strobe(se 1 significa "leggi" --> fa abbassare RDA)
63              WR      : in  std_logic;                    --Write Strobe(se 1 significa "scrivi" --> fa abbassare TBE)
64              PE      : out std_logic;                    --Parity Error Flag
65              FE      : out std_logic;                    --Frame Error Flag
66              OE      : out std_logic;                    --Overwrite Error Flag
67              RST     : in  std_logic := '0');            --Master Reset
68      end component;
69
70
71      signal temp_Address: std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0):= (others => '0');
72      signal MEM_dataIN_temp: std_logic_vector (M-1 downto 0);
73      signal temp_MEM_out: std_logic_vector(M-1 downto 0):= (others => '0');
74      signal MEM_read_temp: std_logic;
75      signal DBIN_temp: std_logic_vector (7 downto 0);
76      signal TBE_temp: std_logic;
77      signal WR_temp: std_logic;
78
79      begin
80
81  --      counter <= temp_Address;
82
83      --UART
84      Uart_B : Rs232RefComp
```

```
85          Port map(
86              TXD => dataOUT,
87              RXD => dataIN,
88              CLK => clk,
89              DBIN => DBIN_temp,
90              DBOUT => MEM_dataIN_temp,
91              RDA => RDA,
92              TBE => TBE_temp,
93              RD => RD,
94              WR => WR_temp,
95              PE => Error(0),
96              FE => Error(1),
97              OE => Error(2),
98              RST => rst
99          );
100
101     --COUNTER
102     ContModN_B : ContMod_N
103          Generic map(
104             N => N
105          )
106         Port map(
107             Clk => clk,
108             rst => rst,
109             en_count => en_count,
110             dataOUT => temp_Address
111         );
112
113     MEM_B : MEM
114         Generic map(
115             N => N,
116             M => M
117         )
118         Port map(
119             CLK => clk,
120             Address => temp_Address,
121             Write => write,
122             Read => MEM_read_temp,
123             dataIN => MEM_dataIN_temp,
124             dataOUT => temp_MEM_out
125         );
126
127 end Structural_B;
```

Mentre la parte di controllo implementa l'automa progettato con tre stati e reset sincrono.

```
15  entity UC_B is
16      Generic (
17          N : integer := 64 -- Numero di locazioni di memoria
18      );
19      Port(
20          clk, rst: in std_logic;
21          RDA: in std_logic;
22          error : in std_logic_vector(2 downto 0);
23          en_count, RD, write: out std_logic
24       );
25  end UC_B;
26
27  architecture Behavioral of UC_B is
28      type stato is (IDLE, AVVIA_RX, AVVIA_SCRITTURA);
29      signal stato_corrente : stato := IDLE;
30      signal stato_prossimo : stato := stato_corrente;
31
32      begin
33
34      mem_stato: process (clk, rst)
35      begin
36          if (rising_edge(clk)) then
37              if (rst = '1') then
38                  stato_corrente <= idle;
39              else
40                  stato_corrente <= stato_prossimo;

41              end if;
42          end if;
43      end process;
44
45      comb: process (stato_corrente, RDA)
46      begin
47
48          case stato_corrente is
49              when IDLE =>
50                  en_count <= '0';
51                  RD <= '0';
52                  write <= '0';
53
54                  if (RDA = '0') then
55                      stato_prossimo <= IDLE;
56
57                  elsif (RDA = '1') then
58                      stato_prossimo <= AVVIA_RX;
59                  end if;
60
61              when AVVIA_RX  =>
62                  RD <= '1';
63                  stato_prossimo <= AVVIA_SCRITTURA;
64
65              when AVVIA_SCRITTURA =>
66                  RD <= '0';

67                  write <= '1';
68                  en_count <= '1';
69                  stato_prossimo <= IDLE;
70
71              when OTHERS =>
72                  stato_prossimo <= IDLE;
73
74          end case;
75      end process;
76
77  end Behavioral;
```

Si procede, dunque, a implementare in maniera strutturale l'intero sistema.

```vhdl
entity sistema is
    Generic (
        N : integer := 16; -- Numero di locazioni di memoria (modificabile)
        M : integer := 8   -- Larghezza in bit di ogni locazione
    );
    Port (
        CLK:        in std_logic;
        RST:        in std_logic;
        Start:      in std_logic;
        stop:       out std_logic
     );
end sistema;

architecture structural_sistema of sistema is

    component NodoA is
        Generic (
            N : integer := 16; -- Numero di locazioni di memoria (modificabile)
            M : integer := 8   -- Larghezza in bit di ogni locazione
        );
        Port (
            CLK:        in std_logic;
            RST:        in std_logic;
            start:      in std_logic;
            RXD:        in std_logic;
            TXD :       out std_logic;

            stop:       out std_logic
        );
    end component;

    component NodoB is
        Generic (
            N : integer := 16; -- Numero di locazioni di memoria (modificabile)
            M : integer := 8   -- Larghezza in bit di ogni locazione
        );
        Port (
            CLK:        in std_logic;
            RST:        in std_logic;
            RXD :       in std_logic;
            TXD :       out std_logic
        );
    end component;

    signal A_TXD_temp : std_logic := '0';
    signal A_RXD_temp : std_logic := '0';

begin

    NODO_A: NodoA
        generic map (
            N => N,
            M => M

        )
        port map(
            CLK => CLK,
            RST => RST,
            start => start,
            RXD => A_RXD_temp,
            TXD => A_TXD_temp,
            stop => stop
        );

     NODO_B: NodoB
        generic map (
            N => N,
            M => M
        )
        port map(
            CLK => CLK,
            RST => RST,
            RXD => A_TXD_temp,
            TXD => A_RXD_temp
        );

end ;
```

Simulazione

Riportiamo di seguito il codice per la simulazione del sistema.

```
entity sistema_tb is
end sistema_tb;

architecture behavioral of sistema_tb is
    -- Dichiarazione del DUT (Device Under Test)
    component sistema
        Generic (
            N : integer := 16;
            M : integer := 8
        );
        Port (
            CLK:    in std_logic;
            RST:    in std_logic;
            Start:  in std_logic;
            stop:   out std_logic
        );
    end component;

    -- Segnali di test
    signal CLK_tb   : std_logic := '0';
    signal RST_tb   : std_logic := '1';
    signal Start_tb : std_logic := '0';
    signal Stop_tb  : std_logic;

    -- Parametri temporali
    constant CLK_PERIOD : time := 20 ns;  -- Clock da 50 MHz

begin
    -- Istanza del sistema da testare
    DUT: sistema
        generic map (
            N => 8,
            M => 8
        )
        port map (
            CLK => CLK_tb,
            RST => RST_tb,
            Start => Start_tb,
            stop => Stop_tb
        );

    -- Processo per generare il clock
    process
    begin
        while now < 11176000 ns loop  -- Simuliamo per 11.176.000 ns
            CLK_tb <= '0';
            wait for CLK_PERIOD / 2;
            CLK_tb <= '1';
            wait for CLK_PERIOD / 2;
        end loop;
        wait;
    end process;

    -- Processo per stimolare i segnali di ingresso
    process
    begin
        -- Fase di reset
        RST_tb <= '1';
        wait for 20 ns;  -- Aspettiamo 20 ns
        RST_tb <= '0';

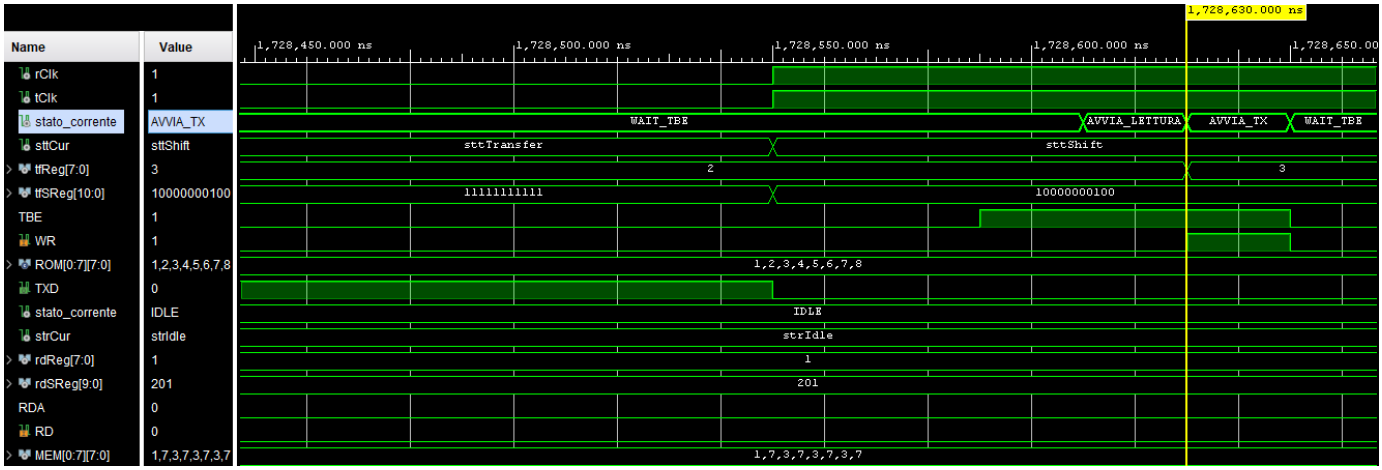
        -- Avvio della comunicazione
        wait for 30 ns;
        Start_tb <= '1';
        wait for 20 ns;
        Start_tb <= '0';

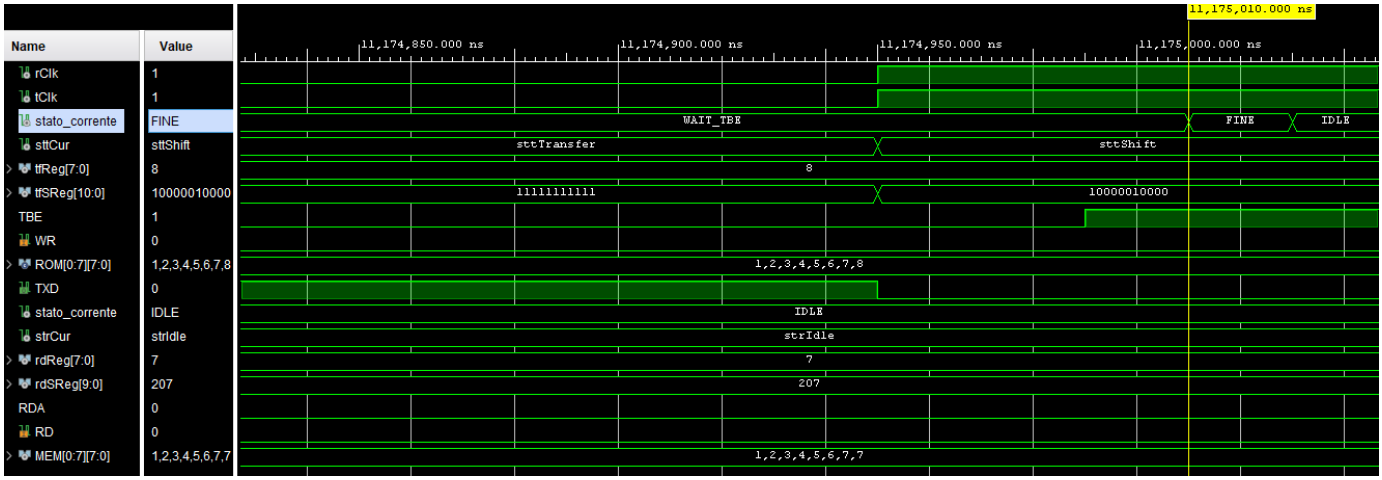
        -- Aspettiamo per osservare la comunicazione tra i nodi
        wait for 200 ns;

        -- Fine della simulazione
        report "Test completato" severity note;
        wait;
    end process;

end behavioral;
```

Infine, riportiamo i risultati di tale simulazione.





Capitolo 7: Switch Multistadio

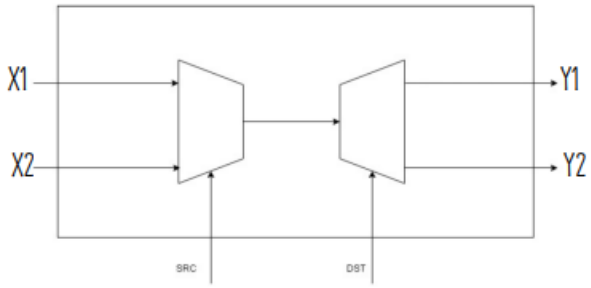
Esercizio 11 – switch multistadio

Esercizio 11.1

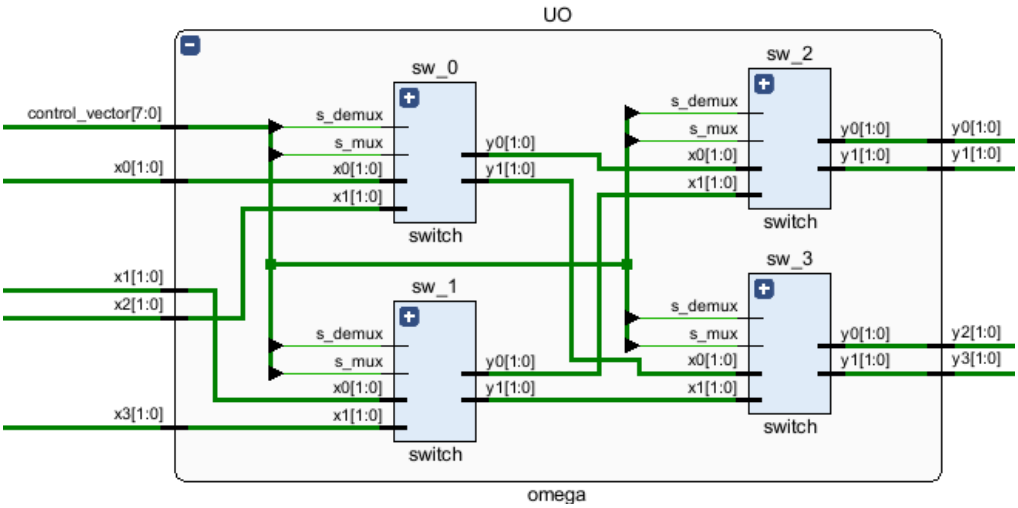
Progettare ed implementare in VHDL uno switch multistadio secondo il modello omega network. Lo switch deve consentire lo scambio di messaggi di 2 bit ciascuno da un nodo sorgente a un nodo destinazione in una rete con 4 nodi, implementando uno schema a priorità fissa fra i nodi (es. nodo 1 più prioritario, con priorità decrescenti fino al nodo 4).

Progetto e architettura

Il componente base dell'omega network è lo **switch** che in generale connette due sorgenti X1 e X2 con due destinazioni Y1 e Y2 sfruttando la connessione tra l'uscita di un MUX 2:1 e l'ingresso di un DEMUX 1:2. In particolare, vederemo una rete di interconnessione multistadio NxN che prende il nome di **omega network**, la quale sfrutta l'algoritmo *perfect shuffling* per realizzare il collegamento tra gli switch posizionati nei l $log_2 N$ stadi intermedi.



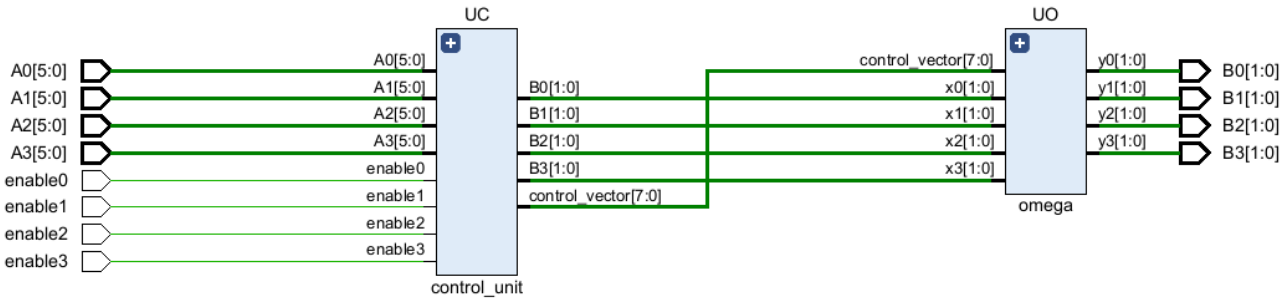
L'unità operativa consiste in quattro switch connessi come in figura utilizzando dei segnali temporanei. Gli indirizzi sorgente e destinazione fungono da segnali di selezione rispettivamente dei multiplexer e dei demultiplexer.



I messaggi da inoltrare attraverso la Omega Network sono composti ciascuno da 6 bit: 2 di indirizzo sorgente, 2 di dato e 2 di indirizzo destinazione.

L'Unità di Controllo stabilisce la priorità in caso di collisioni. Abbiamo deciso di rendere più prioritario il messaggio A3 e, a decrescere, il meno prioritario sarà il messaggio A0.

Il sistema complessivo, invece, si presenta nella seguente configurazione con unità di controllo connessa alla parte operativa:



Implementazione

Partiamo dall'implementazione del componente base, lo switch. Esso è composto da un multiplexer 2:1 seguito da un demultiplexer 1:2. L'approccio scelto è quello strutturale, utilizzando un segnale temporaneo per connettere l'uscita del mux all'ingresso del demux.

```
6  entity switch is
7    Port (
8      x0,x1 : in std_logic_vector (1 downto 0);
9      s_mux, s_demux : in std_logic;
10     y0 : out std_logic_vector(1 downto 0);
11     y1 : out std_logic_vector(1 downto 0)
12   );
13 end switch;
14
15 architecture Structural of switch is
16     component mux_21
17     Port (
18         a0  : in std_logic_vector(1 downto 0);
19         a1  : in std_logic_vector(1 downto 0);
20         s   : in std_logic;
21         y   : out std_logic_vector(1 downto 0));
22     end component;
23
24     component demux_12
25     Port (
26         a : in std_logic_vector(1 downto 0);
27         sel: in std_logic;
28         b0, b1 : out std_logic_vector(1 downto 0));
29     end component;
```

```
31      signal temp : std_logic_vector(1 downto 0);
32
33  begin
34      mux : mux_21
35      Port map(
36          a0  => x0,
37          a1  => x1,
38          s   => s_mux,
39          y   => temp );
40
41      demux :  demux_12
42      Port map(
43          a => temp,
44          sel => s_demux,
45          b0  => y0,
46          b1  =>  y1 );
47
48  end Structural;
```

Sfruttando lo switch elementare, è stata realizzata l'Omega Network composta da 4 switch in due stadi da 2 switch ciascuno, come riportato in fase di progettazione.

```
 7  entity omega is
 8    Port (
 9      x0, x1, x2, x3 :        in std_logic_vector (1 downto 0);
10      control_vector :        in std_logic_vector (7 downto 0);
11      y0, y1, y2, y3 :        out std_logic_vector(1 downto 0)
12    );
13  end omega;
14
15  architecture Structural of omega is
16      component switch
17          Port (
18              x0,x1 : in std_logic_vector (1 downto 0);
19              s_mux, s_demux : in std_logic;
20              y0 : out std_logic_vector(1 downto 0);
21              y1 : out std_logic_vector(1 downto 0)
22          );
23       end component;
24
25       signal u0, u1, u2, u3 : std_logic_vector(1 downto 0);
26  begin
27      sw_0 : switch
28          Port map(
29              x0 => x0,
30              x1 => x2,
31              s_mux => control_vector(7),
```

```
            s_demux => control_vector(6),
            y0 => u0,
            y1 => u1
        );

    sw_1 : switch
        Port map(
            x0 => x1,
            x1 => x3,
            s_mux => control_vector(5),
            s_demux => control_vector(4),
            y0 => u2,
            y1 => u3
        );

    sw_2 : switch
        Port map(
            x0 => u0,
            x1 => u2,
            s_mux => control_vector(3),
            s_demux => control_vector(2),
            y0 => y0,
            y1 => y1
        );

    sw_3 : switch
        Port map(
            x0 => u1,
            x1 => u3,
            s_mux => control_vector(1),
            s_demux => control_vector(0),
            y0 => y2,
            y1 => y3
        );
end Structural;
```

Ogni switch utilizza 2 bit diversi del control_vector al fine di poter gestire correttamente le collisioni. Tale control_vector è fornito dall'unità di controllo, implementata come segue.

```
entity control_unit is
  Port (
    A0, A1 , A2, A3 : in std_logic_vector(5 downto 0);
    enable0, enable1, enable2, enable3 : in std_logic;
    B0, B1, B2, B3 : out std_logic_vector(1 downto 0);
    control_vector : out std_logic_vector (7 downto 0)
  );
end control_unit;

architecture Behavioural of control_unit is

  signal control_SW0 : std_logic_vector(1 downto 0);
  signal control_SW1 : std_logic_vector(1 downto 0);
  signal control_SW2 : std_logic_vector(1 downto 0);
  signal control_SW3 : std_logic_vector(1 downto 0);
  signal dest_nodo0 : std_logic_vector(1 downto 0) := (others => 'U');
  signal dest_nodo1 : std_logic_vector(1 downto 0) := (others => 'U');
  signal dest_nodo2 : std_logic_vector(1 downto 0) := (others => 'U');
  signal dest_nodo3 : std_logic_vector(1 downto 0) := (others => 'U');


begin

  control_vector <= control_SW0 & control_SW1 & control_SW2 & control_SW3;

```

```
31  control_stadio1: process (A0, A1, A2, A3, enable0, enable1, enable2, enable3)
32  begin
33
34    -- Controllo SW0
35    if (enable0 = '1' and enable2 = '0') then --passa A0
36        control_SW0 <= A0(5) & A0(1);
37        B0 <= A0(3 downto 2);
38        --aggiorno destinazioni
39        dest_nodo0 <= A0(1 downto 0);
40        dest_nodo2 <= "--";
41
42    elsif ((enable0 = '0' or enable0 = '1') and enable2 = '1') then --passa A2
43        control_SW0 <= A2(5) & A2(1);
44        B2 <= A2(3 downto 2);
45        --aggiorno destinazioni
46        dest_nodo2 <= A2(1 downto 0);
47        dest_nodo0 <= "--";
48
49    end if;
50
51    -- Controllo SW1
52    if (enable1 = '1' and enable3 = '0') then --passa A1
53        control_SW1 <= A1(5) & A1(1);
54        B1 <= A1(3 downto 2);
55        --aggiorno destinazioni
56        dest_nodo1 <= A1(1 downto 0);
57        dest_nodo3 <= "--";
58
59    elsif ((enable1 = '0' or enable1 = '1') and enable3 = '1') then --passa A3
60        control_SW1 <= A3(5) & A3(1);
61        B3 <= A3(3 downto 2);
62        --aggiorno destinazioni
63        dest_nodo1 <= "--";
64        dest_nodo3 <= A3(1 downto 0);
65
66    end if;
67  end process;
68
69
70
71  control_stadio2: process (dest_nodo0, dest_nodo1, dest_nodo2, dest_nodo3, A3)
72  begin
73
74
75      --controllo SW2
76    if (dest_nodo3 = "00" or dest_nodo3 = "01") then
77
78        control_SW2 <= A3(4) & A3(0);-- passa A3
79
80    elsif ((dest nodo3 /= "00" and dest nodo3 /= "01") and
81           (dest_nodo2 = "00" or dest_nodo2 = "01")) then
82
83        control_SW2 <= A2(4) & A2(0);-- passa A2
84
85    elsif ((dest_nodo3 = "--") and
86           (dest_nodo2 /= "00" or dest_nodo2 = "01") and
87           (dest_nodo1 = "00" or dest_nodo1 = "01")) then
88
89        control_SW2 <= A1(4) & A1(0);-- passa A1
90
91    elsif ((dest_nodo3 /= "00" or dest_nodo2 = "01") and
92           (dest_nodo2 = "--") and
93           (dest_nodo1 /= "00" or dest_nodo1 /= "01")and
94           (dest_nodo0 = "00" or dest_nodo0 = "01"))then
95
96        control_SW2 <= A0(4) & A0(0);-- passa A0
97    end if;
98
99
100
101   --controllo SW3
102   if (dest_nodo3 = "10" or dest_nodo3 = "11") then
103
104       control_SW3 <= A3(4) & A3(0);-- passa A3
105
```

```
106         elsif ((dest_nodo3 /= "10" and dest_nodo3 /= "11") and
107                (dest_nodo2 = "10" or dest_nodo2 = "11")) then
108 
109             control_SW3 <= A2(4) & A2(0);-- passa A2
110 
111         elsif ((dest_nodo3 = "--") and
112                (dest_nodo2 /= "10" or dest_nodo2 = "11") and
113                (dest_nodo1 = "10" or dest_nodo1 = "11")) then
114 
115             control_SW3 <= A1(4) & A1(0);-- passa A1
116 
117         elsif ((dest_nodo3 /= "10" or dest_nodo2 = "11") and
118                (dest_nodo2 = "--") and
119                (dest_nodo1 /= "10" or dest_nodo1 /= "11") and
120                (dest_nodo0 = "10" or dest_nodo0 = "11"))then
121 
122             control_SW3 <= A0(4) & A0(0);-- passa A0
123         end if;
124 
125 
126     end process;
```

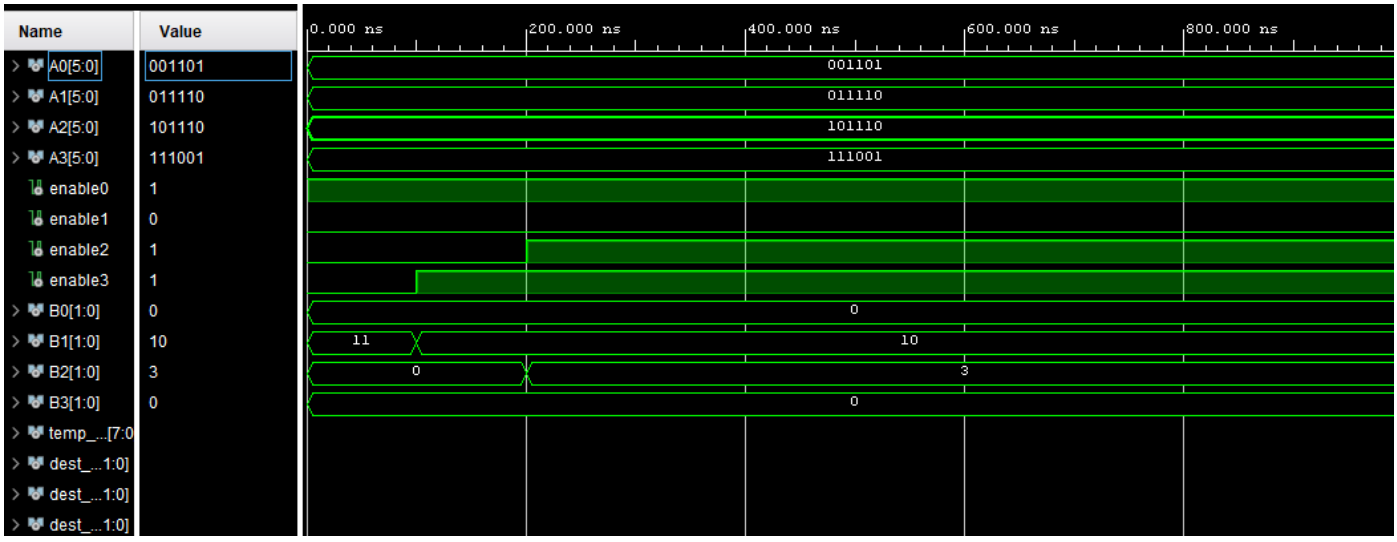
Simulazione

Per simulare il sistema abbiamo implementato il seguente testbench, istanziando l'unità da testare, cioè il sistema complessivo, definendo la forma d'onda del clock e diversi input da testare.

```
7  entity system_tb is
8  end system_tb;
9
10 architecture Behavioral of system_tb is
11
12     component system
13         Port (
14             A0, A1, A2, A3 : in std_logic_vector(5 downto 0);
15             enable0, enable1, enable2, enable3 : in std_logic;
16             B0, B1, B2, B3 : out std_logic_vector(1 downto 0)
17         );
18     end component;
19
20     signal A0, A1, A2, A3 : std_logic_vector(5 downto 0) := (others => '0');
21     signal enable0, enable1, enable2, enable3 : std_logic := '0';
22     signal B0, B1, B2, B3 : std_logic_vector(1 downto 0);
23
24     signal clk : std_logic := '0';
25     constant clk_period : time := 10 ns;
26
27 begin
28
29     uut: system
30         Port map (
31             A0 => A0, A1 => A1, A2 => A2, A3 => A3,
32             enable0 => enable0, enable1 => enable1, enable2 => enable2, enable3 => enable3,
33             B0 => B0, B1 => B1, B2 => B2, B3 => B3
34         );
35
36     clk_process: process
37     begin
38         while true loop
39             clk <= '0';
40             wait for clk_period/2;
41             clk <= '1';
42             wait for clk_period/2;
43         end loop;
44     end process;
45
46     stim_proc: process
47     begin
48
49         A0 <= "001101"; A1 <= "011110"; A2 <= "101110"; A3 <= "111001";
50         enable0 <= '1'; enable1 <= '0'; enable2 <= '0'; enable3 <= '0';
51         wait for 100 ns;
52
53         enable0 <= '1'; enable1 <= '0'; enable2 <= '0'; enable3 <= '1';
54         wait for 100 ns;
55
56         enable0 <= '1'; enable1 <= '0'; enable2 <= '1'; enable3 <= '1';
```

```
57          wait for 100 ns;
58
59          wait;
60      end process;
61
62  end Behavioral;
```

Riportiamo quindi i risultati della simulazione.



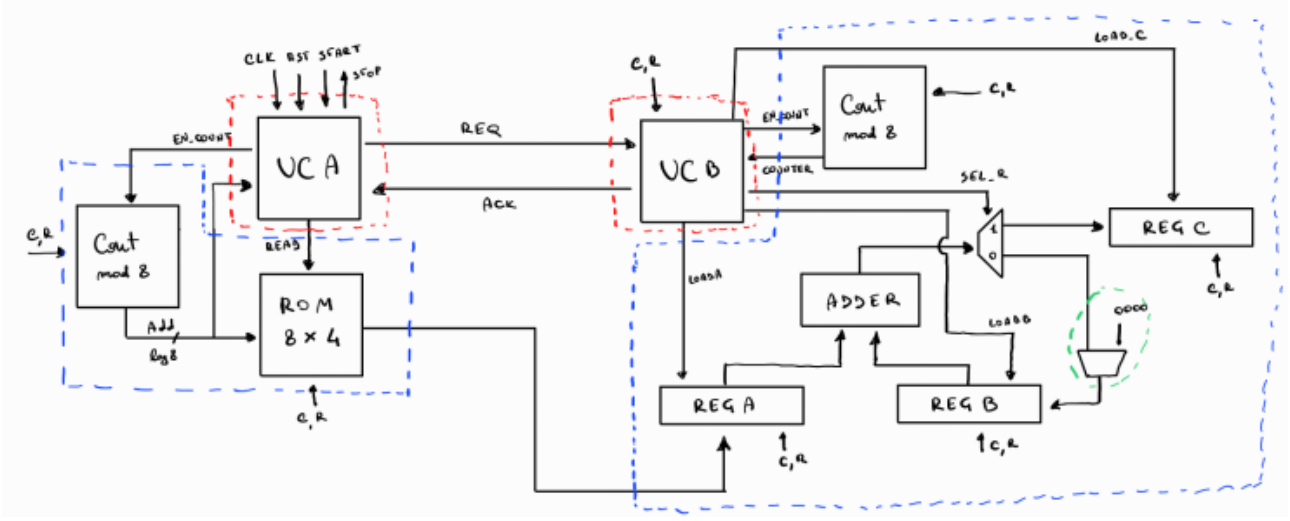
Capitolo 8: Prova Dicembre 2024

Esercizio 12 (prova di esame del 19 dicembre 2024)

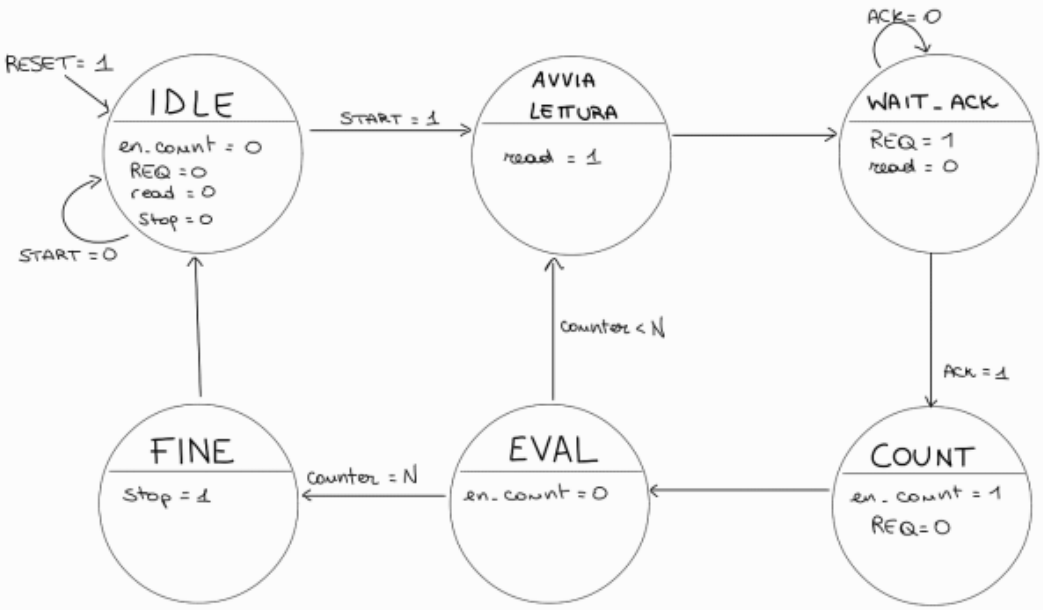
Un sistema è composto da 2 nodi, A e B. A include una ROM (progettata come macchina sequenziale con READ sincrono) di 8 locazioni da 4 bit, mentre B include un sommatore parallelo in grado di effettuare la somma di 2 stringhe di 4 bit ciascuna e un registro R di 4 bit. Il sistema opera come segue: all'arrivo di un segnale di start, A inizia a prelevare gli elementi ROM[i] dalla propria memoria e li invia, uno alla volta, a B mediante handshaking. B somma progressivamente le stringhe ricevute utilizzando il sommatore e alla fine inserisce il risultato nel registro R.
1. Si disegni l'architettura complessiva del sistema tramite un diagramma a blocchi, identificando parte operativa e parte di controllo di ciascun nodo. Ogni nodo deve essere progettato seguendo un approccio strutturale, individuando tutti i componenti, le loro interfacce e le loro interconnessioni.
2. Si progettino le unità di controllo di A e B evidenziando gli stati, gli ingressi e le uscite negli automi risultanti. E' obbligatorio specificare la tempificazione che si intende dare alle macchine (fronte attivo del clock, tempificazione dei segnali di READ/WRITE su registri e memorie).
3. Si progetti il sommatore secondo un'architettura di tipo carry look ahead.
4. Si fornisca l'implementazione in VHDL dell'intero sistema e si proceda alla simulazione nel caso in cui il clock del sistema A e del sistema B siano diversi (A più lento e A più veloce).

Progetto e architettura

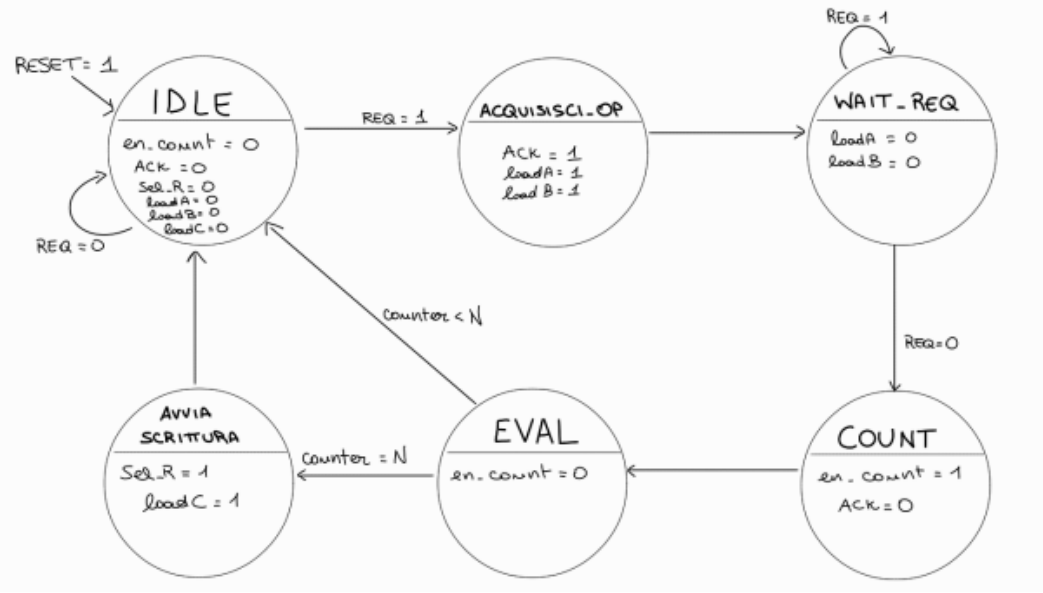
Il sistema vede l'interazione di due 2 A e B, progettati ciascuno tramite l'interazione di un'unità operativa e un'unità di controllo. È stato implementato un protocollo di handshaking semplice, supponendo che i clock dei due nodi abbiano pari frequenza (oppure che quello del nodo B abbia una frequenza maggiore) e si è scelto il fronte di salita come fronte attivo del clock.



A :

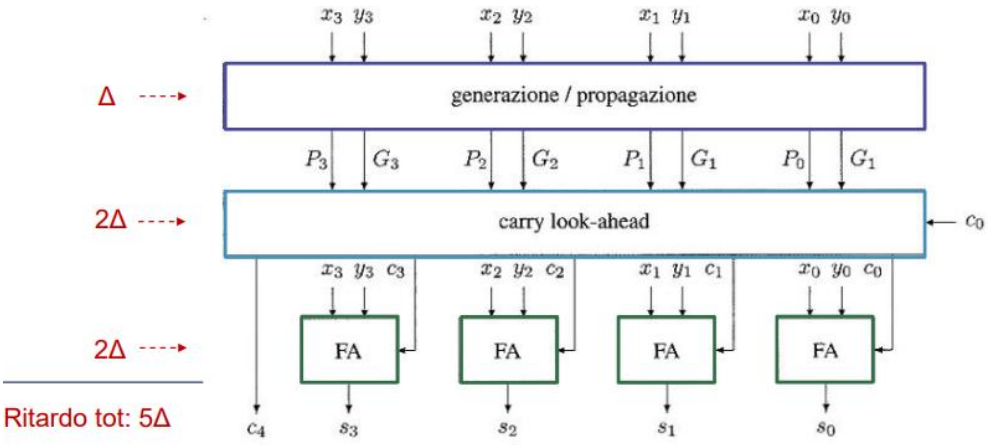

RESET = 1
IDLE
en_count = 0
REQ = 0
read = 0
Stop = 0
START = 0
START = 1
AVVIA LETTURA
read = 1
ACK = 0
WAIT_ACK
REQ = 1
read = 0
ACK = 1
COUNT
en_count = 1
REQ = 0
counter < N
EVAL
en_count = 0
counter = N
FINE
Stop = 1

B :

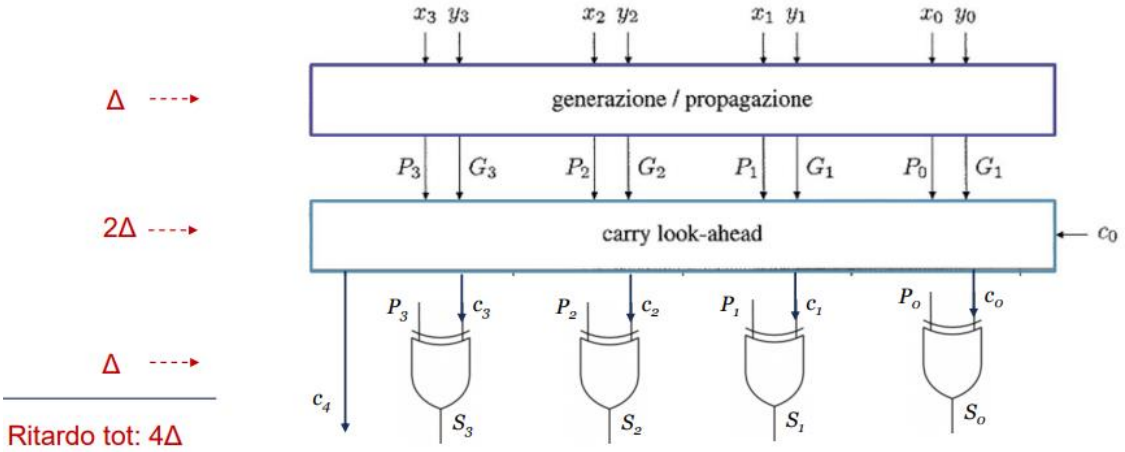

RESET = 1
IDLE
en_count = 0
ACK = 0
Sel_R = 0
loadA = 0
loadB = 0
loadC = 0
REQ = 0
REQ = 1
ACQUISISCI_OP
ACK = 1
loadA = 1
loadB = 1
REQ = 1
WAIT_REQ
loadA = 0
loadB = 0
REQ = 0
COUNT
en_count = 1
ACK = 0
counter < N
EVAL
en_count = 0
counter = N
AVVIA SCRITTURA
Sel_R = 1
loadC = 1

Come richiesto dalla traccia, analizziamo il sommatore **Carry-Look-Ahead**.

Esso strutta le funzioni di generazione e propagazione del riporto, che coincidono rispettivamente con il prodotto (OR logica) e la somma (AND logica) degli operandi.



Il ritardo complessivo può essere ridotto a 4 tempi elementari, sfruttando la relazione di somma pari alla XOR tra la funzione di propagazione e del riporto.



Questa sarà la strada che seguiremo in fase di implementazione.

Implementazione

Il nodo A è suddiviso in parte di controllo e parte operativa. Quest'ultima è composta da una ROM con 8 locazioni, progettata come una macchina sequenziale con segnale di *read* sincrono, e un contatore per indirizzarla.

```
8  entity NodoA is
9      Generic (
10         N : integer := 16; -- Numero di locazioni di memoria (modificabile)
11         M : integer := 8   -- Larghezza in bit di ogni locazione
12     );
13     Port (
14         CLK:        in std_logic;
15         RST:        in std_logic;
16         Start:      in std_logic;
17         ACK:        in std_logic;
18         stop:       out std_logic;
19         REQ:        out std_logic;
20         dataOUT :   out std_logic_vector(M-1 downto 0)
21      );
22 end NodoA;
23
24 architecture structuralNodoA of NodoA is
25
26     component PO_A is
27         Generic (
28             N : integer := 64; -- Numero di locazioni di memoria
29             M : integer := 8   -- Larghezza in bit di ogni locazione
30         );
31         Port (
32             clk, rst : in std_logic;
33             en_count, read : in std_logic;
34             dataOUT : out std_logic_vector(M-1 downto 0);
35             counter: out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0)
36         );
37     end component;
38
39     component UC_A is
40         Generic (
41             N : integer := 64 -- Numero di locazioni di memoria
42         );
43         Port(
44             clk, rst, start, ACK : in std_logic;
45             counter : in std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0);
46             en_count, read, REQ, stop: out std_logic
47         );
48     end component;
49
50     signal Read_temp : std_logic := '0';
51     signal counter_temp : std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0):= (others => '0');
52     signal en_count_temp : std_logic := '0';
53
54
55 begin
56
57     Parte_operativa: PO_A
```

```
        generic map (
            N => N,
            M => M
        )
        port map(
            CLK => CLK,
            RST => RST,
            en_count => en_count_temp,
            Read => Read_temp,
            dataOUT => dataOUT,
            counter => counter_temp
        );

    Unita_controllo: UC_A
        generic map (
            N => N
        )
        port map(
            CLK => CLK,
            RST => RST,
            Start => Start,
            ACK => ACK,
            counter => counter_temp,
            en_count => en_count_temp,
            Read => Read_temp,

            REQ => REQ,
            stop => stop
        );

end ;
```

Vediamo nel dettaglio l'implementazione della parte operativa di A.

```
entity PO_A is
    Generic (
        N : integer := 64; -- Numero di locazioni di memoria
        M : integer := 8   -- Larghezza in bit di ogni locazione
    );
    Port (
        clk, rst : in std_logic;
        en_count, read : in std_logic;
        dataOUT : out std_logic_vector(M-1 downto 0);
        counter: out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0)
    );
end PO_A;

architecture structural_A of PO_A is

    component ContMod_N is
        Generic (
            N : integer := 64 -- Modulo del contatore
        );
        Port (
            CLK:         in std_logic;  -- clock
            RST:         in std_logic;  -- reset asincrono
            en_count:    in std_logic; --segnale di conteggio
            dataOUT:     out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0) -- uscita di conteggio
        );
```

```
    end component;

    component ROM_N_M is
        Generic (
            N : integer := 64; -- Numero di locazioni di memoria
            M : integer := 8   -- Larghezza in bit di ogni locazione
        );
        Port (
            CLK     : in std_logic;
            Address : in std_logic_vector (integer(ceil(log2(real(N))))-1 downto 0);
            Read    : in std_logic;
            dataOUT : out std_logic_vector (M-1 downto 0)
        );
    end component;

    signal Address_temp: std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0):= (others => '0');

    begin

    counter <= Address_temp;

    --COUNTER
    ContModN_A : ContMod_N
         Generic map(
            N => N

         )
        Port map(
            clk => clk,
            rst => rst,
            en_count => en_count,
            dataOUT => Address_temp
        );

    ROM_A : ROM_N_M
        Generic map(
            N => N,
            M => M
        )
        Port map(
            CLK => clk,
            Address => Address_temp,
            Read => read,
            dataOUT => dataOUT
        );

end Structural_A;
```

La parte di controllo, invece, implementa l'automa riportato nella fase di progettazione.

```
entity UC_A is
    Generic (
        N : integer := 64 -- Numero di locazioni di memoria
    );
    Port(
        clk, rst, start, ACK : in std_logic;
        counter : in std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0);
        en_count, read, REQ, stop: out std_logic
     );
end UC_A;

architecture Behavioral of UC_A is
    type stato is (IDLE, AVVIA_LETTURA, WAIT_ACK, COUNT, EVAL, FINE);
    signal stato_corrente : stato := IDLE;
    signal stato_prossimo : stato := stato_corrente;

    begin

    mem_stato: process (clk, rst)
    begin
        if (rising_edge(clk)) then
            if (rst = '1') then
                stato_corrente <= idle;
            else
                stato_corrente <= stato_prossimo;
```

```
            end if;
        end if;
    end process;

    comb: process (stato_corrente, start, counter, ACK)
    begin

        case stato_corrente is
            when IDLE =>
                en_count <= '0';
                REQ <= '0';
                read <= '0';
                stop <= '0';

                if (start='0') then
                    stato_prossimo <= IDLE;

                elsif (start='1') then
                    stato_prossimo <= AVVIA_LETTURA;
                end if;

            when AVVIA_LETTURA  =>
                read <= '1';
                stato_prossimo <= WAIT_ACK;

            when WAIT_ACK =>
                REQ <= '1';
                read <= '0';

                if (ACK = '0') then
                    stato_prossimo <= WAIT_ACK;

                elsif (ACK = '1') then
                    stato_prossimo <= COUNT;
                end if;

            when COUNT =>
                en_count <= '1';
                REQ <= '0';
                stato_prossimo <= EVAL;

            when EVAL =>
                en_count <= '0';
                if (counter = std_logic_vector(to_unsigned(0, counter'length)))then
                    stato_prossimo <= FINE;
                else
                    stato_prossimo <= AVVIA_LETTURA;
                end if;

            when FINE =>
                stop <= '1';
                stato_prossimo <= IDLE;

            when OTHERS =>
                stato_prossimo <= IDLE;

        end case;
    end process;

end Behavioral;
```

Anche l'implementazione del nodo B è suddivisa in parte di controllo e parte operativa. Quest'ultima, in questo caso, comprende tre registri A, B, C, un addizionatore, un contatore e un demultiplexer 1:2.

```
8  entity NodoB is
9      Generic (
10         N : integer := 16; -- Numero di locazioni di memoria (modificabile)
11         M : integer := 8   -- Larghezza in bit di ogni locazione
12     );
13     Port (
14         CLK:        in std_logic;
15         RST:        in std_logic;
16         REQ:        in std_logic;
17         dataIN :    in std_logic_vector(M-1 downto 0);
18         ACK:        out std_logic
19
20      );
21 end NodoB;
22
23 architecture structuralNodoB of NodoB is
24
25     component PO_B is
26         Generic (
27             N : integer := 64; -- Numero di locazioni di memoria
28             M : integer := 8   -- Larghezza in bit di ogni locazione
29         );
30         Port (
31             clk, rst : in std_logic;
32             en_count, loadA, loadB, loadC, Sel_R : in std_logic;

33             dataIN : in std_logic_vector(M-1 downto 0);
34             counter: out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0)
35         );
36     end component;
37
38     component UC_B is
39         Generic (
40             N : integer := 64 -- Numero di locazioni di memoria
41         );
42         Port(
43             clk, rst, REQ : in std_logic;
44             counter : in std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0);
45             en_count, Sel_R, loadA, loadB, loadC, ACK: out std_logic
46         );
47     end component;
48
49     signal Sel_R_temp : std_logic := '0';
50     signal counter_temp : std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0):= (others => '0');
51     signal en_count_temp : std_logic := '0';
52     signal loadA_temp : std_logic := '0';
53     signal loadB_temp : std_logic := '0';
54     signal loadC_temp : std_logic := '0';
```

```
begin

    Parte_operativa: PO_B
        generic map (
            N => N,
            M => M
        )
        port map(
            CLK => CLK,
            RST => RST,
            en_count => en_count_temp,
            loadA => loadA_temp,
            loadB => loadB_temp,
            loadC => loadC_temp,
            Sel_R => Sel_R_temp,
            dataIN => dataIN,
            counter => counter_temp
        );

    Unita_controllo: UC_B
        generic map (
            N => N
        )
        port map(
            CLK => CLK,
            RST => RST,
            REQ => REQ,
            counter => counter_temp,
            en_count => en_count_temp,
            Sel_R => Sel_R_temp,
            loadA => loadA_temp,
            loadB => loadB_temp,
            loadC => loadC_temp,
            ACK => ACK
        );

end ;
```

E di seguito l'unità operativa di B.

```
entity PO_B is
    Generic (
        N : integer := 64; -- Numero di locazioni di memoria
        M : integer := 8   -- Larghezza in bit di ogni locazione
    );
    Port (
        clk, rst : in std_logic;
        en_count, loadA, loadB, loadC, Sel_R : in std_logic;
        dataIN : in std_logic_vector(M-1 downto 0);
        counter: out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0)
    );
end PO_B;

architecture structural_B of PO_B is

    component ContMod_N is
        Generic (
            N : integer := 64 -- Modulo del contatore
        );
        Port (
            CLK:         in std_logic;  -- clock
            RST:         in std_logic;  -- reset asincrono
            en_count:    in std_logic; --segnale di conteggio
            dataOUT:     out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0) -- uscita di conteggio
        );
```

```
32      end component;
33
34      component Adder_M is
35          Generic (
36              M : integer := 8  -- Numero di bit (default: 8)
37          );
38          Port (
39              A  : in  std_logic_vector(M-1 downto 0);
40              B   : in  std_logic_vector(2*M-1 downto 0);
41              SUM    : out std_logic_vector(2*M-1 downto 0)  -- Somma M-bit
42          );
43      end component;
44
45      component REG_M is
46          Generic (
47              M : integer := 8
48          );  -- Parametro per la larghezza del registro
49          Port (
50              dataIN  : in std_logic_vector(M-1 downto 0);
51              clk, rst, load : in std_logic;
52              dataOUT : out std_logic_vector(M-1 downto 0)
53          );
54      end component;
55
56      component DemuxM_1_2 is

57          generic (M : integer range 0 to 32 := 8);
58          port(
59              dataIN : in std_logic_vector(M-1 downto 0);  -- Ingresso unico
60              s : in std_logic;                            -- Selettore
61              dataOUT0, dataOUT1 : out std_logic_vector(M-1 downto 0) -- Uscite
62          );
63      end component;
64
65      signal temp_REG_C_in: std_logic_vector(2*M-1 downto 0):= (others => '0');
66      signal temp_REG_B_in: std_logic_vector(2*M-1 downto 0):= (others => '0');
67      signal temp_MEM_out: std_logic_vector(M-1 downto 0):= (others => '0');
68      signal temp_REG_A_out: std_logic_vector(M-1 downto 0):= (others => '0');
69      signal temp_REG_B_out: std_logic_vector(2*M-1 downto 0):= (others => '0');
70      signal temp_REG_C_out: std_logic_vector(2*M-1 downto 0):= (others => '0');
71      signal temp_adder_out: std_logic_vector(2*M-1 downto 0):= (others => '0');
72
73      begin
74
75      ContModN_B : ContMod_N
76           Generic map(
77              N => N
78           )
79          Port map(
80              clk => clk,
81              rst => rst,

82              en_count => en_count,
83              dataOUT => counter
84          );
85
86      Adder_B : Adder_M
87          Generic map(
88              M => M
89          )
90          Port map(
91              A => temp_REG_A_out,
92              B => temp_REG_B_out,
93              SUM => temp_adder_out
94          );
95
96      REG_A : REG_M
97          Generic map (
98              M => M
99          )
100         Port map(
101             dataIN => dataIN,
102             clk => clk,
103             rst => rst,
104             load => loadA,
105             dataOUT => temp_REG_A_out
106         );
```

```
108     REG_B : REG_M
109         Generic map (
110             M => 2*M
111         )
112         Port map(
113             dataIN => temp_REG_B_in,
114             clk => clk,
115             rst => rst,
116             load => loadB,
117             dataOUT => temp_REG_B_out
118         );
119 
120     REG_C : REG_M
121         Generic map (
122             M => 2*M
123         )
124         Port map(
125             dataIN => temp_REG_C_in,
126             clk => clk,
127             rst => rst,
128             load => loadC,
129             dataOUT => temp_REG_C_out
130         );
131 
132     Demux_B : DemuxM_1_2

133         Generic map (
134             M => 2*M
135         )
136         Port Map(
137             dataIN => temp_adder_out,
138             s => Sel_R,
139             dataOUT0 => temp_REG_B_in,
140             dataOUT1 => temp_REG_C_in
141         );
142 
143 
144 
145 end Structural_B;
```

La parte di controllo, invece, implementa l'automa riportato nella fase di progettazione.

```
 7 entity UC_B is
 8     Generic (
 9         N : integer := 64 -- Numero di locazioni di memoria
10     );
11     Port(
12         clk, rst, REQ : in std_logic;
13         counter : in std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0);
14         en_count, Sel_R, loadA, loadB, loadC, ACK: out std_logic
15      );
16 end UC_B;
17 
18 architecture Behavioral of UC_B is
19     type stato is (IDLE, ACQUISISCI_OP, WAIT_REQ, COUNT, EVAL, AVVIA_SCRITTURA, WAIT_SCRITTURA);
20     signal stato_corrente : stato := IDLE;
21     signal stato_prossimo : stato := stato_corrente;
22 
23     begin
24 
25     mem_stato: process (clk, rst)
26     begin
27         if (rising_edge(clk)) then
28             if (rst = '1') then
29                 stato_corrente <= idle;
30             else
31                 stato_corrente <= stato_prossimo;
```

```
            end if;
        end if;
    end process;

    comb: process (stato_corrente, counter, REQ)
    begin

        case stato_corrente is
            when IDLE =>
                en_count <= '0';
                ACK <= '0';
                Sel_R <= '0';
                loadA <= '0';
                loadB <= '0';
                loadC <= '0';

                if (REQ = '0') then
                    stato_prossimo <= IDLE;

                elsif (REQ = '1') then
                    stato_prossimo <= ACQUISISCI_OP;
                end if;

            when ACQUISISCI_OP =>
                ACK <= '1';

                loadA <= '1';
                loadB <= '1';
                stato_prossimo <= WAIT_REQ;

            when WAIT_REQ =>
                loadA <= '0';
                loadB <= '0';

                if (REQ = '1') then
                    stato_prossimo <= WAIT_REQ;

                elsif (REQ = '0') then
                    stato_prossimo <= COUNT;
                end if;

            when COUNT =>
                en_count <= '1';
                ACK <= '0';
                stato_prossimo <= EVAL;

            when EVAL =>
                en_count <= '0';
                if (counter = std_logic_vector(to_unsigned(0, counter'length)))then
                    stato_prossimo <= AVVIA_SCRITTURA;
                else

                     stato_prossimo <= IDLE;
                 end if;

             when AVVIA_SCRITTURA =>
                 Sel_R <= '1';
                 loadC <= '1';
                 stato_prossimo <= IDLE;

             when OTHERS =>
                 stato_prossimo <= IDLE;

         end case;
     end process;

end Behavioral;
```

Il sistema complessivo è realizzato in maniera strutturale come nel seguente codice, sfruttando tre *signal* per REQ, ACK e per i dati trasmessi da A a B.

```
entity sistema is
    Generic (
        N : integer := 16; -- Numero di locazioni di memoria (modificabile)
        M : integer := 8   -- Larghezza in bit di ogni locazione
    );
    Port (
        CLK_A:      in std_logic;
        CLK_B:      in std_logic;
        RST:        in std_logic;
        Start:      in std_logic;
        stop:       out std_logic
     );
end sistema;

architecture structural_sistema of sistema is

    component NodoA is
        Generic (
            N : integer := 16; -- Numero di locazioni di memoria (modificabile)
            M : integer := 8   -- Larghezza in bit di ogni locazione
        );
        Port (
            CLK:        in std_logic;
            RST:        in std_logic;
            Start:      in std_logic;

            ACK:        in std_logic;
            stop:       out std_logic;
            REQ:        out std_logic;
            dataOUT :   out std_logic_vector(M-1 downto 0)
        );
    end component;

    component NodoB is
        Generic (
            N : integer := 16; -- Numero di locazioni di memoria (modificabile)
            M : integer := 8   -- Larghezza in bit di ogni locazione
        );
        Port (
            CLK:        in std_logic;
            RST:        in std_logic;
            REQ:        in std_logic;
            dataIN :    in std_logic_vector(M-1 downto 0);
            ACK:        out std_logic
        );
    end component;

    signal REQ_temp : std_logic := '0';
    signal ACK_temp : std_logic := '0';
    signal data_temp : std_logic_vector(M-1 downto 0) := (others => '0');

begin

    NODO_A: NodoA
        generic map (
            N => N,
            M => M
        )
        port map(
            CLK => CLK_A,
            RST => RST,
            start => start,
            ACK => ACK_temp,
            stop => stop,
            REQ => REQ_temp,
            dataOUT => data_temp
        );

     NODO_B: NodoB
        generic map (
            N => N,
            M => M
        )
        port map(
            CLK => CLK_B,
            RST => RST,
```

```
            REQ => REQ_temp,
            dataIN => data_temp,
            ACK => ACK_temp

        );

end ;
```

Per il punto 3), abbiamo implementato in modo behavioral un sommatore carry-look-ahead, la cui architettura è riportata nel paragrafo di progettazione.

```
entity carry_look_ahead is
    Port (
        A, B : in STD_LOGIC_VECTOR(3 downto 0); -- operandi
        Cin  : in STD_LOGIC;
        Sum  : out STD_LOGIC_VECTOR(3 downto 0);
        Cout : out STD_LOGIC
    );
end carry_look_ahead;

architecture Behavioral of carry_look_ahead is
    signal P, G   : STD_LOGIC_VECTOR(3 downto 0); -- Propagate & Generate
    signal C      : STD_LOGIC_VECTOR(4 downto 0); -- Carry signals

begin
    C(0) <= Cin;

    -- Calcolo generazione e propagazione
    G <= A and B;  -- Generate: G(i) = A(i) * B(i)
    P <= A xor B;  -- Propagate: P(i) = A(i) ⊕ B(i)

    -- Calcolo riporto
    C(1) <= G(0) or (P(0) and C(0));
    C(2) <= G(1) or (P(1) and C(1));
    C(3) <= G(2) or (P(2) and C(2));
    C(4) <= G(3) or (P(3) and C(3));

    Sum <= P xor C(3 downto 0);

    Cout <= C(4);
end Behavioral;
```

Simulazione

Segue il testbench implementato per la simulazione del sistema.

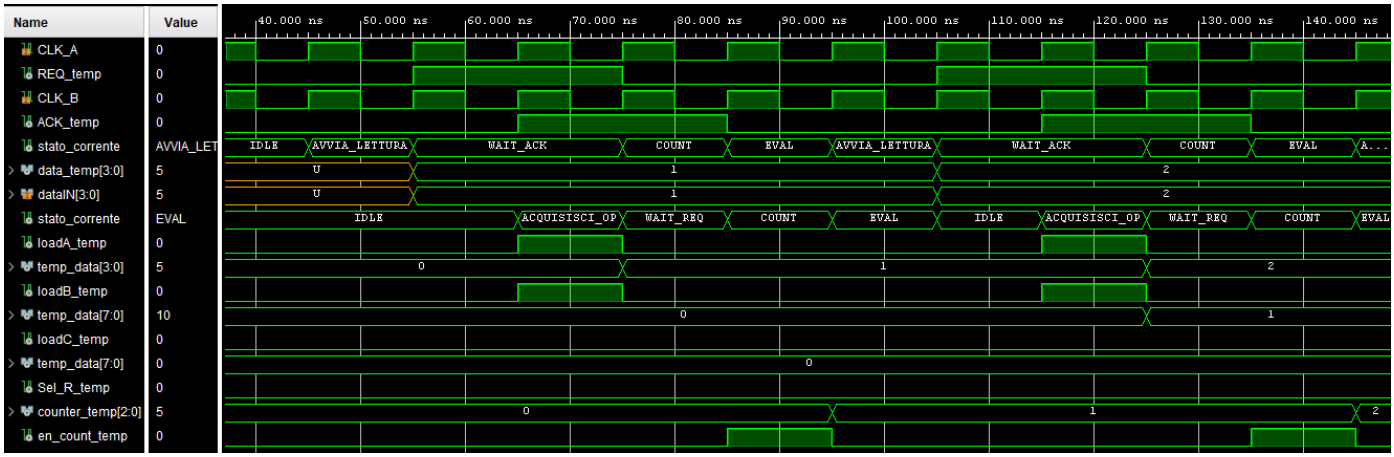
```
5  entity testbench_sistema is
6  end testbench_sistema;
7
8  architecture tb of testbench_sistema is
9
10     constant CLK_PERIOD : time := 10 ns; -- Periodo del clock
11     signal CLK_A   : std_logic := '0';
12     signal CLK_B   : std_logic := '0';
13     signal RST     : std_logic := '0';
14     signal Start : std_logic := '0';
15     signal stop  : std_logic := '0';
16
17     -- Instanza del UUT (Unit Under Test)
18     component sistema
19         Generic (
20             N : integer := 8;
21             M : integer := 4
22         );
23         Port (
24             CLK_A   : in std_logic;
25             CLK_B   : in std_logic;
26             RST     : in std_logic;
27             Start : in std_logic;
28             stop  : out std_logic
29         );

30     end component;
31
32 begin
33
34     -- Instanza del sistema
35     UUT: sistema
36         generic map (
37             N => 8,
38             M => 4
39         )
40         port map (
41             CLK_A   => CLK_A,
42             CLK_B   => CLK_B,
43             RST     => RST,
44             Start => Start,
45             stop  => stop
46         );
47
48     -- Processo per generare il clock
49     process
50     begin
51         while now < 3000 ns loop -- Simulazione per 1000 ns
52             CLK_A <= '0';
53             wait for CLK_PERIOD/2;
54             CLK_A <= '1';
```

```
55              wait for CLK_PERIOD/2;
56          end loop;
57          wait;
58      end process;
59
60      process
61      begin
62          while now < 3000 ns loop -- Simulazione per 1000 ns
63              CLK_B <= '0';
64              wait for CLK_PERIOD/2;
65              CLK_B <= '1';
66              wait for CLK_PERIOD/2;
67          end loop;
68          wait;
69      end process;
70
71      -- Processo per generare i segnali di test
72      process
73      begin
74          -- Reset iniziale
75          RST <= '1';
76          wait for 20 ns;
77          RST <= '0';
78          wait for 20 ns;

80          -- Attivazione del segnale Start
81          Start <= '1';
82          wait for 30 ns;
83          Start <= '0';
84
85          -- Attendere il completamento dell'operazione
86          wait for 3000 ns;
87
88          -- Fine simulazione
89          wait;
90      end process;
91
92  end tb;
```

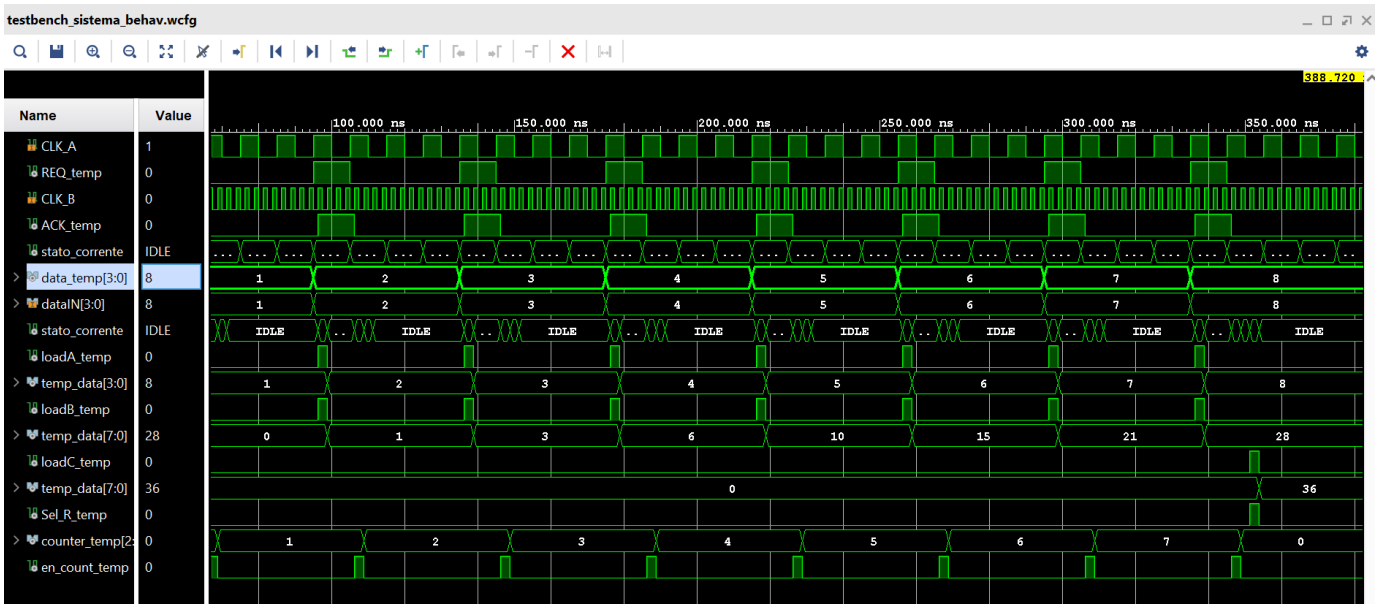
Da cui abbiamo ottenuto i seguenti risultati.



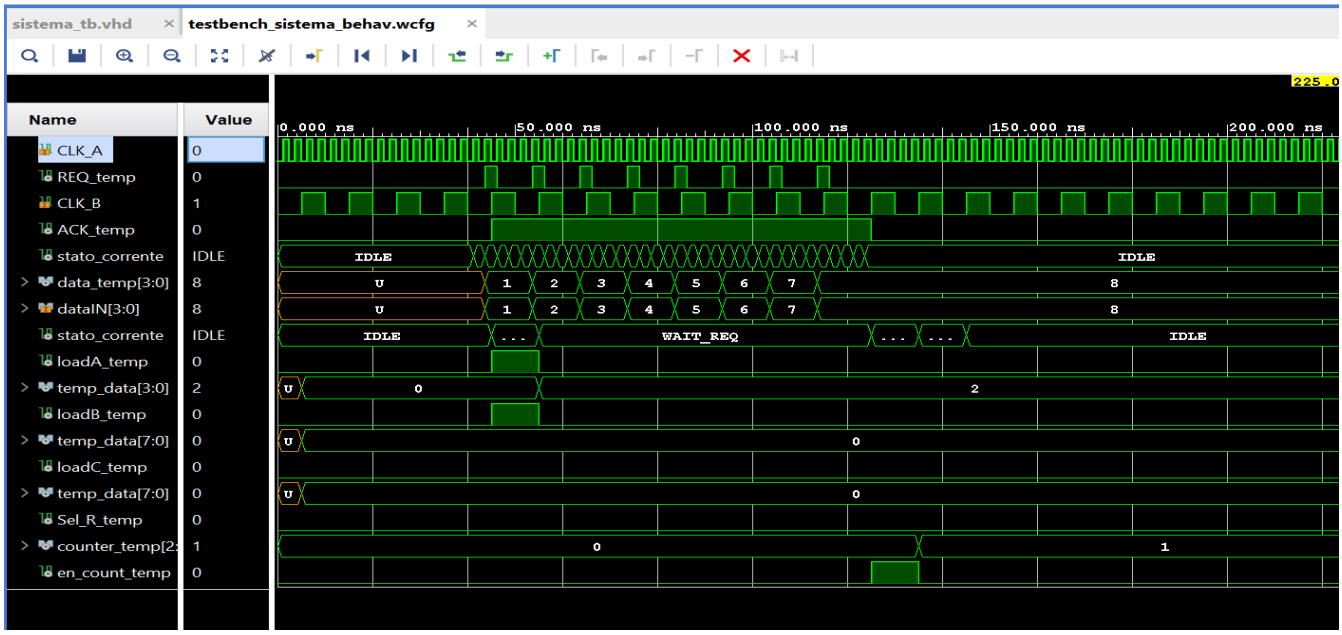


Di seguito riportiamo invece le simulazioni in cui i clock dei sistemi A e B siano diversi.

1. CLOCK A < CLOCK B (ricevitore più veloce)



2. CLOCK A > CLOCK B (trasmettitore più veloce)



Come si nota dalla simulazione l'handshaking viene effettuato correttamente tra i due sistemi esclusivamente nel caso in cui i clock dei sistemi siano uguali e nel caso in cui chi riceve abbia un clock più veloce.

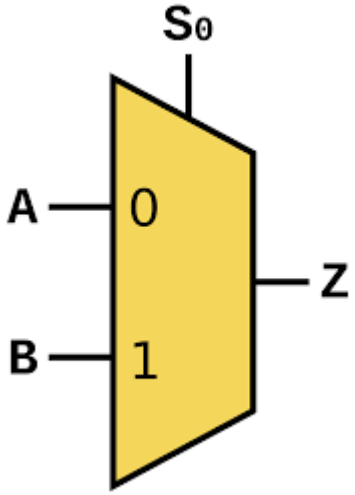
Il problema nel caso il trasmettitore è più veloce è il numero di richieste effettuate senza risposta.

Appendice

Multiplexer 2:1

Progetto e architettura

Un **multiplexer** (MUX) è un circuito logico combinatorio che seleziona uno tra diversi segnali di ingresso e lo instrada verso un'unica uscita, in base ai valori di specifiche linee di selezione. Ogni combinazione dei bit di selezione determina quale ingresso viene trasmesso all'uscita. I multiplexer sono fondamentali in elettronica digitale e telecomunicazioni, poiché permettono di ridurre il numero di linee di trasmissione, ottimizzando l'uso delle risorse hardware. Sono utilizzati in applicazioni come la commutazione di dati, la gestione della memoria nei microprocessori e la trasmissione di segnali in sistemi di comunicazione.



Implementazione

```
29  entity mux_2_1 is
30      Port (
31          a0  : in std_logic;
32          a1  : in std_logic;
33          s   : in std_logic;
34          y   : out std_logic
35      );
36  end mux_2_1;
37
38  architecture dataflow of mux_2_1 is
39      begin
40          y <= ((a0 AND (NOT s)) OR (a1 AND s));
41  end dataflow;
```

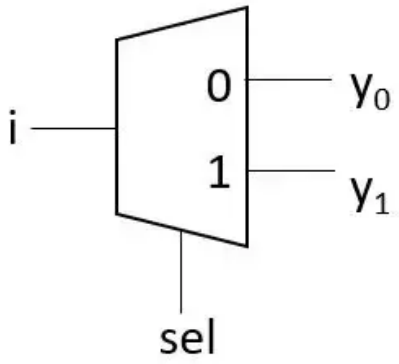
Demultiplexer 1:2

Progetto e architettura

Un **demultiplexer** è un circuito combinatorio che prende un singolo segnale di ingresso e lo inoltra a una delle diverse uscite selezionate da un insieme di linee di selezione. Un DEMUX indirizza un ingresso verso una delle sue uscite in base ai segnali di selezione.

Il DEMUX ha:

- 1 ingresso (D)
- n linee di selezione (S) per determinare l'uscita attiva.
- 2^n uscite (Y0, Y1, ... $Y(2^n-1)$)



1:2 DEMUX

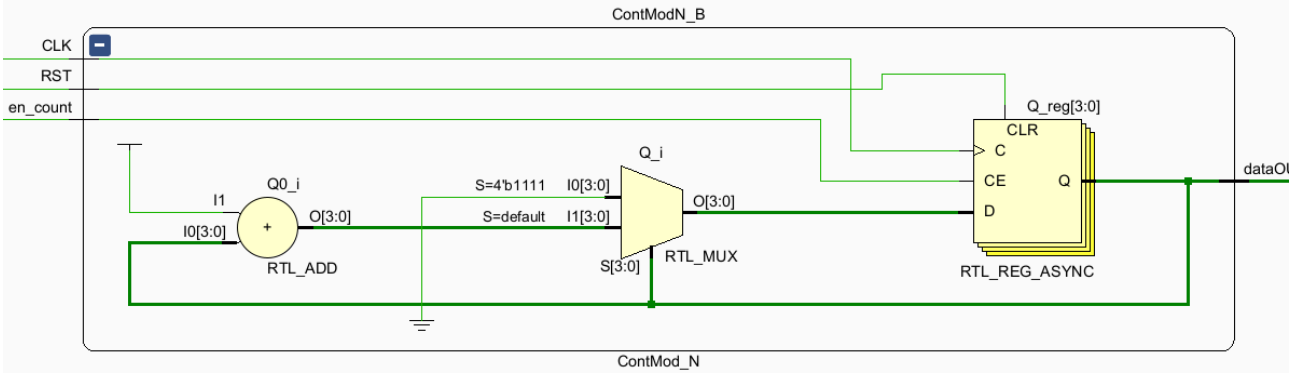
Implementazione

```
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity DemuxM_1_2 is
6      generic (M : integer range 0 to 32 := 8);
7      port(
8          dataIN : in std_logic_vector(M-1 downto 0);  -- Ingresso unico
9          s : in std_logic;                            -- Selettore
10         dataOUT0, dataOUT1 : out std_logic_vector(M-1 downto 0) -- Uscite
11     );
12 end DemuxM_1_2;
13
14 architecture rtl of DemuxM_1_2 is
15 begin
16     dataOUT0 <= dataIN when s = '0' else (others => '0');  -- Attivo su y0 se s=0
17     dataOUT1 <= dataIN when s = '1' else (others => '0');  -- Attivo su y1 se s=1
18 end rtl;
```

Contatore modulo N

Progetto e architettura

Il codice implementa un **contatore modulo N** in VHDL, che conta ciclicamente da 0 a N−1N-1N−1 e poi si azzera. L'entity ContMod_N ha un parametro generico N per impostare il modulo del conteggio e include tre segnali di ingresso: CLK (clock), RST (reset asincrono) e en_count (abilitazione del conteggio), oltre all'uscita dataOUT, un std_logic_vector con un numero di bit pari a ceil(log2(N)). L'architettura definisce il segnale Q, un intero che memorizza il valore corrente del contatore. Nel processo di conteggio, se RST è attivo ('1'), il contatore viene azzerato; al fronte di salita di CLK, se en_count è alto, Q viene incrementato e, se raggiunge N-1, torna a 0, realizzando così un conteggio ciclico modulo N. Infine, Q viene convertito in std_logic_vector per l'uscita dataOUT, permettendo di rappresentare il valore corrente del contatore con il numero minimo di bit necessari.



Implementazione

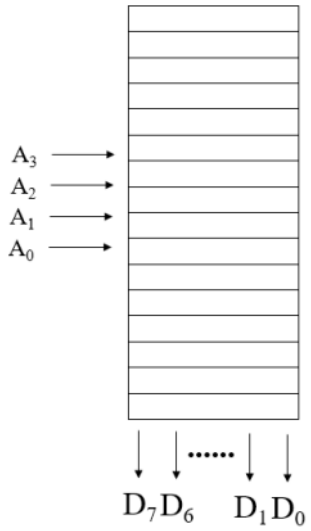
```
5  use IEEE.math_real.all;
6
7  entity ContMod_N is
8      Generic (
9          N : integer := 64 -- Modulo del contatore
10     );
11     Port (
12         CLK:         in std_logic;  -- clock
13         RST:         in std_logic;  -- reset asincrono
14         en_count:    in std_logic; --segnale di conteggio
15         dataOUT:     out std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0) -- uscita di conteggio
16     );
17 end ContMod_N;
18
19 architecture Behavioral of ContMod_N is
20     signal Q : integer range 0 to N-1 := 0;
21
22 begin
23     Counter : process(CLK, RST)
24     begin
25         if (RST = '1') then
26             Q <= 0;
27         elsif (rising_edge(CLK)) then
28             if (en_count = '1') then
29                 Q <= Q + 1;
30                 if(Q = N - 1) then
31                     Q <= 0;
32                 end if;
33             end if;
34         end if;
35     end process;
36
37     dataOUT <= std_logic_vector(to_unsigned(Q, integer(ceil(log2(real(N))))));
38 end Behavioral;
```

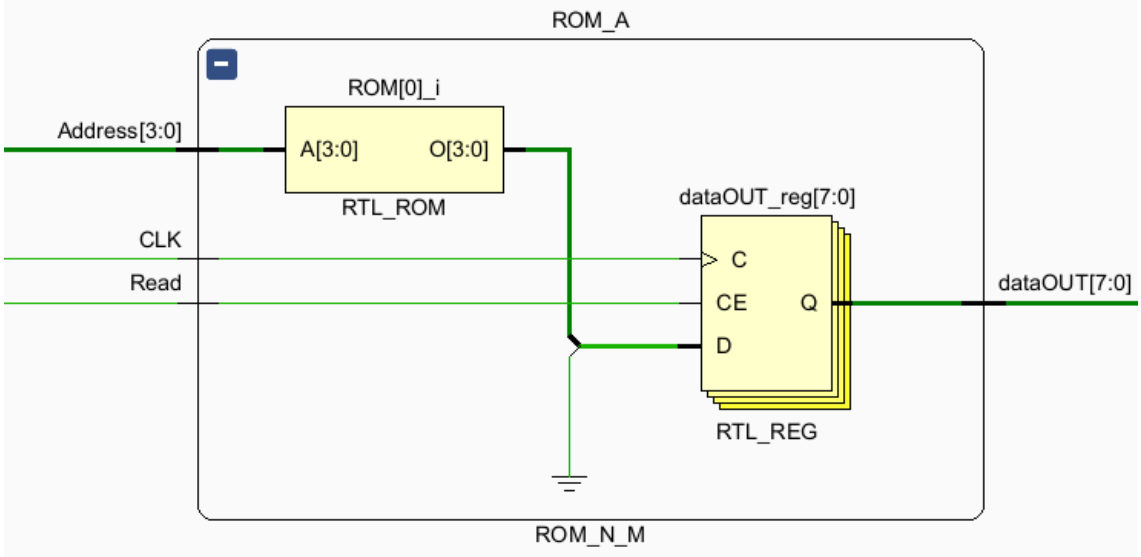
ROM

Progetto e architettura

Le memorie a sola lettura ROM (read-only memory) sono memorie non volatili, in cui i dati sono definiti una volta per tutte all'atto della realizzazione del circuito. La memoria conserva i suoi dati anche in assenza di alimentazione.

Una ROM possiede P linee di ingresso, che costituiscono l'indirizzo della memoria. Con P ingressi è possibile indirizzare 2P locazioni di memoria, ognuna delle quali memorizza un dato codificato su Q bit. Inserendo in ingresso uno specifico indirizzo, si ottiene in uscita il dato (la "parola") immagazzinato nella locazione di memoria selezionata.





Implementazione

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4  use IEEE.math_real.all;
5
6  entity ROM_N_M is
7      Generic (
8          N : integer := 64; -- Numero di locazioni di memoria
9          M : integer := 8   -- Larghezza in bit di ogni locazione
10     );
11     Port (
12         CLK     : in std_logic;
13         Address : in std_logic_vector (integer(ceil(log2(real(N))))-1 downto 0);
14         Read    : in std_logic;
15         dataOUT : out std_logic_vector (M-1 downto 0)
16     );
17 end ROM_N_M;
18
19 architecture Behavioral of ROM_N_M is
20
21     -- Definizione di una memoria con i valori da 1 a 8 inizializzati
22     type MEMORY_TYPE is array (0 TO N-1) of std_logic_vector(M-1 downto 0);
23
24     -- Inizializzazione della memoria con valori da 1 a 8
25     constant ROM : MEMORY_TYPE := (
26         0  => std_logic_vector(to_unsigned(1, M)),
27         1  => std_logic_vector(to_unsigned(2, M)),
28         2  => std_logic_vector(to_unsigned(3, M)),
29         3  => std_logic_vector(to_unsigned(4, M)),
30         4  => std_logic_vector(to_unsigned(5, M)),
31         5  => std_logic_vector(to_unsigned(6, M)),
32         6  => std_logic_vector(to_unsigned(7, M)),
33         7  => std_logic_vector(to_unsigned(8, M)),
34         others => (others => '0') -- Riempimento con zeri per le altre locazioni
35     );
37 begin
38
39     get_data : process (CLK)
40     begin
41         if rising_edge(CLK) then
42             if Read = '1' then
43                 dataOUT <= ROM(to_integer(unsigned(Address)));
44             end if;
45         end if;
46     end process get_data;
47
48 end Behavioral;
```

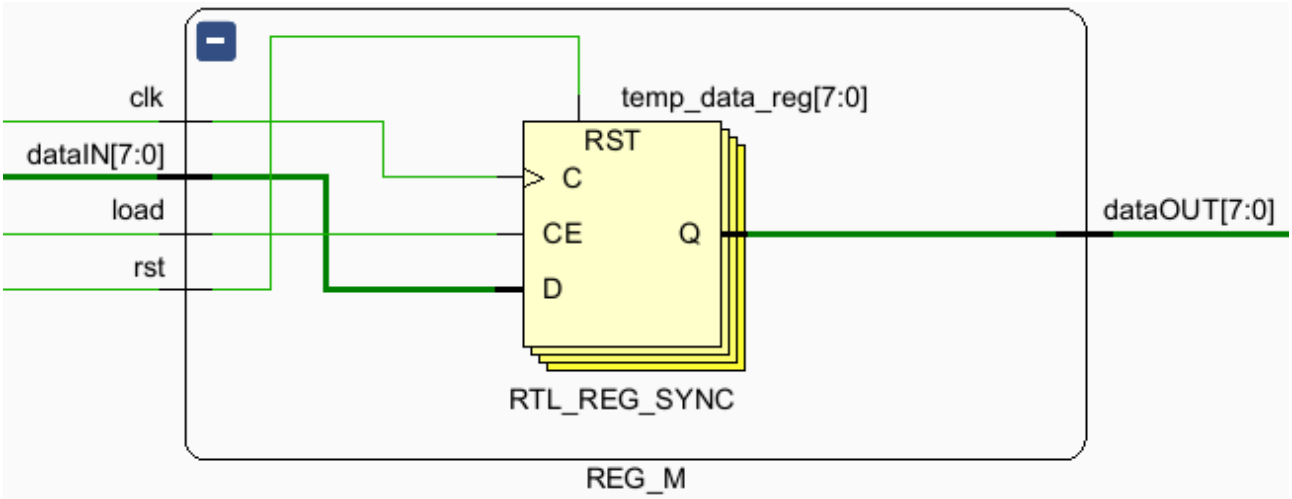
Registro

Progetto e architettura

Un registro parallelo-parallelo generico su M bit serve a **memorizzare e trasferire dati in parallelo** all'interno di un sistema digitale.

L'entity definisce tre ingressi di controllo: clk (clock), rst (reset sincrono) e load (caricamento dei dati), oltre a un ingresso dataIN e un'uscita dataOUT, entrambi di M bit. Nell'architettura, il segnale temp_data funge da registro interno per memorizzare i dati. Il processo R_PP è attivato sul **fronte di salita del clock (clk)**: se rst è attivo ('1'), il registro viene azzerato (temp_data è riempito con zeri), altrimenti, se load è alto ('1'), il valore presente su dataIN viene caricato in temp_data.

Infine, il contenuto di temp_data viene assegnato all'uscita dataOUT, aggiornando così il valore del registro. Questo registro è utile in sistemi digitali per la memorizzazione temporanea di dati e può essere utilizzato per sincronizzare segnali o memorizzare stati interni.



Implementazione

```
4  -- Registro parallelo-parallelo generico su M bit
5  entity REG_M is
6      Generic (M : integer := 8);  -- Parametro per la larghezza del registro
7      Port (
8          dataIN  : in std_logic_vector(M-1 downto 0);
9          clk, rst, load : in std_logic;
10         dataOUT : out std_logic_vector(M-1 downto 0)
11     );
12 end REG_M;
13
14 architecture behavioural of REG_M is
15     signal temp_data: std_logic_vector(M-1 downto 0);
16
17 begin
18     R_PP: process(clk)
19     begin
20         if rising_edge(clk) then
21             if rst = '1' then
22                 temp_data <= (others => '0');
23             elsif load = '1' then
24                 temp_data <= dataIN;
25             end if;
26         end if;
27     end process;
28
29     dataOUT <= temp_data;
30 end behavioural;
```